

Specification and Verification of Multi-clock Systems using a Temporal Logic with Clock Constraints

Journal:	<i>Formal Aspects of Computing</i>
Manuscript ID	FAC-2022-0033.R1
Manuscript Type:	Original Research Paper
Date Submitted by the Author:	n/a
Complete List of Authors:	Zhang, yuanrui; Southwest University, School of Mathematics and Statistics Mallet, Frederic; Universite Cote d'Azur Zhang, Min; East China Normal University Liu, Zhiming; Northwest University
Specialty/Area of Expertise:	Logic and verification, Modal and temporal logics, Verification by model checking

Under consideration for publication in Formal Aspects of Computing

Specification and Verification of Multi-clock Systems using a Temporal Logic with Clock Constraints

Yuanrui Zhang^{1,2}, Frédéric Mallet³, Min Zhang⁴ and Zhiming Liu^{2,5}

¹School of Mathematics and Statistics, Southwest University, China

²RISE Laboratory, College of Computer & Information Science, Southwest University, China

³University Cote d'Azur, CNRS, Inria, I3S, France

⁴Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, China

⁵Center for Intelligent and Embedded Software, Northwest Polytechnical University, China

Abstract. The polychronous or multi-clock paradigm is adequate to model large distributed systems where achieving a full timed synchronization is not only very costly, but also often not necessary. They consist of a set components with loose synchronization constraints. We study an approach where those components are orchestrated using *logical clocks*, made popular by L. Lamport and synchronous languages. The temporal and causal specification of those systems is built by defining a set of *clock relations* that would constrain when clocks can tick or must not tick, thus defining families of *valid schedules*. In this paper, we propose a specification language, called $LTL_c/CCSL$, for specifying temporal properties of multi-clock systems. While traditional temporal logics (LTL, MTL, CTL*), whether linear or branching, rely on a global step, our language, $LTL_c/CCSL$, builds a partial order on the logical clocks, thus allowing both a hierarchical approach based on refinement of clock hierarchies, and compositionality as what happens in one clock domain may remain largely independent of what happen in other domains. This good property helps preserve the properties without requiring to perform the proofs again. An $LTL_c/CCSL$ system consists of a clock temporal logic LTL_c , accompanied with a clock calculus called CCSL for specifying clock relations. We build the syntax and semantics of LTL_c and link its semantics with CCSL. After that we mainly focus on the verification of $LTL_c/CCSL$ specifications by using model checking. We show how $LTL_c/CCSL$ can be used for specifying multi-clock systems with an example.

Keywords: Multi-clock Systems; Temporal Logic; Clock Constraints; Formal Specification; Model Checking

Correspondence and offprint requests to: Zhiming Liu, Springer-Verlag London Limited, Sweetapple House, Catteshall Road, Godalming, Surrey GU7 3DJ, UK. e-mail: chris@svl.co.uk

2

Yuanrui Zhang, Frédéric Mallet, Min Zhang and Zhiming Liu

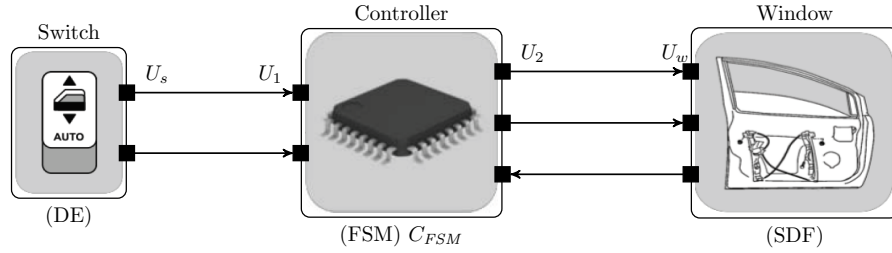


Fig. 1. An Example of Multi-clock Systems

1. Introduction

Large-scaled distributed real-time systems, such as cyber-physical systems [1] and internet of things [2], are integrations of heterogeneous subsystems that are interconnected through a network. These systems are often complex, with components loosely coupled with each other that are designed using different models of computations (MoCs). Specifying time constraints of this type of systems is of a great challenge. Because each component may agree on a different local notion of time and there is no consensus of a notion of time by all components in a global view. In such a distributed system, *clock relations* between events play a dominant role, as what really matter between different components is the relative ordering between events, rather than the physical times at which events occur [3, 4].

Consider an example of distributed systems in Fig. 1, where a power window system of a vehicle consists of three components: a window control switch, a central controller and the window itself. 3 components are assumed to be modelled by different MoCs and their notions of time may vary (will be fully illustrated in Sect. 4). For instance, in the switch, time does not matter. A message “Up” is only sent to the controller through port U_s (say event a) when a user presses the button. On the other hand, in the controller, all events are assumed to be triggered frequently by an electronic signal, say, every 10ms. Therefore, only every 10ms the controller can listen to and receive a message from the switch through port U_1 (say event b). Here, the relation between events a and b has nothing to do with explicit time values. It is just a logical relation, called *causality*, such that “each occurrence of event b relies on an occurrence of event a ”. Similarly, it does not matter when (at exactly what time) a control signal moving the window up is sent to the window through port U_2 (say event c), but whether this event can immediately react to a trigger of a base signal C_{FSM} that represents the local time of the controller, before the next trigger of signal C_{FSM} . This relation, called *synchrony*, says that “event c occurs simultaneously with signal C_{FSM} ” or more precisely, at the same *instant*.

In this example, and in the paper in general, we use interchangeably, the wording *clock* instead of event, as *logical clocks* [5] ultimately capture remarkable discrete events of the system. The *ticks* of a clock represent the sequence of occurrences of its corresponding event. The logical relations seen above, also called *clock relations*, coordinate the behaviours of different components so that they act accurately and timely to their local notions of time. The idea of abstracting explicit time and only considering relative relations of occurrences of events has been deeply embedded into the design paradigms of nowadays popular modelling languages for real-time distributed systems, like Esterel [6], Signal [7], Simulink [8], Ptolemy [9], UML/MARTE [10].

A system without a global notion of time is *multi-clocked* (also called *polychronous* in some references such as [11]), in the sense that clocks may remain independent from each other without necessarily having a (known) relation between them. We do not have the mechanisms to compare all the events, or we do not want to pay the cost of being able to do so. In the example above, we have seen that event a does not follow any time restriction since it comes as a user input. In the controller, for example, event c may act in a not very precise time interval due to, e.g., modelled by a software programming language such as Javascript where precise time requirements cannot be guaranteed (see the example in [12]). While on the other hand, event d , that the window receives a message from port U_w , is triggered strictly periodically by an electronic signal, like every 16 ms. Here, events a , c and d are not comparable.

For multi-clock systems, temporal properties are, and should be, only related to local notions of time. For instance, the property that “at any time, the message sent from the port U_s is equivalent to the message received from the port U_1 ” is only related to the local clock (event) b , which occurs every 10 ms. In a multi-

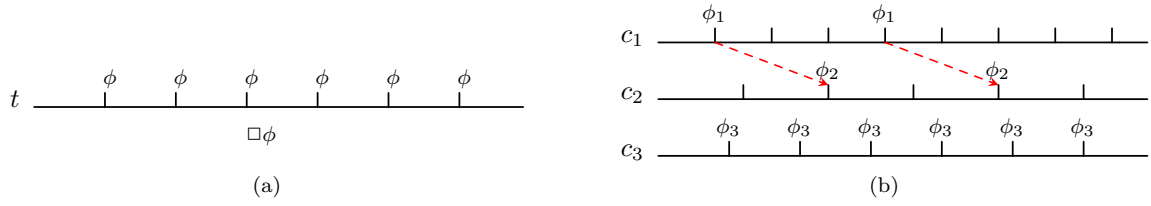


Fig. 2. A global time line (left) vs. multiple local time lines (right)

clock system, it is more suitable to specify temporal properties with respect to local clocks of components, rather than a set of global temporal states. Instead of a temporal property like “property ϕ holds at all states” described by an LTL formula $\Box\phi$ (see Fig. 2 (a)), we prefer *local temporal properties* such as:

- (1) “at all states where clock c_3 ticks, property ϕ_3 holds” (Fig. 2 (b)), or
- (2) “whenever property ϕ_1 holds on the states where clock c_1 ticks, there will be a state where clock c_2 ticks at which property ϕ_2 holds” (indicated by dashed arrows in Fig. 2 (b)).

These temporal properties are built upon “multiple time lines” indicated by many local clocks. They capture the time structure of a multi-clock system in a clear way. Moreover, as we will show, they are more robust to changes within the system models, since the time properties may remain completely independent from the new clocks that might be introduced later on, or from other clock domains.

In this paper, we propose a specification language to handle explicitly logical clocks and clock relations, so as to specify temporal properties of multi-clock systems, as discussed above. The language is a temporal logic called *clock temporal logic* (LTL_c for short), which is accompanied with a clock calculus, namely *Clock Constraint Specification Language* (CCSL) [13], for specifying logical relations among clocks. Unlike traditional temporal logics such as LTL [14], MTL [15] or MITL (metric interval temporal logic) [16] which only focuses on “a property on a set of global states”, LTL_c binds each atomic proposition and each temporal operator with a clock, like $\Box_c$, where the clock (here c) plays the role of a filter that “extracts” the states at which it ticks. For example, formula $\Box_{c_3}\phi_3$, which specifies the property (1) above, expresses that at every state **where clock c_3 ticks**, the formula ϕ_3 holds. Clocks appearing in LTL_c formulae thus form a partially-ordered time model where multiple time lines exist. This is more general than branching time as we show in the paper. CCSL expresses pure logical relations between clocks. For instance, the causality relation mentioned above can be expressed as a clock relation $a \preceq b$ in CCSL. CCSL provides a kernel set of clock expressions that are expressive enough to capture most interesting clock relations in distributed real-time systems [17].

An $LTL_c/CCSL$ specification is expressed as a pair $\langle\phi, \Phi\rangle$, where ϕ is an LTL_c formula and Φ is a set of CCSL expressions, or called *clock constraints*. Hence, clocks become *first-class citizens* in the logic, providing a hierarchical model of time for specifying properties. It syntactically isolates temporal properties that are only related to local clocks. Clock constraints stand for capturing the temporal and time interrelations among components of the system. Such a hierarchical structure also supports a convenient refinement process of time constraints by simply adding new clock constraints into the set Φ of an $LTL_c/CCSL$ specification $\langle\phi, \Phi\rangle$, while keeping the form of the LTL_c formula ϕ unchanged.

As an example, consider an $LTL_c/CCSL$ specification

$$sp = \langle\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2), \{c_1 \prec c_2\}\rangle$$

where formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ actually captures Property (2) mentioned above, which is based on the local clocks c_1 and c_2 . Clock constraint $c_1 \prec c_2$ describes a *precedence relation* between c_1 and c_2 , meaning “every tick of c_2 is dependent on a tick of c_1 ”. Intuitively, formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ is independent from the constraint $c_1 \prec c_2$, since its meaning has nothing to do with how the two clocks relate to each other. Assuming in a refined system model where a local clock c_3 has a relation with c_2 as $c_3 \preceq c_2$, then the new specification becomes

$$sp' = \langle\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2), \{c_1 \prec c_2, c_3 \preceq c_2\}\rangle,$$

where the temporal property $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ still holds.

In this paper, after establishing $LTL_c/CCSL$, we mainly focus on the verification of $LTL_c/CCSL$ specifications. We propose a model checking approach for verifying an $LTL_c/CCSL$ specification in a multi-clock

system, modelled as a *clock transition system* (CTS). The main idea behind that is similar to the model checking approach for LTL, which relies on the transformation from logical formulae into transition systems [18]. Since the automata of $LTL_c/CCSL$ specifications might be infinite (as shown in Sect. 5.2), we only succeed in verifying a subset of $LTL_c/CCSL$ specifications whose state space is bounded.

We begin by firstly building CTSs as an automata semantics for multi-clock systems and *CTSs with accepting states* (CTSA) as an automata semantics for LTL_c formulae and CCSL constraints (Sect. 5.1). Then the verification of $LTL_c/CCSL$ specifications can be carried out by transforming both LTL_c formulae and CCSL constraints into CTSA. Reachability analysis is made on the product of the CTS of a multi-clock system and these CTSA. The transformation from LTL_c formulae into CTSA relies on the transformation from LTL formulae into Büchi automata, so we analyze the expressiveness of LTL_c and show how to encode an LTL_c formula into an LTL formula (Sect. 2.2). To prove the validity of our verification method, we study the observation theory of CCSL (Sect. 5.3), which reveals the characteristics of CTSs/CTSA as observers.

The rest of the paper is organized as follows. In Sect. 2, we propose LTL_c and compare its expressiveness with LTL. In Sect. 3, we give the semantics of $LTL_c/CCSL$ specifications. Sect. 4 analyzes a case study and shows how $LTL_c/CCSL$ specifications can be used to specify properties of multi-clock systems. Sect. 5 mainly introduces a method for verification of $LTL_c/CCSL$ specifications. In Sect. 6 we discuss related work. Sect. 7 concludes this paper and discusses possible future work.

2. Clock Temporal Logic (LTL_c)

In this section, we propose a temporal logic, called *clock temporal logic* (LTL_c for short¹), to specify temporal properties based on multiple local clocks. We first introduce the syntax and semantics of LTL_c in Sect. 2.1, then analyze its expressiveness in Sect. 2.2.

2.1. Syntax and Semantics of LTL_c

Before we define the syntax of LTL_c , we introduce the notion of logical clocks. In a distributed system, a clock captures a remarkable discrete events, the ticks of a clock represent the sequence of occurrences of its corresponding event.

Definition 2.1 (Logical clocks). A logical clock c is an infinite sequence of ticks, denoted as:

$$c = c(1)c(2)...c(i)....$$

Our goal is to design a logic that can naturally express temporal properties based on local times without the notion of a global time. To this end, we extend LTL by assigning a clock to each atomic formula and temporal operator, to express that the atomic formula or temporal operator relates to a time recorded by that clock. An atomic formula must be associated to a clock because when we say a proposition holds, we actually mean it holds at the states where the clock ticks. In the following, sometimes we also call a state an *instant*, and call a state where a clock ticks an *instant of that clock*.

Definition 2.2 (Syntax of LTL_c). The syntax of LTL_c is defined as follows in BNF form:

$$\phi ::= tt \mid p_c \mid \neg\phi \mid \phi \wedge \phi \mid \mathbf{X}_c\phi \mid \varphi \mathbf{U}_c\varphi.$$

tt is Boolean true. And we use ff to represent Boolean false. p represents an atomic proposition. The temporal operators $\mathbf{X}$, $\mathbf{U}$ are the *next* and *until* operators in LTL (see Appendix A for a brief introduction to LTL). c is a *clock*. Each atomic formula p is associated to a clock c , meaning that we only care the truth of p at the instants of clock c . Each temporal operator ($\mathbf{X}$, $\mathbf{U}$) is associated to a clock c , meaning that it only works at the instants of clock c .

For a formula ϕ of the form $\mathbf{X}_c\psi$ or $\psi_1\mathbf{U}_c\psi_2$, we sometimes omit the associated clock of any atomic subformula of ϕ or of any temporal operator of a subformula of ϕ if it is c . For example, formula $\mathbf{X}_c((p_1)_c \wedge \mathbf{X}_c(p_2)_c)$ is often simply written as $\mathbf{X}_c(p_1 \wedge \mathbf{X}p_2)$ where the associated clocks of p_1 , p_2 and the operator $\mathbf{X}$ are

¹ We use LTL_c instead of CTL, to distinguish it from that famous logic CTL by E. Clarke et al.

omitted. Sometimes we use φ_c to denote a formula φ in which the associated clocks of all atomic formulae and all temporal operators are c . For example, formula $(p_1 \wedge \mathbf{X}p_2)_c$ just means $(p_1)_c \wedge \mathbf{X}_c(p_2)_c$.

Given an LTL_c formula ϕ , we use $\mathcal{C}(\phi)$ to denote the set of all clocks that appear in ϕ . For example, $\mathcal{C}(\mathbf{X}_c((p_1)_{c_1} \wedge \mathbf{X}_c(p_2)_{c_2})) = \{c, c_1, c_2\}$.

Intuitively, LTL_c formulae capture the temporal properties relative to local times represented by logical clocks. The truth of an LTL_c formula not only depends on the truth of all of its atomic propositions, but also all of its associated clocks. Clocks form a partially-ordered time model in which the concept of a global time may not exist since clocks can be independent to each other.

For example, the formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ discussed in Sect. 1 captures a temporal property (the property (2) in Sect. 1) relative to the logical clocks c_1 and c_2 . Its time model is shown in Fig. 2 (b), where there does not exist a minimum granularity of time to synchronize clocks c_1 and c_2 . The truth of this formula also depends on how the clocks c_1 and c_2 tick at each instant.

The semantics of our proposed LTL_c is defined over a global configuration of clocks, in which whether a clock ticks or not at each instant is well defined. We first introduce the concept of traces and configurations based on which the semantics of LTL_c is defined in Def. 2.4.

Definition 2.3 (Trace and Configuration). A *trace* π is an infinite sequence:

$$\pi = q_0q_1\dots q_i\dots,$$

where each q_i represents the system state at instant i .

Given a set of atomic propositions Π and a set of clocks Clk , let $S(\pi)$ be the set of states in the trace π , a *configuration* is defined as a triple $\langle \pi, \theta, \xi \rangle$ where $\theta : S(\pi) \rightarrow 2^\Pi$ maps each state to a set of atomic propositions holding at the state, $\xi : S(\pi) \rightarrow 2^{Clk}$ maps each state to a set of clocks that tick at the state.

For any position $i \geq 0$, we use $\pi[i]$, $\pi[0, i]$ and $\pi[i, \infty]$ to denote the $(i + 1)$ -th element q_i of π , the finite prefix $q_0q_1\dots q_i$ of π and the suffix $q_iq_{i+1}\dots$ of π respectively.

The configuration $\langle \pi, \theta, \xi \rangle$ describes an execution trace π with θ capturing its behaviour and ξ capturing its time model. ξ describes the schedule of all clocks at each instant.

Definition 2.4 (Semantics of LTL_c). Given an LTL_c formula ϕ , for any configuration $\langle \pi, \theta, \xi \rangle$, the trace π satisfies ϕ with respect to θ and ξ , denoted as $\pi \models_{\theta, \xi} \phi$, is defined inductively according to the syntactical structure of ϕ :

1. $\pi \models_{\theta, \xi} p_c$ iff $c \in \xi(\pi[0])$ and $p \in \theta(\pi[0])$.
2. $\pi \models_{\theta, \xi} \neg\phi$ iff $\pi \not\models_{\theta, \xi} \phi$.
3. $\pi \models_{\theta, \xi} \phi_1 \wedge \phi_2$ iff $\pi \models_{\theta, \xi} \phi_1$ and $\pi \models_{\theta, \xi} \phi_2$.
4. $\pi \models_{\theta, \xi} \mathbf{X}_c\phi$ iff there exists $i > 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$, and for any k with $0 < k < i$, $c \notin \xi(\pi[k])$.
5. $\pi \models_{\theta, \xi} \phi_1 \mathbf{U}_c\phi_2$ iff there exists $i \geq 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi_2$, and for any $0 \leq j < i$, if $c \in \xi(\pi[j])$ then $\pi[j, \infty] \models_{\theta, \xi} \phi_1$.

We simply write $\pi \models \phi$ if θ and ξ are clear or do not matter in the context.

The formula p_c expresses that at current state, the clock c must tick while the proposition p holds. The formula $\mathbf{X}_c\phi$ means that at the next state at which the clock c ticks, ϕ holds. The formula $\phi_1 \mathbf{U}_c\phi_2$ expresses that ϕ_1 holds at each state where the clock c ticks until a state where ϕ_2 holds.

From Def. 2.4, it is easy to see that in LTL_c temporal formulae, a clock plays as the role of a filter that can extract those states at which it ticks on an execution trace. In other words, LTL_c formulae only concern states at which relevant clocks tick.

Like LTL, in LTL_c we introduce the *always* operator $\Box$ and the *eventually* operator $\Diamond$ for conveniently expressing safety and liveness properties, they are defined as:

$$\Diamond_c\phi = tt\mathbf{U}_c\phi, \Box_c\phi = \neg\Diamond_c\neg\phi.$$

From Def. 2.4 we can easily prove that the meanings of $\Diamond_c\phi$ and $\Box_c\phi$ are as stated as follows:

- $\pi \models_{\theta, \xi} \Diamond_c\phi$ iff there exists $i \geq 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$.
- $\pi \models_{\theta, \xi} \Box_c\phi$ iff for all $i \geq 0$, if $c \in \xi(\pi[i])$ then $\pi[i, \infty] \models_{\theta, \xi} \phi$.

Intuitively, the formula $\Box_c\phi$ means that at all states where the clock c ticks, the formula ϕ holds. The formula $\Diamond_c\phi$ means that there exists a state where the clock c ticks and ϕ holds.

2.2. Expressiveness of LTL_c and its Relation to LTL

The definition of traces (Def. 2.3) and the semantics of LTL_c (Def. 2.4) actually have indicated that an LTL_c formula can be expressed as an LTL formula if we take the occurrences of clocks as a special type of atomic propositions. For any clock c , its occurrence at each state can be captured as a proposition, named P_c . So its absence at each state can be captured as $\neg P_c$. With this observation, according to Def. 2.4, LTL_c can be encoded into LTL as stated in the following definition.

Definition 2.5 (Encoding from LTL_c Formulae to LTL Formulae). Any LTL_c formula ϕ can be encoded into an LTL formula, denoted by $\phi^\mathfrak{D}$. The encoding is defined inductively as follows:

1. $p_c^\mathfrak{D} = P_c \wedge p$.
2. $(\neg\phi)^\mathfrak{D} = \neg\phi^\mathfrak{D}$.
3. $(\phi_1 \wedge \phi_2)^\mathfrak{D} = \phi_1^\mathfrak{D} \wedge \phi_2^\mathfrak{D}$.
4. $(\mathbf{X}_c\phi)^\mathfrak{D} = \mathbf{X}(\neg P_c \mathbf{U}(P_c \wedge \phi^\mathfrak{D}))$.
5. $(\phi_1 \mathbf{U}_c \phi_2)^\mathfrak{D} = (P_c \rightarrow \phi_1^\mathfrak{D}) \mathbf{U}(P_c \wedge \phi_2^\mathfrak{D})$.

For any clock c , P_c is the corresponding atomic proposition that indicates the occurrence of c .

By encoding clocks as special atomic propositions, an execution trace over a set of clocks can then be modeled as an execution trace without clocks. Formally, we have the following definition.

Definition 2.6 (Configuration vs. Ordinary Configuration). A configuration $\langle \pi, \theta, \xi \rangle$ can be encoded as an ordinary configuration (see Appendix A for its definition) denoted as $\langle \pi, \theta^\xi \rangle$ without mapping ξ . Mapping θ^ξ is defined as:

$$\theta^\xi(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}.$$

The semantics of LTL_c formulae and their encoded LTL formulae have the following relationship.

Theorem 2.1. Given an LTL_c formula ϕ , for any configuration $\langle \pi, \theta, \xi \rangle$, let $\phi^\mathfrak{D}$ be the encoded LTL formula of ϕ , $\langle \pi, \theta^\xi \rangle$ be the corresponding ordinary configuration of $\langle \pi, \theta, \xi \rangle$, then

$$\pi \models_{\theta, \xi} \phi \text{ iff } \pi \models_{\theta^\xi} \phi^\mathfrak{D},$$

where the satisfaction relation $\models_\theta$ for any θ is defined in Appendix A.

Thm. 2.1 can be proved inductively based on the syntax of LTL_c (Def. 2.4).

The proof of Thm. 2.1 We proceed the proof by induction on the syntax of LTL_c formula ϕ .

If $\phi = p_c$, we prove that $\pi \models_{\theta, \xi} p_c$ iff $\pi \models_{\theta^\xi} p_c$. By the semantics of p_c (Def. 2.4), the semantics of $p_c^\mathfrak{D}$ (Appendix A) and the relation between θ and θ^ξ in Def. 2.6, we have: $\pi \models_{\theta, \xi} p_c$ iff $c \in \xi(\pi[0])$ and $p \in \theta(\pi[0])$ iff $\pi \models_{\theta^\xi} P_c$ and $\pi \models_{\theta^\xi} p$ iff $\pi \models_{\theta^\xi} P_c \wedge p$ iff $\pi \models_{\theta^\xi} p_c^\mathfrak{D}$.

For the inductive case, we only take $\phi = \mathbf{X}_c\phi$ for an example, other cases are similar. By inductive hypothesis, we know that for any configuration $\langle \pi', \theta', \xi' \rangle$, $\pi' \models_{\theta', \xi'} \phi$ iff $\pi' \models_{\theta', \xi'} \phi^\mathfrak{D}$. Now we prove that $\pi \models_{\theta, \xi} \mathbf{X}_c\phi$ iff $\pi \models_{\theta^\xi} (\mathbf{X}_c\phi)^\mathfrak{D}$. Again, by the semantics of $\mathbf{X}_c\phi$ in Def. 2.4, the semantics of $(\mathbf{X}_c\phi)^\mathfrak{D}$ in Appendix A and the relation between θ and θ^ξ in Def. 2.6, we have: $\pi \models_{\theta, \xi} \mathbf{X}_c\phi$ iff there exists $i > 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$ and for any k with $0 < k < i$, $c \notin \xi(\pi[k])$ iff there exists $i > 0$ s.t. $\pi[i, \infty] \models_{\theta^\xi} P_c$ and $\pi[i, \infty] \models_{\theta^\xi} \phi^\mathfrak{D}$ and for any k with $0 < k < i$, $\pi[k, \infty] \models \neg P_c$ iff $\pi \models_{\theta^\xi} \mathbf{X}(\neg P_c \mathbf{U}(P_c \wedge \phi^\mathfrak{D}))$ iff $\pi \models_{\theta^\xi} (\mathbf{X}_c\phi)^\mathfrak{D}$. $\square$

Generally, LTL_c is less expressive than LTL since in LTL_c each atomic and temporal formula must be associated to a clock while an LTL formula does not have to be related to an atomic proposition of a clock. However, imagining that we allow a fictitious global clock, say c_0 , to tick at any time (i.e., at any states of any traces), then in this sense any LTL formula can be expressed by an LTL_c formula, which is formed by simply assigning all of its atomic formulae and temporal operators to the clock c_0 . Formally, suppose there exists a clock c_0 s.t. for any configuration $\langle \pi, \theta, \xi \rangle$ and any state $q \in S(\pi)$, $c_0 \in \xi(q)$ holds, then an LTL formula ϕ can be encoded into the LTL_c formula ϕ_{c_0} .

Notice that although the occurrences of clocks in LTL_c in fact can be encoded as atomic propositions in LTL in terms of expressiveness, clock itself is an explicit notion in the model of practical systems and is independent from the notion of system states. One characteristic of clocks that is distinguished from general atomic propositions is that a clock is always an infinite sequence and ticks for infinitely many times in a reactive system model.

3. LTL_c with a Set of Clock Constraints

In Sect. 2 we have seen that in LTL_c all temporal properties are based on multiple clocks and these clocks together form a partially-ordered time model. In multi-clock systems, clocks are often constrained by some tense or loose logical relations which cannot be expressed by LTL_c formulae itself. In this section, we use a pure clock calculus called CCSL to capture these constraints. CCSL helps coordinate the local clocks of different system components by describing how clocks are related to each other using an expressive set of clock-oriented expressions. By associating CCSL constraints to LTL_c formulae, we introduce $LTL_c/CCSL$ specifications for specifying properties of multi-clock systems. In Sect. 4, we show how $LTL_c/CCSL$ specifications can be used to specify multi-clock systems in a hierarchical manner.

In Sect. 3.1 we first introduce the syntax and semantics of CCSL in the context of the semantics of LTL_c . Then in Sect. 3.2 we define $LTL_c/CCSL$ specifications. At last, in Sect. 3.3 we compare the expressiveness of $LTL_c/CCSL$ specifications with that of LTL.

3.1. Clock Constraints

CCSL (clock constraint specification language) [13, 19, 17] is a clock-oriented specification language for specifying clock constraints. It was initially proposed as a companion language for UML/MARTE [10]. In CCSL, clocks are taken as a primitive and clock constraints are expressed as logical relations between clocks. As an independent clock calculus, CCSL is mainly used for coordinating different components, which may be modelled by different MoCs, of a distributed system.

The definition of CCSL proposed in this section is mainly based on the previous works [19, 17], but is altered in order to adapt the semantics of LTL_c . We also introduce the concepts of dependence relation between clocks (Def. 3.3) and well-defined clock constraints (Def. 3.3), which were not addressed in the previous works. They are necessary for building a rigorous theory of CCSL in LTL_c .

A *CCSL constraint* (or a *clock constraint*) can be either a *clock relation* or a *clock definition*. Clock relations describe binary relationships between clocks, while clock definitions define new clocks by combining existed clocks using different operators.

The syntax of clock constraints is given in the following definition.

Definition 3.1 (Clock Constraint). A clock constraint Cn is given as follows in the BNF form:

$$Cn ::= Rel \mid c \triangleq e,$$

where Rel is called a “clock relation”, defined as:

$$Rel ::= c_1 \prec c_2 \mid c_1 \preceq c_2 \mid c_1 \# c_2 \mid c_1 \subseteq c_2,$$

$c \triangleq e$ is called a “clock definition”, where the expression e is defined as:

$$e ::= c_1 \mid c_1 + c_2 \mid c_1 * c_2 \mid c_1 \blacktriangleright c_2 \mid c_1 \triangleright c_2 \mid c_1 \curvearrowright c_2 \mid c_1 \propto n \mid c_1 \$ n \mid c_1 \vee c_2 \mid c_1 \wedge c_2.$$

c must be a different clock from c_1 and c_2 , $n \geq 1$.

The intuitive meaning of each clock constraint is given below after the semantics of CCSL constraints is introduced (in Def. 3.5). A set of CCSL constraints is also called a *CCSL specification*, denoted by Φ . We write Φ^r and Φ^d to denote the set of clock relations and the set of clock definitions in Φ respectively, therefore $\Phi = \Phi^r \cup \Phi^d$.

A specification Φ should at least contains one clock relation, otherwise the specification does not give any constraints between the clocks. Therefore, we stipulate that for any specification Φ , $\Phi^r \neq \emptyset$.

In a CCSL specification, if a clock, named c , is defined by some clock definition $c \triangleq e$, we call it a *defined*

clock. We call a clock *pure clock* if it is not defined by any clock definitions. For example, in specification $\{c_1 \prec c_2, c_1 \triangleq c_3 + c_4\}$, the clocks c_2, c_3, c_4 are pure clocks, while the clock c_1 is a defined clock.

The clock constraints defined in Def. 3.1 are the main focus in this paper, however, we also need to introduce the negation of a clock relation Rel , denoted as $\sim Rel$, which is used in the verification of $LTL_c/CCSL$ specifications in Sect. 5.5. We use Cn^+ to denote any clock constraint or the negation of any clock relation. Formally, we define

$$Cn^+ ::= Cn \mid \sim Rel.$$

For convenience, in the rest of the paper, we also call the negation of a clock relation a “clock constraint”. So we also call Cn^+ a “clock constraint”.

Given a set of clock constraints Φ , we use $\sim \Phi$ to denote the set of clock constraints obtained by replacing every clock relation in Φ with its negation. Formally,

$$\sim \Phi = \{c \triangleq e \mid (c \triangleq e) \in \Phi^d\} \cup \{\sim Rel \mid Rel \in \Phi^r\}.$$

Given a clock constraint Cn^+ , we use $\mathcal{C}(Cn^+)$ to denote the set of all clocks that appear in Cn^+ .

There exist dependence relations between clocks. Clocks appearing in one constraint may be defined in some clock definition of another constraint. For example, in specification $\{c_1 \prec c_2, c_1 \triangleq c_3 + c_4\}$, the clock c_1 in the constraint $c_1 \prec c_2$ is defined in the clock definition $c_1 \triangleq c_3 + c_4$. The following definition defines the dependence relation between clocks.

Definition 3.2 (Dependence Relation between Clock Constraints). A clock constraint Cn_1^+ *directly depends on* a clock definition $Cn_2^+ = (c \triangleq e)$, denoted by $Cn_1^+ \rightsquigarrow Cn_2^+$, if c appears in Cn_1^+ . We use $\rightsquigarrow^+$ to denote the transitive closure of $\rightsquigarrow$. If $Cn_1^+ \rightsquigarrow^+ Cn_2^+$, we say Cn_1^+ *depends on* Cn_2^+ .

Apparently one situation that needs to be avoided when defining a CCSL specification is that the clocks of two constraints come from the definitions of each other. One example is constraints $\{c_1 \triangleq c_2 + c_3, c_2 \triangleq c_1 * c_4\}$, where the clock c_2 in the constraint $c_1 \triangleq c_2 + c_3$ comes from the definition $c_2 \triangleq c_1 * c_4$, while the clock c_1 in the constraint $c_2 \triangleq c_1 * c_4$ comes from the definition $c_1 \triangleq c_2 + c_3$. To forbid ill-formed constraints like this, we introduce the notion of *well-defined constraints* in the following definition.

Definition 3.3 (Well-defined Clock-constraint Set). A set of clock constraints Φ is well defined iff in Φ any two clock constraints are not dependent on each other.

Note that Def. 3.3 also forbids the situation when one clock is defined in two different constraints at the same time, e.g., constraints $\{c_1 \triangleq c_2 + c_3, c_1 \triangleq c_4 * c_5\}$.

In this paper, all clock-constraint sets we consider are assumed to be well defined.

In order to associate CCSL constraints to LTL_c formulae, we give the semantics of CCSL constraints over the clock sequences of configurations, which is called the *clock schedules* of configurations.

Definition 3.4 (Clock Schedule). A clock schedule σ (or simply a “schedule”) is an infinite clock sequence:

$$\sigma = C_0 C_1 \dots C_i \dots$$

where each C_i is a set of clocks that tick at the instant i .

Given a configuration $\langle \pi, \theta, \xi \rangle$, we call

$$\xi(q_0)\xi(q_1)\dots\xi(q_i)\dots$$

the clock schedule of $\langle \pi, \theta, \xi \rangle$, denoted as $\sigma^{\langle \pi, \theta, \xi \rangle}$. When θ and ξ are clear and can be omitted in the context, we often call $\xi(q_0)\xi(q_1)\dots\xi(q_i)\dots$ the schedule of the trace π , denoted as σ^π .

The semantics of CCSL constraints given below mainly follows [19, 17]. The main difference is that in order to define that a clock schedule satisfies a set of clock constraints, we define the semantics of both clock relations and clock definitions with respect to a set of clock definitions.

Definition 3.5 (Semantics of CCSL Constraints). The semantics of CCSL constraints is given w.r.t a set of clock definitions $\mathbf{D}$. They are shown in Tables 1 and 2, where the history of a clock schedule σ (w.r.t. $\mathbf{D}$): $\delta_{\mathbf{D}}^\sigma : Clk \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$ is defined as follows:

$$\delta_{\mathbf{D}}^{\sigma, \mathbf{D}}(c, i) = |\{j \mid 0 \leq j \leq i, \sigma[j] \models c, \mathbf{D}\}|.$$

$\sigma \models c_1 \subseteq c_2, \mathbf{D}$	iff	for any $i \geq 0$, if $\sigma[i] \models c_1, \mathbf{D}$, then $\sigma[i] \models c_2, \mathbf{D}$.	(Subclock)
$\sigma \models c_1 \# c_2, \mathbf{D}$	iff	for any $i \geq 0$, $\sigma[i] \not\models c_1, \mathbf{D}$ or $\sigma[i] \not\models c_2, \mathbf{D}$ holds.	(Exclusion)
$\sigma \models c_1 \prec c_2, \mathbf{D}$	iff	for any $i \geq 0$, $\delta^{\sigma, \mathbf{D}}(c_1, i) > \delta^{\sigma, \mathbf{D}}(c_2, i)$, or $\delta^{\sigma, \mathbf{D}}(c_1, i) = \delta^{\sigma, \mathbf{D}}(c_2, i)$ and $\sigma[i] \not\models c_2, \mathbf{D}$.	(Precedence)
$\sigma \models c_1 \preceq c_2, \mathbf{D}$	iff	for any $i \geq 0$, $\delta^{\sigma, \mathbf{D}}(c_1, i) \geq \delta^{\sigma, \mathbf{D}}(c_2, i)$ holds.	(Causality)

Table 1. Semantics of CCSL Relations

$\sigma[i] \models c, \mathbf{D}$ (c is pure)	iff	$c \in \sigma[i]$.	
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c'\}$	iff	$\sigma[i] \models c', \mathbf{D}$.	(Identity)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 + c_2\}$	iff	$\sigma[i] \models c_1, \mathbf{D}$ or $\sigma[i] \models c_2, \mathbf{D}$.	(Union)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 * c_2\}$	iff	$\sigma[i] \models c_1, \mathbf{D}$ and $\sigma[i] \models c_2, \mathbf{D}$.	(Intersection)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 \blacktriangleright c_2\}$	iff	$\sigma[i] \models c_2, \mathbf{D}$ and there is a j s.t. (1) $0 \leq j < i$ and $\sigma[j] \models c_1, \mathbf{D}$, and (2) $\sigma[k] \not\models c_2, \mathbf{D}$ for all $j \leq k < i$.	(Strict Sample)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 \triangleright c_2\}$	iff	$\sigma[i] \models c_2, \mathbf{D}$ and there is a j s.t. (1) $0 \leq j \leq i$ and $\sigma[j] \models c_1, \mathbf{D}$, and (2) $\sigma[k] \not\models c_2, \mathbf{D}$ for all $j \leq k < i$.	(Sample)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 \curvearrowright c_2\}$	iff	$\sigma[i] \models c_1, \mathbf{D}$ and $\sigma[j] \not\models c_2, \mathbf{D}$ for all $0 \leq j \leq i$.	(Interruption)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c' \propto n\}$	iff	$\sigma[i] \models c', \mathbf{D}$ and $\delta^{\sigma, \mathbf{D}}(c', i) = n \cdot \delta^{\sigma, \mathbf{D}}(c, i)$.	(Periodicity)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c' \$ n\}$	iff	$\delta^{\sigma, \mathbf{D}}(c, i) = \max(\delta^{\sigma, \mathbf{D}}(c', i) - n, 0)$.	(Delay)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 \wedge c_2\}$	iff	$\delta^{\sigma, \mathbf{D}}(c, i) = \max(\delta^{\sigma, \mathbf{D}}(c_1, i), \delta^{\sigma, \mathbf{D}}(c_2, i))$.	(Infimum)
$\sigma[i] \models c, \mathbf{D}\{c \triangleq c_1 \vee c_2\}$	iff	$\delta^{\sigma, \mathbf{D}}(c, i) = \min(\delta^{\sigma, \mathbf{D}}(c_1, i), \delta^{\sigma, \mathbf{D}}(c_2, i))$.	(Supremum)

Table 2. Semantics of CCSL Definitions

The history of clock schedule $\delta_{\mathbf{D}}^{\sigma}$ records the number of counts each clock has ticked up to the current instant. The satisfaction relation $\sigma \models A, \mathbf{D}$ (in Tables 1 and 2) means that σ satisfies the expression A under the set of clock definitions $\mathbf{D}$. We use $\mathbf{D}\{Cn\}$ to mean that Cn is a constraint appearing in $\mathbf{D}$, i.e., $Cn \in \mathbf{D}$. We often write $\sigma \models Cn, \mathbf{D}$ simply as $\sigma \models Cn$ when $\mathbf{D} = \emptyset$.

Table 1 lists the semantics of 4 basic clock relations. Subclock and Exclusion describe a synchronous relation between two events. Subclock captures a constraint between c_1 and c_2 in which c_1 can only tick if c_2 ticks at the same instant; Exclusion captures a constraint between c_1 and c_2 in which c_1, c_2 cannot tick at the same instant. Precedence and Causality describe asynchronous dependence relations between two events. Causality captures a constraint between c_1 and c_2 in which the tick of the clock c_2 is caused by the tick of the clock c_1 . In other words, c_2 cannot tick before c_1 ticks. Precedence captures a similar constraint as Causality but it does not allow two clocks tick at the same instant. In other words, it expresses that c_1 ticks strictly faster than c_2 ticks.

Table 2 lists the semantics of 10 basic clock definitions and 1 item (the 1st item) for the case when c is pure. Identity defines a clock c that ticks just as the clock c' ticks. Union defines a clock c that ticks at the instant when either c_1 or c_2 ticks. Intersection defines a clock c that ticks at the instant when both c_1 and c_2 tick. Sample defines a clock c that samples c_1 based on c_2 . c ticks at the instant when c_2 ticks for the first time after a tick of c_1 . Strict Sample is similar to Sample, but it does not allow the clock c to tick at the instant when both c_2 and c_1 tick. Interruption defines a clock c that ticks at the instant when c_1 ticks until c_2 ticks for the first time. Periodicity defines a clock c that ticks every n ticks of the clock c' . c ticks at the instant when c' ticks, and there are n ticks of c' (including the tick of c' at this instant) after the last instant when c ticks. Delay defines a clock c that ticks at the instant when c' ticks and before the instant c' has ticked for at least n times. Infimum defines the slowest clock that is at least as fast as both c_1 and c_2 . c ticks at the instant when the faster clock between c_1 and c_2 ticks. Supremum defines the fastest clock that is at least as slower as both c_1 and c_2 . c ticks at the instant when the slower clock between c_1 and c_2 ticks.

Besides the basic clock constraints defined above, we also introduce a clock relation called *synchronization*, denoted as $c_1 \approx c_2$. Two clocks are synchronized when they always tick at the same instant. Formally, $c_1 \approx c_2$

10

Yuanrui Zhang, Frédéric Mallet, Min Zhang and Zhiming Liu

is defined such that

$$c_1 \approx c_2 \text{ iff } c_1 \preceq c_2 \text{ and } c_2 \preceq c_1.$$

Intuitively, it means that c_1 and c_2 tick simultaneously only when their ticks are caused by each other.

The semantics of the negations of clock relations is given as the following definition.

Definition 3.6 (Semantics of the Negations of Clock Relations). Given a clock relation Rel and a set of clock definitions $\mathbf{D}$, the semantics of $\sim Rel$ is defined s.t. for any schedule σ ,

$$\sigma \models \sim Rel, \mathbf{D} \text{ iff } \sigma \not\models Rel, \mathbf{D}.$$

After giving the semantics of each clock expression, we define the semantics of a clock specification. A schedule satisfies a set of clock constraints, iff it satisfies each clock relation under the set of all clock definitions of this set.

Definition 3.7 (Semantics of Clock Specifications). The semantics of a well-defined clock constraints Φ is given as follows: for any clock schedule σ ,

$$\sigma \models \Phi \text{ iff } \sigma \models Cn, \Phi^d \text{ for each } Cn \in \Phi^r,$$

where Φ^r and Φ^d are the sets of clock relations and clock definitions in Φ respectively.

3.2. $LTL_c/CCSL$ Specifications

We consider the LTL_c formulae, whose partially-ordered time model is coordinated by CCSL expressions. We call them $LTL_c/CCSL$ specifications, whose definition is introduced as follows.

Definition 3.8 ($LTL_c/CCSL$ Specification). An LTL_c formula accompanied with a set of CCSL constraints, called a “ $LTL_c/CCSL$ specification”, is a pair $\langle \phi, \Phi \rangle$, where ϕ is an LTL_c formula, and Φ is a set of CCSL constraints associated to the clocks appearing in ϕ .

The constraints Φ captures the time model of the LTL_c formula ϕ , i.e., how clocks in ϕ behave and relate to each other. Notice that LTL_c formulae are in fact independent from the selection of clock calculi. Other clock calculi instead of CCSL could also be considered as a companion of LTL_c to describe its time model.

The semantics of $LTL_c/CCSL$ specifications is given as follows by linking the semantics of LTL_c and the semantics of CCSL with configurations.

Definition 3.9 (Semantics of $LTL_c/CCSL$ Specifications). Given a configuration $\langle \pi, \theta, \xi \rangle$ and an $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$, the trace π satisfies the specification $\langle \phi, \Phi \rangle$ with respect to θ and ξ , denoted by $\pi \models_{\theta, \xi} \langle \phi, \Phi \rangle$, is defined as:

$$\pi \models_{\theta, \xi} \langle \phi, \Phi \rangle \text{ iff } \pi \models_{\theta, \xi} \phi \text{ and } \sigma^\pi \models \Phi.$$

3.3. Expressiveness of $LTL_c/CCSL$ Specifications

From Sect. 2.2 we see that the expressiveness of LTL_c stays in that of LTL. The expressiveness of $LTL_c/CCSL$ specifications, however, is different from that of LTL. The reason is that CCSL and LTL have different expressiveness. As indicated in [20], on the one hand, for some CCSL constraints whose corresponding transition systems (introduced in Sect. 5.1) have infinite states, there exists no LTL formulae corresponding to them since every LTL formula corresponds to a finite state transition system. For example, constraint $c_1 < c_2$, whose corresponding transition system (shown in Fig. 3 in Sect. 5.2) is infinite, cannot be expressed by any LTL formula. On the other hand, CCSL fails in expressing liveness properties as expressed by LTL formulae, since all CCSL constraints are safety properties according to its semantics (Def. 3.5). One can refer to [20] for more details about the comparison between them.

4. Specification of Multi-clock Systems using $LTL_c/CCSL$ Specifications

In this section, we show how to specify properties of multi-clock systems using $LTL_c/CCSL$ specifications through a simple example. As we will see in this section, $LTL_c/CCSL$ specifications are natural to specify

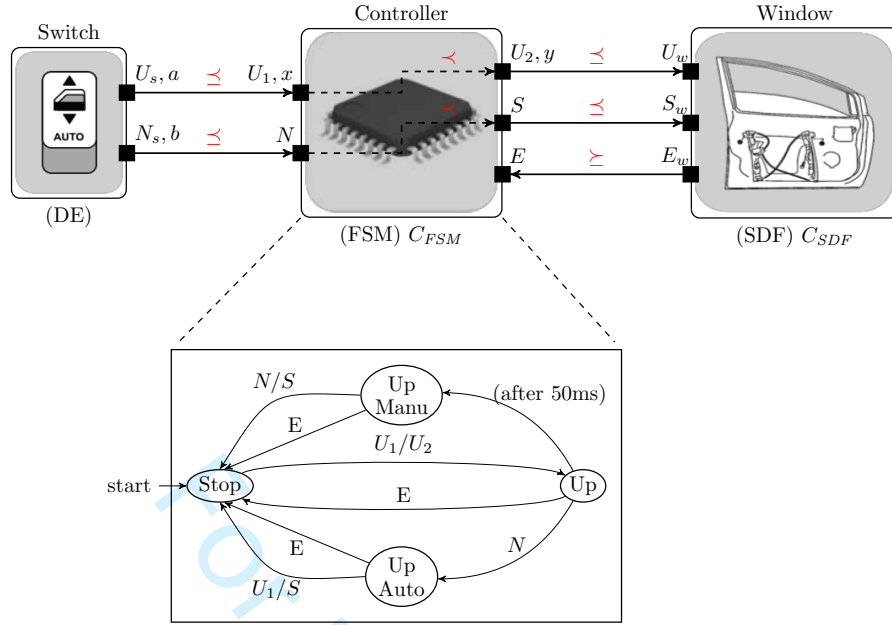


Fig. 3. Power window system

multi-clock systems: with CCSL to specify logical constraints between the clocks of different components and LTL_c to specify value-related properties upon these clocks. We show how $LTL_c/CCSL$ specifications follow a hierarchy manner to specify multi-clock systems, with CCSL providing a convenient way for refining the clock constraints.

4.1. An Example of Multi-clock Systems

As already mentioned in Sect. 1, we introduce a simple multi-clock system — a power window (PW) system, which originally came from [21]. As shown in Fig. 3, the system consists of three components: a control switch, a window controller and a window.

The three components run in parallel and interact with each other by sending and receiving messages through different ports (i.e., U_s , N_s , U_1 , U_2 , N , S , E , U_w , S_w , E_w). Each port is associated to a clock (denoted by the same name) representing the occurrences of an event (denoted by the same name) on this port. For example, at every instant when the clock U_s occurs, the switch sends a message from the port U_s , while at every instant when the clock U_1 occurs, the controller receives a message from the port U_1 . We associate variables to some ports in order to store the messages sent or received from the ports. For example, we associate a variable a to the port U_s . A message sent from the port U_s is stored in the variable a , while a message received from the port U_1 is stored in the variable x .

In the PW system, we assume that three components are modeled by different MoCs: a discrete event (DE) model [22], a finite state machine (FSM) and a synchronous data flow (SDF) [22] respectively. As will be seen below, the PW system is a multi-clock system since the clocks of each component represent different times and there does not exist a global clock that synchronizes all the clocks of the whole system.

The behaviour of each component is illustrated as follows:

- The control switch is described as a DE model, where a behaviour is represented as a set of discrete events and each event is tagged with a time value, called a *timestamp*, indicating when it occurs. The control switch continuously receives a user's inputs from the environment. The event U_s or N_s is triggered when the position is changed to *up* or *neutral* by the user through pressing or releasing the button Up^2 . The

² for simplifying the model, we omit the discussion for the behaviour that the user presses or releases the button *Down*. We also omit the corresponding behaviours in the window controller and the window as shown below.

upper part of Fig. 4 shows a possible behaviour of the user's inputs on the control switch, where the occurrences of the events U_s , N_s are tagged with values of the variables a , b and with timestamps whose unit is millisecond (abbreviated as *ms*).

- The window controller interacts with both the control switch and the window. It receives messages from the control switch, and send messages to the window motor to move up and down the window. The window controller is modeled by a Mealy-style timed FSM as shown in the lower part of Fig. 3. The window controller offers two modes for moving up the window³:

- 1) The mode *Up Manual*, in which the window is moved up manually. If the user keeps pressing the button U_p for 50ms or more than 50ms, then the window controller will enter in this mode. The window controller will then send messages to the window motor to move the window up until the user release the button U_p .
- 2) The mode *Up Automatic*, in which the window is moved up automatically. If the user releases the button U_p in less than 50ms, then the window controller will enter in this mode. The window controller will then send messages to the window motor to move the window up until the button is pressed again.

The transition of the FSM can be a timed transition, which indicates an elapse of time, or an event transition in a *guard/action* style, which means that once the *guard* is satisfied, the *action* is fired without time consuming.

The behaviour of the window controller is described as follows. At the state *Stop*, the window controller receives a message from the control switch that the button U_p has been pressed through the port U_1 and send a message through the port U_2 to the window motor to move up the window. This behaviour is captured by an event transition U_1/U_2 from the state *Stop* to the state *Up*. At the state *Up*, the window controller returns to the state *Stop* if it receives a message from the window through the port E that the window is fully closed. If the window controller receives a message from the control switch through the port N that the button U_p has been released in less than 50ms, it will move to the state *Up Auto* (the *Up Automatic* mode), where

- 1) if the window controller receives a message from the window through the port E that the window is fully closed, it will move to the state *Stop*;
- 2) if the window controller receives a message from the control switch through the port U_1 that the button U_p is pressed again, it will send a message through the port S to the window motor to stop moving the window.

On the other side, at the state *Up*, if the window controller does not receive the message from the control switch in 50ms, it will move to the state *Up Manu* (the *Up Manual* mode), where

- 1) if the window controller receives a message from the window through the port E that the window is fully closed, it will move to the state *Stop*;
- 2) if the window controller receives a message from the control switch through the port N that the button U_p is released by the user, it will send a message to the window motor to stop moving the window through the port S .

The middle part of Fig. 4 gives a possible schedule of the events in the FSM of the window controller, where δ_1 , δ_2 , δ_3 represent the time intervals between the receiving of the messages through the events U_1 and N .

- The window is modeled as an SDF, in which the physical time of events is ignored and the logical ordering of events is focused. The window motor receives a message from the window controller through the ports U_w and S_w and move the window up and down according to the message⁴. The window gives a feedback to the window controller through the port E_w after the window is fully closed. The lower part of Fig. 4 describes a possible schedule of the events in the SDF of the window, where only the ordering of events is captured.

The three components together form the PW system.

³ like in the control switch, we omit the discussion of the behaviour for moving down the window.

⁴ like in the control switch and the window controller, we omit the discussion of the behaviour that the window motor moves the window down.

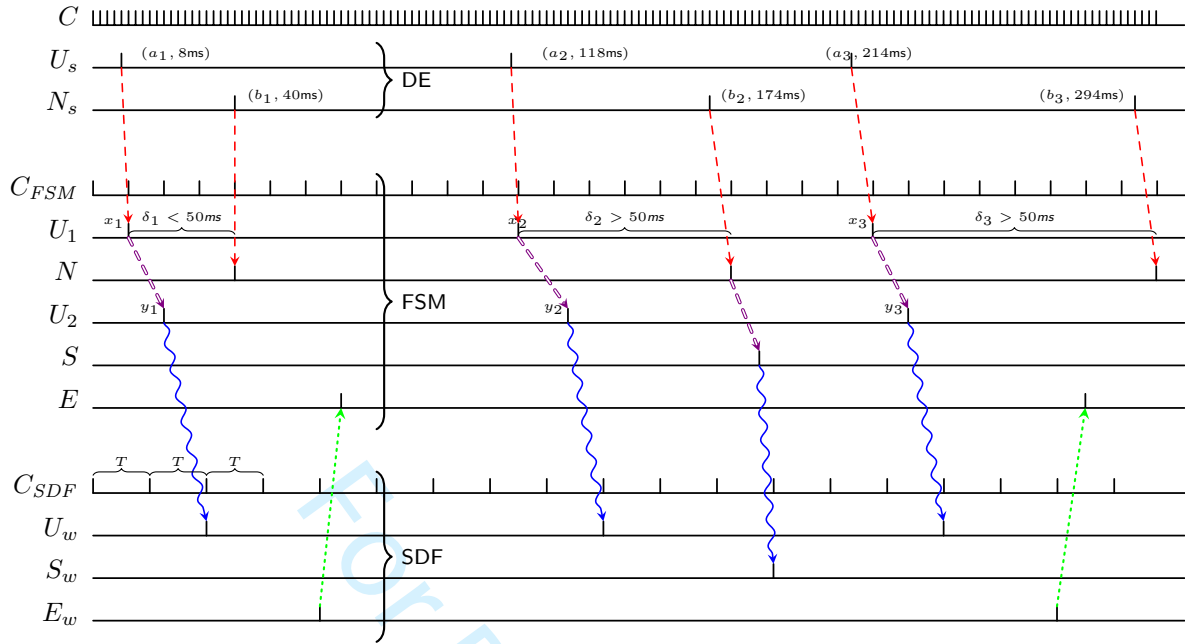


Fig. 4. A schedule of power window system

4.2. Specifying the PW System with $LTL_c/CCSL$ Specifications

In such a model of the PW system, as we see, each component has its own notion of time and it makes no sense to say something about the “global state” of the model. So at this abstraction level, tradition LTL is not suitable to specify the system anymore.

In the following, we show that in the PW model, we can specify the properties based on the local times using $LTL_c/CCSL$ specifications.

4.2.1. Local Time Properties in a Single Component

In the window controller, we consider the property:

“the value received from the port U_1 is sent from the port U_2 immediately in the window controller”.

This is a property that only relates to the local time, indicated by the clocks U_1 and U_2 , in a component. We can specify the property as follows:

$$spec_{sp1} = \langle \Box_{U_1} (x = v \rightarrow \mathbf{X}_{U_2} (y = v)), \Phi_{sp1} \rangle,$$

where $\Phi_{sp1} = \emptyset$. Note that for the window controller component, the “immediate” moment to send the value from port U_2 is the next tick of the clock U_2 . The specification means that whenever the clock U_1 ticks, if the variable x equals to a value v , then the variable y equals to the same value when the next time the clock U_2 ticks. The clocks U_1 and U_2 are not further constrained so Φ_{sp1} is empty.

We can see that the specification $spec_{sp1}$ is completely local to the window controller component in the sense that it has nothing to do with the times of other components.

4.2.2. Temporal Properties between Components

In the PW system, three components have their local times which are independent from each other. However, at some level of modelling, it is required that these component should be coordinated in some way so that the communication between components can work as expected. This is called *time adaption* [21] between different MoCs. $LTL_c/CCSL$ can be used either to specify or to enforce this type of system behaviours depending on different requirements.

For instance, as already explained in Sect. 1, the communication between the window controller and the control switch should be asynchronous (indicated by the red dashed arrows in Fig. 4), because a user input from the control switch can happen at any time, while an event for receiving an input in the window controller can only occur over a period of time triggered by an electronic signal. To describe this communication, we build a set of clock constraints as follows:

$$U_s \preceq U_1, N_s \preceq N.$$

These causality relations actually capture an dependency relation between the events of two different components.

Similarly, the communication between the window controller and the window should also be asynchronous (indicated by the blue snake arrows and green dashed arrows in Fig. 4). As we will see, the local times of the two components are different so their events do not have to occur at the same time. Therefore, we have the following clock constraints between their events as follows:

$$U_2 \preceq U_w, S \preceq S_w, E_w \preceq E.$$

To sum it up, the clock specification that describes the time adaption between the three components is:

$$\Phi_{sp2} = \{U_s \preceq U_1, N_s \preceq N, U_2 \preceq U_w, S \preceq S_w, E_w \preceq E\}.$$

Except for time requirements, we can also consider data dependencies between the three components. The process of specifying data constraints between different MoCs is called *data adaption* in [21].

For example, we can consider the property:

“The value sent through the port U_s will be eventually received from the port U_1 ”.

It can be specified by an $LTL_c/CCSL$ specification given below:

$$spec_{sp2} = \langle \Box_{U_s}(a = v \rightarrow \Diamond_{U_1}(x = v)), \Phi_{sp2} \rangle,$$

which means that whenever the clock U_s ticks, if the variable a equals to a value v , then the variable x will equal to the same value eventually when the clock U_1 ticks. Note that this specification is different from $spec_{sp1}$ because it allows the window controller to delay dealing with the message v . This means that at the port U_1 , a buffer is needed to store the messages coming from the port U_s .

From $spec_{sp2}$ we can see that formula $\Box_{U_s}(a = v \rightarrow \Diamond_{U_1}(x = v))$ actually is independent from the clock relations between clocks U_s and U_1 . It is easy to change the time constraints between the two components without changing their local temporal properties. In this example, if U_s and U_1 are synchronized to each other, then we can change $spec_{sp2}$ by simply replacing the relation $U_s \preceq U_1$ in Φ_2 with $U_s \approx U_1$. In this case, formula $\Box_{U_s}(a = v \rightarrow \Diamond_{U_1}(x = v))$ is actually equivalent to $\Box_{U_s}(a = x)$, as U_s and U_1 always tick at the same instant.

4.2.3. Refinement of $LTL_c/CCSL$ Specifications

As mentioned in Sect.1, one of advantages of $LTL_c/CCSL$ is that it can well support the refinement of the time model of a multi-clock system.

For instance, in Sect. 4.2.1, we do not consider any time requirements in the window controller. Based on $spec_{sp1}$, we now consider a more refined time model by enriching the CCSL specification Φ_{sp1} with some clock constraints between the inputs and outputs of the window controller. For example, we require that after the triggering of each input event, there must exist some delay for, e.g. inner computations, before an output event is triggered. This actually means that the relation between the inputs and the outputs must be a precedence relation. In other words, the relation is causal, but not synchronous. Using CCSL, we can describe as follows:

$$U_1 \prec U_2, N \prec S.$$

An example of their relations is shown by the violet dashed arrows in Fig. 4.

The specification that refines $spec_{sp1}$ can thus be expressed as:

$$spec_{sp3} = \langle \Box_{U_1}(x = v \rightarrow \mathbf{X}_{U_2}(y = v)), \Phi_{sp3} \rangle,$$

where $\Phi_{sp3} = \{U_1 \prec U_2, N \prec S\}$.

In Sect. 4.2.2, based on the specification $spec_{sp2}$, we can consider a more refined time model between the three components, by introducing a more explicit connection between their local times.

For example, in Fig. 4, we define the new local clocks C_{FSM} and C_{SDF} to represent the the minimum granularity of times of the window controller and the window respectively. So all local clocks of the two components should be synchronized by C_{FSM} and C_{SDF} respectively. Thus we have the following clock constraints:

$$\Phi_0 = \{ U_1 \subseteq C_{FSM}, N \subseteq C_{FSM}, U_2 \subseteq C_{FSM}, S \subseteq C_{FSM}, E \subseteq C_{FSM}, \\ U_w \subseteq C_{SDF}, S_w \subseteq C_{SDF}, E_w \subseteq C_{SDF} \}.$$

Here we use Subclock relation to express the synchronous relations between clocks. For example, $U_1 \subseteq C_{FSM}$ means that U_1 ticks at the same instant where C_{FSM} ticks.

After introducing C_{FSM} and C_{SDF} , now we make a further refinement of the time model by defining an explicit the local times of the window controller and the window. This can be realized by building a logical relations with a clock that represents a granularity of physical time. We assume such a clock C that ticks every 2ms. In this example, we require that C_{FSM} ticks (exactly) every 5 ticks of C (i.e. every 10ms) and C_{SDF} ticks every 8 ticks of C (i.e. every 16ms). Their logical relations are described by

$$\Phi_1 = \{ C_{FSM} \approx u_1, C_{SDF} \approx u_2, u_1 \triangleq C \propto 5, u_2 \triangleq C \propto 8 \},$$

where u_1, u_2 are two new clocks. We use the relation $\approx$ (whose definition is given below Def. 3.5) to express an *exact* synchronous relation between clocks. For example, $C_{FSM} \approx u_1$ means that C_{FSM} and u_1 tick exactly at the same instants. Note that the physical time interval (2ms) of clock C is actually not important, what really matter is the logical relations between clocks.

The specification after refining the time model of $spec_{sp2}$ is thus defined as

$$spec_{sp4} = \langle \Box_{U_s}(a = v \rightarrow \Diamond_{U_1}(x = v)), \Phi_{sp4} \rangle,$$

where $\Phi_{sp4} = \Phi_{sp3} \cup \Phi_0 \cup \Phi_1$.

From this example we can see that such a connection of the time notions can be carried out at any modelling level of a system by simply adding new clock constraints into the old clock specification, without changing the form of the LTL_c formula itself. Such a separation of concerns of the properties and the time model is natural for specifying multi-clock systems.

5. Verification of $LTL_c/CCSL$ Specifications

In this section, we propose a model checking approach for verification of $LTL_c/CCSL$ specifications. Similar to the model checking technique for LTL [18], the main idea behind our method is to transform both LTL_c formulae and CCSL specifications into a type of transition systems, and then make reachability analysis on the product of these transition systems and a system model. Since CCSL specifications are not always bounded (see Sect. 5.2), it is not always possible to verify $LTL_c/CCSL$ specifications in a model checking procedure. So in this paper we only consider the verification of a subset of $LTL_c/CCSL$ specifications where the set of CCSL constraints of an LTL_c formula is bounded.

In Sect. 5.1, firstly, we begin by introducing a special transition system, called *clock transition system* (CTS), as an automata semantics for multi-clock systems. Then we introduce a more general CTS with accepting states (CTSA) for capturing the semantics of LTL_c formulae and CCSL constraints. Secondly, in Sect. 5.2, we introduce the CTSA semantics of CCSL constraints. Then we analyze the relationship between CCSL constraints and their corresponding CTSA in Sect. 5.4, according to a proposed observation theory of CCSL introduced in Sect. 5.3. Based on the preparation works in Sect. 5.1, 5.2, 5.3 and 5.4, thirdly, in Sect. 5.5, we prove the main theorem — Thm. 5.4 — for model checking of $LTL_c/CCSL$ specifications. The dependence relations between different lemmas, propositions and theorems in Sect. 5 are shown in Fig. 5. Based on Thm. 5.4, we propose a model checking algorithm — Algorithm 2 — for verifying $LTL_c/CCSL$ specifications. Because of the expressiveness of LTL_c stays in LTL, the transformation from LTL_c formulae into CTSA can be realized by the transformation from LTL formulae into Büchi automata, while the transformation from CCSL constraints into CTS is straightforward. Finally, we make an analysis to the complexity of our model checking algorithm in Sect. 5.6.

16

Yuanrui Zhang, Frédéric Mallet, Min Zhang and Zhiming Liu

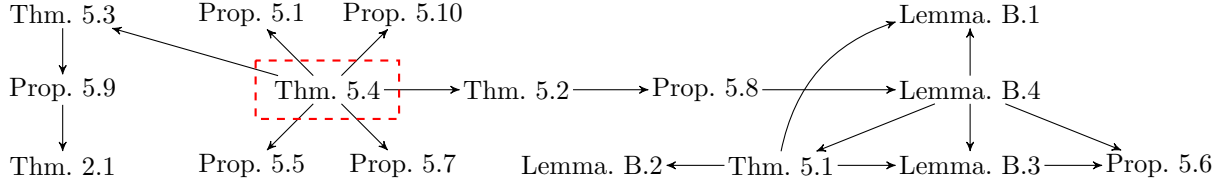


Fig. 5. Dependence relations between lemmas, propositions and theorems

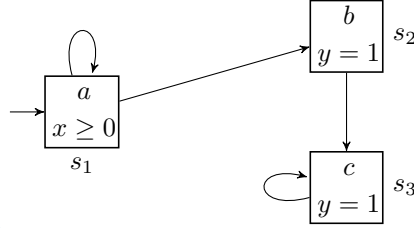


Fig. 6. An example of CTSs

5.1. Clock Transition System

We introduce a transition system, called *clock transition system* (CTS), to capture the global configurations of clocks defined in Def. 2.3. CTS enriches ordinary transition systems (abbreviated as OTS, cf. Appendix A) with states labelled by a set of clocks, which, as first-class citizens, indicate their occurrences at the current instant. Based on CTSs we introduce a more general type of transition systems, called *CTSs with accepting states* (CTSA), to capture the semantics of LTL_c formulae and CCSL constraints.

The idea of introducing clocks into transition systems was initially proposed in [17], where a clock-labelled transition system (cLTS) was defined as an automata semantics of CCSL. In cLTS, the occurrences of clocks were expressed as labels on transitions, rather than labels on states. cLTSs are equivalent to our proposed CTS in terms of expressiveness, but they have a simpler form with less states. In this paper, we use CTSs/CTSA as automata models of CCSL because they are more convenient in theoretical analysis of verification of LTL_c /CCSL specifications.

Definition 5.1 (Clock Transition System (CTS)). A transition system with clocks is a 7-tuple $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ with the following components:

1. Π is a set of general atomic propositions.
2. Clk is a set of clocks.
3. Q is a (possibly infinite) set of states.
4. I is a finite set of initial states.
5. $\theta : Q \rightarrow 2^\Pi$ maps each state to the set of all propositions that are true in the state.
6. $\xi : Q \rightarrow 2^{Clk}$ maps each state to a set of clocks that tick in the state.
7. $\mathcal{T} \subseteq Q \times Q$ is a set of transitions.

We use p to represent a general atomic proposition. A transition $\langle q, q' \rangle \in \mathcal{T}$ means a computation that changes the state q to the state q' , and during the computation all clocks in $\xi(q')$ tick. $\xi(q)$ is often called the *clock behaviour* in a state q . We often use B to denote a clock behaviour.

Fig. 6 shows a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ where $\Pi = \{x \geq 0, y = 1\}$, $Clk = \{a, b, c\}$, $Q = \{s_1, s_2, s_3\}$, $I = \{s_1\}$, θ is defined s.t. $\theta(s_1) = \{x \geq 0\}$, $\theta(s_2) = \theta(s_3) = \{y = 1\}$. ξ is defined s.t. $\xi(s_1) = \{a\}$, $\xi(s_2) = \{b\}$ and $\xi(s_3) = \{c\}$. $\mathcal{T} = \{\langle s_1, s_1 \rangle, \langle s_1, s_2 \rangle, \langle s_2, s_3 \rangle, \langle s_3, s_3 \rangle\}$.

Given a CTS $\mathcal{M}$, we often use $P_{\mathcal{M}}$, where $P \in \{\Pi, Clk, Q, I, \theta, \xi, \mathcal{T}\}$, to represent the parameter P of $\mathcal{M}$. For example, $Clk_{\mathcal{M}}$ represents the set of clocks of $\mathcal{M}$.

Definition 5.2 (Trace, Configuration and Clock Schedule of CTS). Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ be a CTS, trace $\pi = q_0 q_1 \dots q_i \dots$ is a trace of $\mathcal{M}$ is defined s.t. $\langle q_i, q_{i+1} \rangle \in \mathcal{T}$ for each $i \geq 0$.

$\langle \pi, \theta, \xi \rangle$ is a configuration of $\mathcal{M}$, σ^π is a clock schedule of $\mathcal{M}$.

A trace captures one possible behaviour of a CTS model.

We use $Trace(\mathcal{M})$ to denote the set of all traces accepted by $\mathcal{M}$.

We use $Sche(\mathcal{M})$ to denote the set of all schedules accepted by $\mathcal{M}$, it is defined as

$$Sche(\mathcal{M}) = \{\sigma \mid \exists \pi. (\sigma^\pi = \sigma \wedge \pi \in Trace(\mathcal{M}))\}.$$

For a trace π (the same for a schedule σ^π) and a position $i \geq 0$, we use $\pi[i]$, $\pi[0, i]$ and $\pi[i, \infty]$ to denote the $(i + 1)$ -th element q_i of π , the finite prefix $q_0 q_1 \dots q_i$ of π and the suffix $q_i q_{i+1} \dots$ of π respectively. Given a state $q \in Q$, we use $Trace(\mathcal{M}, q)$ (the same for $Sche(\mathcal{M}, q)$) to denote the set of all traces starting from q in $\mathcal{M}$: $Trace(\mathcal{M}, q) = \{\pi \mid \pi \in Trace(\mathcal{M}), \pi[0] = q\}$.

Definition 5.3 (Projection of Clock Schedules on CTSs). Given a schedule $\sigma = C_0 C_1 \dots C_i \dots$, the projection of σ on a CTS $\mathcal{M}$, denoted by $\sigma|_{\mathcal{M}}$, is defined as:

$$\sigma|_{\mathcal{M}} = (C_0 \cap Clk_{\mathcal{M}})(C_1 \cap Clk_{\mathcal{M}}) \dots (C_i \cap Clk_{\mathcal{M}}) \dots$$

The parallel composition describes the behaviour when considering two CTSs run in parallel. It was first introduced in [17] for cLTS, here we give a version for our proposed CTSs.

Definition 5.4 (Parallel Composition of CTSs). Given two CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2$), the parallel composition is defined as a CTS

$$\mathcal{M}_1 \parallel \mathcal{M}_2 = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle,$$

where

1. $\Pi = \Pi_1 \cup \Pi_2$;
2. $Clk = Clk_1 \cup Clk_2$;
3. $Q = \{\langle q_1, q_2 \rangle \mid q_1 \in Q_1, q_2 \in Q_2, Clk_1 \cap \xi_2(q_2) = Clk_2 \cap \xi_1(q_1), \Pi_1 \cap \theta(q_2) = \Pi_2 \cap \theta(q_1)\}$;
4. $I = \{\langle q_1, q_2 \rangle \mid q_1 \in I_1, q_2 \in I_2, \langle q_1, q_2 \rangle \in Q\}$;
5. for each $\langle q_1, q_2 \rangle \in Q$, $\theta(\langle q_1, q_2 \rangle) = \theta_1(q_1) \cup \theta_2(q_2)$;
6. for each $\langle q_1, q_2 \rangle \in Q$, $\xi(\langle q_1, q_2 \rangle) = \xi_1(q_1) \cup \xi_2(q_2)$;
7. each transition $\langle \langle q_1, q_2 \rangle, \langle q'_1, q'_2 \rangle \rangle \in \mathcal{T}$ iff
 - a) $\langle q_1, q_2 \rangle \in Q$ and $\langle q'_1, q'_2 \rangle \in Q$, and
 - b) $\langle q_1, q'_1 \rangle \in \mathcal{T}_1$ and $\langle q_2, q'_2 \rangle \in \mathcal{T}_2$.

In the parallel composition of any two CTSs, any state $\langle q_1, q_2 \rangle$ must satisfy the conditions $Clk_1 \cap \xi(q_2) = Clk_2 \cap \xi(q_1)$ and $\Pi_1 \cap \theta(q_2) = \Pi_2 \cap \theta(q_1)$. The former means that at the state two CTSs must agree on the behaviours of their common clocks (i.e., tick or not tick), while the latter means that at the state two CTSs must agree on whether an atomic proposition is true or not.

Fig. 7 shows that two CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2$), where for simplicity we assume $\Pi_1 = \Pi_2 = \emptyset$, are composed together as $\mathcal{M}_1 \parallel \mathcal{M}_2$. Each rectangle node represents a state and each arrow represents a transition. We use a state pointed by an arrow coming from outside to represent an initial state. To simplify the graphs, we use a bidirectional arrow between nodes A and B to represent both a transition from A to B and a transition from B to A . For example, in the CTS $\mathcal{M}_1$ of Fig. 7, we use a bidirectional arrow between the state s_1 and the state s_2 to represent both transitions $\langle s_1, s_2 \rangle$ and $\langle s_2, s_1 \rangle$. Each state in $\mathcal{M}_1 \parallel \mathcal{M}_2$ must satisfy the conditions in item 3. For example, the state $\langle s_3, t_3 \rangle$ satisfies that $\xi_1(s_3) \cap Clk_2 = \{c, a, b\} \cap \{d, b, c\} = \{b, c\} = \{d, b, c\} \cap \{c, a, b\} = \xi_2(t_3) \cap Clk_1$.

The parallel composition is associative, as stated in the following proposition, which is proved in Appendix B.

Proposition 5.1 (Associativity of Parallel Composition). Given three CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2, 3$),

$$(\mathcal{M}_1 \parallel \mathcal{M}_2) \parallel \mathcal{M}_3 = \mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \mathcal{M}_3).$$

By Prop. 5.1, given n CTSs $\mathcal{M}_i$ ($1 \leq i \leq n$), the product $\mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \dots \parallel (\mathcal{M}_{n-1} \parallel \mathcal{M}_n) \dots)$ can be simply written as $\mathcal{M}_1 \parallel \mathcal{M}_2 \parallel \dots \parallel \mathcal{M}_n$, or $\parallel_{1 \leq i \leq n} \mathcal{M}_i$.

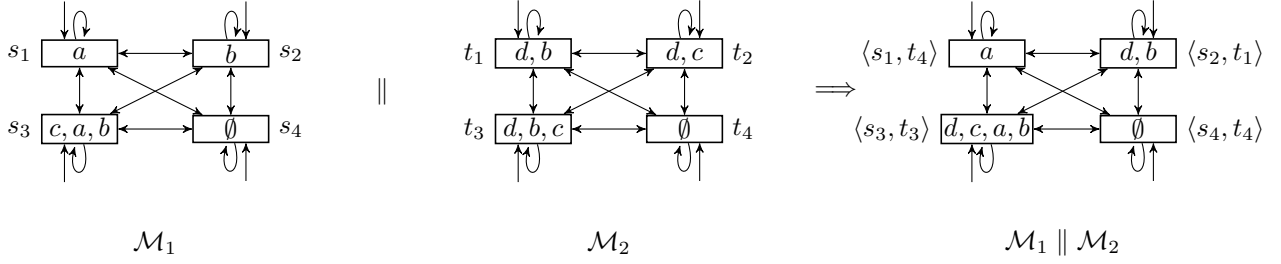


Fig. 7. An example of parallel composition

Definition 5.5 (Projection of States and Traces on CTSs). Given $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($1 \leq i \leq n$) and a product $\|_{1 \leq i \leq n} \mathcal{M}_i = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$, for any state $\langle q_1, \dots, q_n \rangle \in Q$, we define

$$\langle q_1, \dots, q_n \rangle|_{\mathcal{M}_i} = q_i (1 \leq i \leq n)$$

as the projection of the state on the CTS $\mathcal{M}_i$.

Given a trace $\pi = q_0 q_1 \dots q_i \dots \in Trace(\|_{1 \leq i \leq n} \mathcal{M}_i)$, the projection of π on $\mathcal{M}_j$ ($1 \leq j \leq n$) is defined as

$$\pi|_{\mathcal{M}_j} = (q_0|_{\mathcal{M}_j})(q_1|_{\mathcal{M}_j})(q_i|_{\mathcal{M}_j}) \dots$$

For example, in the CTS $\mathcal{M}_1 \parallel \mathcal{M}_2$ above in Fig. 7, let $\pi = \langle s_1, t_4 \rangle \langle s_1, t_4 \rangle \langle s_1, t_4 \rangle \langle s_4, t_4 \rangle \langle s_3, t_3 \rangle \dots$ be a trace of $\mathcal{M}_1 \parallel \mathcal{M}_2$, then its projection on $\mathcal{M}_1$ is $\pi|_{\mathcal{M}_1} = s_1 s_1 s_1 s_4 s_3 \dots$.

For a trace of the parallel composition of several CTSs, it is natural to think that the projection of the trace on one of these CTSs should also be one of its traces. The next proposition states this property.

Proposition 5.2. Given n CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($1 \leq i \leq n$), for any trace π ,

$$\text{if } \pi \in Trace(\|_{1 \leq i \leq n} \mathcal{M}_i), \text{ then } \pi|_{\mathcal{M}_1} \in Trace(\mathcal{M}_1), \dots, \pi|_{\mathcal{M}_n} \in Trace(\mathcal{M}_n).$$

Proof. Obviously from Def. 5.4. $\square$

With CTS we can introduce the satisfaction relation between CTSs and LTL_c formulae.

Definition 5.6 (Satisfaction Relation between CTSs and LTL_c Formulae). Given a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ and an LTL_c formula ϕ , for any trace $\pi \in Trace(\mathcal{M})$, we define $\pi \models_{\mathcal{M}} \phi$ as:

$$\pi \models_{\mathcal{M}} \phi \text{ iff } \pi \models_{\theta, \xi} \phi,$$

where θ and ξ appear as the parameters of $\mathcal{M}$.

For any state $q \in Q$, $q \models_{\mathcal{M}} \phi$ is defined s.t.

$$\text{for any trace } \pi \in Trace(\mathcal{M}, q), \pi \models_{\mathcal{M}} \phi.$$

In Def. 5.6, we define that a trace π satisfies an LTL_c formula ϕ in a CTS $\mathcal{M}$ as the satisfaction relation between the trace π , as the trace of the configuration $\langle \pi, \theta, \xi \rangle$, and the formula ϕ , which has been defined in Def. 2.4.

In verification of $LTL_c/CCSL$ specifications, when introducing the automata semantics of CCSL constraints (in Sect. 5.2) and of LTL_c formulae (in Sect. 5.5.1) below, we need a type of more general CTSs with accepting states to allow CTSs to reject a certain set of traces. We introduce *CTSs with accepting states* (abbreviated as CTSA) as follows.

Definition 5.7 (Clock Transition Systems with Accepting States). A clock transition system with accepting states (CTSA) extends a CTS with a set of accepting states $\mathcal{E}$. It is defined as an 8-tuple

$$\mathcal{M}(\mathcal{E}) = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle, \text{ where } \mathcal{E} \subseteq Q.$$

A trace $\pi = q_0 q_1 \dots q_i \dots$ is accepted by $\mathcal{M}(\mathcal{E})$ iff

$$\text{there exists an infinite number of } i \geq 0 \text{ s.t. } q_i \in \mathcal{E}.$$

Similar to CTSs, we denote the set of all traces accepted by a CTSA $\mathcal{M}(\mathcal{E})$ as $Trace(\mathcal{M}(\mathcal{E}))$. And we define the set of all schedules accepted by $\mathcal{M}(\mathcal{E})$ as: $Sche(\mathcal{M}(\mathcal{E})) = \{\sigma \mid \exists \pi. (\sigma^\pi = \sigma \wedge \pi \in Trace(\mathcal{M}(\mathcal{E})))\}$.

When no confusion occurs, sometimes we simply write a CTSA $\mathcal{M}(\mathcal{E})$ as $\mathcal{M}$ to hide $\mathcal{E}$.

Any CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \mathcal{T} \rangle$ can be taken as a special CTSA with all states are accepting states, i.e., $\mathcal{M} = \mathcal{M}(Q) = \langle \Pi, Clk, Q, I, \theta, \mathcal{T}, Q \rangle$. Therefore, sometimes we do not distinguish CTSs and CSAAs and also call a CTS a “CTSA”.

Definition 5.8 (Or-Parallel-Composition of CSAAs). Given two CSAAs $\mathcal{M}_i(\mathcal{E}_i) = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$), we define the product

$$\mathcal{M}(\mathcal{E}) = \mathcal{M}_1(\mathcal{E}_1) \parallel \mathcal{M}_2(\mathcal{E}_2)$$

just as in Def. 5.4, but with the accepting states $\mathcal{E}$ defined as:

$$\mathcal{E} = \{ \langle q_1, q_2 \rangle \mid q_1 \in \mathcal{E}_1 \text{ or } q_2 \in \mathcal{E}_2 \}.$$

We abuse the symbol $\parallel$, which is used in Def. 5.4 above, for the composition between CSAAs.

Def. 5.8 is a general version for Def. 5.4 since CTSs are special CSAAs. It is called “Or-composition” because we only require that a trace of the composition passes the accepting states of one of its components. So Prop. 5.2 does not hold for the composition between CSAAs. Actually, it is easy to see that for any trace $\pi \in \text{Trace}(\mathcal{M}(\mathcal{E}))$, $\pi|_{\mathcal{M}_1(\mathcal{E}_1)} \in \text{Trace}(\mathcal{M}_1(\mathcal{E}_1))$ or $\pi|_{\mathcal{M}_2(\mathcal{E}_2)} \in \text{Trace}(\mathcal{M}_2(\mathcal{E}_2))$ holds.

5.2. CTSA Semantics of Clock Constraints

We introduce the CTSA semantics for clock constraints in CCSL. Each clock constraint corresponds to a CTSA which exactly captures its clock behaviour. An automata semantics of CCSL were given in [17], where it is based on cLTSs. Here we give an automata semantics of CCSL based on CSAAs.

When discussing model checking of $LTL_c/CCSL$ specifications in Sect. 5.5, we also need to introduce the CTSA semantics for the negations of clock relations. The CTSA of the negation of a clock relation refuses all traces whose schedules satisfy this clock relation.

Definition 5.9 (CTSA Semantics of Clock Constraints). CTSA Semantics of CCSL constraints (including the negations of clock relations) is given in Table 3 below and Table 4 in Appendix C.

Given a clock constraint Cn^+ , we often denote its CTSA as $\mathcal{M}^{Cn^+}$.

Table 3 gives the CTSA semantics of each clock relation and its negation. We use a rectangle with a double border to represent an accepting state. For example, in the CTSA $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$, the states $(s_0, \emptyset)$, (s_0, c_1) and (s_0, c_1, c_2) are accepting states, and the state (s_0, c_2) is both an initial state and an accepting state. As explained below Def. 5.4 in Sect. 5.1, for simplicity we use a bidirectional arrow between nodes A and B to represent both a transition from A to B and a transition from B to A . All states of any CTSA in Table 3 are only associated to sets of clocks, in other words, for any CTSA $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$, $\Pi = \emptyset$. Each state is denoted in the form $(T, c_1, \dots, c_k)$ if the set of clocks associated to the state is $\{c_1, \dots, c_k\}$. T is a tag that classifies the states into different categories in order to make the CSAAs easier to understand, as is explained below.

The automata of all clock relations are in fact CTSs, so we do not draw their accepting states. The difference between the automata of a clock relation and its negation is that those states of the CTS of a clock relation are not accepting states in the CTSA of the negation of the clock relation. The latter contains some extra accepting states, tagged by s_0 , that the former does not have. A trace accepted by the CTS of a clock relation is just rejected by the CTSA of the negation of the clock relation, and vice versa. For example, trace $\pi = (s, \emptyset)(s, c_2)(s, c_1, c_2)(s, c_2)(s, c_1, c_2)(s, c_2)(s, c_1, c_2) \dots$, which is accepted by the CTS $\mathcal{M}^{c_1 \subseteq c_2}$, is rejected by the CTSA $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$ since it never passes any accepting states of $\mathcal{M}^{c_1 \subseteq c_2}$. On the other hand, trace $\pi' = (s, \emptyset)(s, c_1)(s, c_1, c_2)(s_0, c_2)(s_0, \emptyset)(s_0, \emptyset)(s_0, \emptyset) \dots$, which is accepted by the CTSA of the negation of $c_1 \subseteq c_2$, is rejected by the CTS $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$, since once it passes one of the accepting states tagged by s_0 , it can never return to the states not tagged by s_0 .

In the CSAAs of some clock constraints, e.g., $c_1 \prec c_2$, $\sim(c_1 \prec c_2)$, $c_1 \preceq c_2$ and $\sim(c_1 \preceq c_2)$, the states tagged by different names, such as $s_1, s_2, \dots$, have different indications on the differences between the numbers of the ticks of clocks. For example, in $\mathcal{M}^{c_1 \prec c_2}$, the states tagged by different names (drawn in different dashed red rectangles) indicate different differences between the numbers of the ticks of the clocks c_1 and c_2 , denoted by $h(c_1, c_2)$. When a trace of $\mathcal{M}^{c_1 \prec c_2}$ passes any states with the same name, the values of $h(c_1, c_2)$ are always the same. For example, given a set of clock definitions $\mathbf{D}$, let $\pi = (s_2, c_1)(s_3, c_1)(s_2, c_2)(s_3, c_1)(s_2, c_2)(s_3, c_1) \dots$

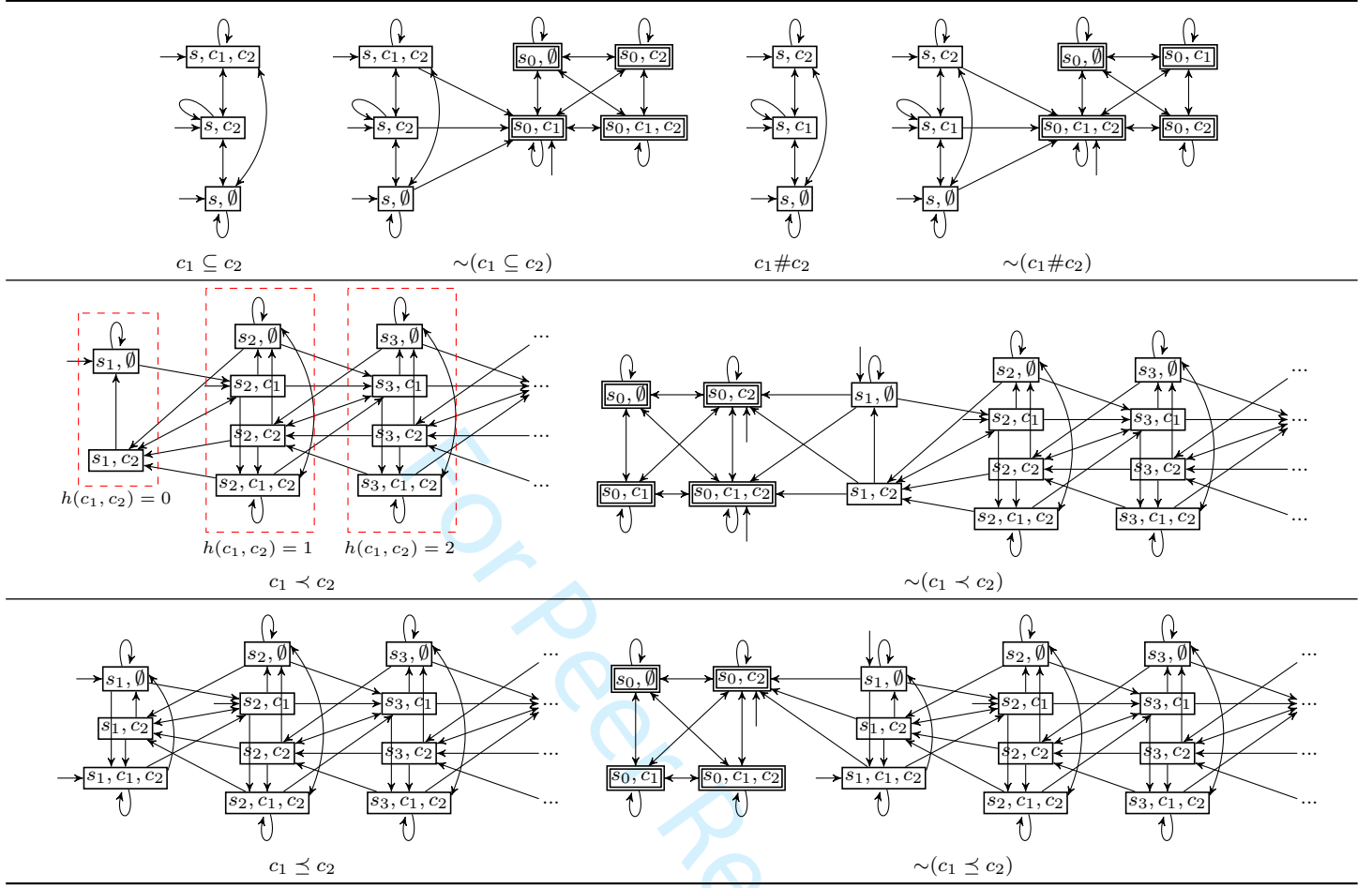


Table 3. CTSA semantics of clock relations and their negations

be a trace accepted by $\mathcal{M}^{c_1 < c_2}$, each time when the trace π passes the states tagged with s_2 (when $i = 0, 2, 4, 6, \dots$), we have $|\delta^{\sigma^\pi, \mathbf{D}}(k, c_1) - \delta^{\sigma^\pi, \mathbf{D}}(k, c_2)| = h(c_1, c_2) = 1$.

Table 4 (in Appendix C) gives the CTSA semantics of clock definitions. All automata in Table 4 are CTSs so we omit their accepting states. Their explanation is similar to that of the CTSAs of clock relations and their negations given here and we omit the details.

With the CTSA semantics for each constraint, we define the CTSA semantics for a set of constraints.

Definition 5.10 (CTSA Semantics of a Set of Clock Constraints). Given a set of clock constraints Φ , its CTSA semantics is defined as the parallel composition of the CTSAs of all constraints in Φ , i.e.,

$$\parallel_{Cn^+ \in \Phi} \mathcal{M}^{Cn^+}.$$

Sometimes, we also denote the CTSA of a set of constraints Φ by $\mathcal{M}^\Phi$. For example, given $\mathcal{M}^{c_1 < c_2}$ and $\mathcal{M}^{c_1 \leq c_2}$ shown in Table 3, then $\mathcal{M}^{c_1 < c_2} \parallel \mathcal{M}^{c_1 \leq c_2}$ is the CTS of the clock constraints $\{c_1 < c_2, c_1 \leq c_2\}$, denoted by $\mathcal{M}^{\{c_1 < c_2, c_1 \leq c_2\}}$.

From Table 3 and 4 we can see that the CTSAs of some clock constraints, e.g., $c_1 < c_2$, have an infinite number of states. Following [23], we call a constraint (resp. a set of constraints) whose corresponding CTSA has a finite number of states as a *bounded constraint* (resp. a set of *bounded constraints*). Though the CTSAs of some constraints may be infinite, the parallel compositions of infinite CTSAs are not necessarily infinite. Thus a set of constraints might be bounded even some of the constraints are unbounded. One typical example

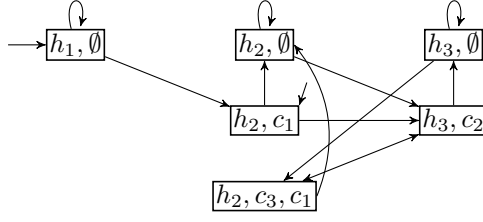


Fig. 8. The CTS of clock constraints $\{c_1 < c_2, c_2 < c_3, c_3 = c_1 \$ 1\}$

is that $\mathcal{M}^{\{c_1 < c_2, c_3 \triangleq c_1 \$ 1, c_2 < c_3\}}$ (shown in Fig. 8) is finite while $\mathcal{M}^{c_1 < c_2}$ and $\mathcal{M}^{c_2 < c_3}$ are infinite (see Table 3). More details about bounded CCSL constraints can be found in [23].

It is natural to think that there exists a correspondence relationship between the CTSA semantics of clock constraints given in this section and the schedule-based semantics given in Sect. 3.1. Intuitively, as indicated in the above analysis, for clock relations, when $\mathbf{D} = \emptyset$, we can see that the schedules of a clock relation or its negation exactly correspond to the traces of its CTSA. The situation when $\mathbf{D} \neq \emptyset$ is more complex. More details about their relation is fully analyzed in Sect. 5.4.

5.3. Observation Theory of CCSL

An *observer* “observes” any behaviour of its environment and behaves accordingly. In the traditional model checking theory of LTL [18], to verify that a system model $\mathcal{M}$ satisfies an LTL formula ϕ , reachability analysis is made on the product $\mathcal{M} \parallel \mathcal{M}^{\neg\phi}$ of $\mathcal{M}$ and the model $\mathcal{M}^{\neg\phi}$ of the LTL formula $\neg\phi$. In $\mathcal{M} \parallel \mathcal{M}^{\neg\phi}$, actually there is an implicit relation between $\mathcal{M}$ and $\mathcal{M}^{\neg\phi}$, that any behaviour of $\mathcal{M}$ must be observed by $\mathcal{M}^{\neg\phi}$ in the sense that $\mathcal{M}^{\neg\phi}$ can proceed given any trace of $\mathcal{M}$. We call this relation an *observation relation* in our context in terms of CTSs, which is formally stated as follows.

Definition 5.11 (Observation Relation). Given two automata $\mathcal{M}_1$ and $\mathcal{M}_2$, we say $\mathcal{M}_2$ *observes* $\mathcal{M}_1$, abbreviated as $\mathcal{M}_2 \text{ obs } \mathcal{M}_1$, if for any trace $\pi \in \text{Trace}(\mathcal{M}_1)$,

there exists a trace $\pi' \in \text{Trace}(\mathcal{M}_1 \parallel \mathcal{M}_2)$ s.t. $\pi'|_{\mathcal{M}_1} = \pi$.

Note that the automata we mean in Def. 5.11 is general and can be either OTSs/Büchi automata or CTSs/CTSAs.

In the model checking theory of LTL, the observation relation $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}$ is always guaranteed by the transformation from the LTL formula $\neg\phi$ into the Büchi automaton $\mathcal{M}^{\neg\phi}$. However, since the transformation from CCSL constraints into CTSAs is not based on that of LTL (and neither CCSL and LTL have the same expressiveness), we need to prove that the observation relation also holds between a system model and the CTSA of a set of CCSL constraints.

To this end, in this subsection, we propose an observation theory for CCSL. Our goal is to study how the CTSA of a clock constraint can be an observer, how a composed observer is formed and under what conditions the observation relation between CTSAs can be defined. The proposed theory is necessary in analysis of the relation between clock constraints and their CTSAs in Sect. 5.4 and of the proof of the model checking theory of *LTL_c/CCSL* specifications in Sect. 5.5.

While studying the observation relation in CCSL below, we only focus on the clock behaviours of CTSAs since in the CTSA of a clock constraint, the set of atomic propositions is always empty so that the parallel composition of two CTSAs is only determined by their clock behaviours.

We begin by firstly defining what is an observer. Intuitively, in CCSL, an observer over a set of clocks A is a special CTS/CTSA such that at any instant, no matter what a clock behaviour $B \subseteq A$ is offered by the environment, it can always proceed by agreeing on B .

Definition 5.12 (Observer). Let $\mathcal{M} = \langle \Pi, \text{Cclk}, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA with $\Pi = \emptyset$. We call $\mathcal{M}$ an observer of a given clock behaviour $F \subseteq \text{Cclk}$ if for any clock behaviour $X \subseteq F$ ($X \neq \emptyset$),

1. there exists an initial state $i \in I$ s.t. $\xi(i) \cap F = X$, and
2. for any $q \in Q$, there exists a transition $\langle q, q' \rangle \in \mathcal{T}$ s.t. $\xi(q') \cap F = X$.

An observer is denoted as $\mathcal{M}[F, C]$, with $C = Clk - F$.

We often call F the set of *free clocks*, and call C the set of *constrained clocks* of an observer $\mathcal{M}[F, C]$.

Sometimes we use $F_{\mathcal{M}}$, $C_{\mathcal{M}}$ to represent the sets of free clocks and constrained clocks respectively of an observer $\mathcal{M}$ respectively.

In Def. 5.12, the observation made by an observer is reflected by the inclusion relation $X \subseteq \xi(i)$ (resp. $X \subseteq \xi(q')$). Intuitively, it means that for any clock behaviour $X \subseteq F$ offered by the environment, the observer $\mathcal{M}$ agrees on it by offering $\xi(i)$ (resp. $\xi(q')$). It is possible that $X \subset \xi(i)$ (resp. $X \subset \xi(q')$), which means that set $\xi(i)$ (resp. $\xi(q')$) might include other clocks not in X (and F).

As an example, $\mathcal{M}^{\sim(c_1 \prec c_2)}$ (see Table 3) is an observer of the clock behaviour $\{c_1, c_2\}$. At any state, if a clock behaviour, e.g. $X = \{c_1\} \subseteq \{c_1, c_2\}$, is offered, $\mathcal{M}^{\sim(c_1 \prec c_2)}$ can always proceed by offering $\{c_1\}$ or $\{c_1, c_2\}$. In a state of $\mathcal{M}^{\sim(c_1 \prec c_2)}$, e.g. $(s_1, \emptyset)$, there exists a transition $\langle (s_1, \emptyset), (s_2, \{c_1\}) \rangle$ with $\xi((s_2, \{c_1\})) \cap \{c_1, c_2\} = \{c_1\} \cap \{c_1, c_2\} = \{c_1\} = X$. The CTSA $\mathcal{M}^{c \triangleleft c_1 + c_2}$ (see Table 4 in Appendix C) is an observer with the set of free clocks $\{c_1, c_2\}$ and the set of constrained clocks $\{c\}$. At any state, it observes any subset of $\{c_1, c_2\}$ and triggers the clock c accordingly. For example, in the state $(s, \{c, c_1\})$, for the clock behaviour X , there exists a transition $\langle (s, \{c, c_1\}), (s, \{c, c_1\}) \rangle$ with $\xi((s, \{c, c_1\})) \cap \{c_1, c_2\} = \{c, c_1\} \cap \{c_1, c_2\} = \{c_1\} = X$.

Proposition 5.3. For the negation of any clock relation Rel , its CTSA $\mathcal{M}^{\sim Rel}$ is an observer of the set of free clocks $\mathcal{C}(Rel)$ (with the set of constrained clocks $\emptyset$). For any clock definition $c \triangleleft e$, its CTSA $\mathcal{M}^{c \triangleleft e}$ is an observer of the set of free clocks $\mathcal{C}(e)$ (with the set of constrained clocks $\{c\}$).

Prop. 5.3 is straightforward according to the CTSA semantics of clock constraints given in Sect. 5.2. We have seen the examples $\mathcal{M}^{\sim(c_1 \prec c_2)}$ and $\mathcal{M}^{c \triangleleft c_1 + c_2}$ as two observers above.

Note that for any clock relation Rel , $\mathcal{M}^{Rel}[\mathcal{C}(Rel), \emptyset]$ is not an observer. For example, in $\mathcal{M}^{c_1 \prec c_2}$ (see Table 3), for the state $(s_1, \emptyset)$ and the clock behaviour $X = \{c_2\}$, there exists no transition $\langle (s_1, \emptyset), q \rangle$ such that $\xi(q) \cap \mathcal{C}(c_1 \prec c_2) = \{c_2\} = X$.

Following the dependence relation between clock constraints defined in Def. 3.2 in Sect. 3.1, we can also define a similar dependence relation between observers.

Definition 5.13 (Dependence Relation between Observers). Given two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E} \rangle$ ($i = 1, 2$), we say $\mathcal{M}_1$ *directly depends on* $\mathcal{M}_2$, denoted as $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$, iff $C_2 \cap Clk_1 \neq \emptyset$. Similarly in Def. 3.2, we use $\rightsquigarrow^+$ to denote the transitive closure of $\rightsquigarrow$. We say $\mathcal{M}_1$ *depends on* $\mathcal{M}_2$ if $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}_2$.

Sometimes, we also use $\mathcal{M}_1 \rightsquigarrow\!\!\rightsquigarrow \mathcal{M}_2$ to represent both $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}_2$ and $\mathcal{M}_2 \rightsquigarrow^+ \mathcal{M}_1$, and use $\mathcal{M}_1 \uparrow\!\!\uparrow \mathcal{M}_2$ to express that $\mathcal{M}_1$ and $\mathcal{M}_2$ are independent from each other.

A set of observers A is *well defined* if in A there is no two observers that are dependent on each other.

$\mathcal{M}_1$ *only depends on* $\mathcal{M}_2$, denoted by $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$, is defined such that $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$ and there is no other $\mathcal{M}$ s.t. $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}$ and $\mathcal{M} \rightsquigarrow^+ \mathcal{M}_2$.

The next proposition shows that for a subset of clock constraints and their corresponding observers, the dependence relation defined on them (in Def. 3.2 and Def. 5.13 respectively) are equivalent.

Proposition 5.4. (i) For any clock constraints Cn_1^+ and Cn_2^+ , which are either the negations of some clock relations or clock definitions, $Cn_1^+ \rightsquigarrow Cn_2^+$ iff $\mathcal{M}^{Cn_1^+} \rightsquigarrow \mathcal{M}^{Cn_2^+}$.

(ii) A set of clock constraint Φ is well defined iff $\bigcup_{Cn^+ \in \Phi} \{\mathcal{M}^{Cn^+}\}$ is well defined.

We omit the proof of Prop. 5.4. Prop. 5.4 (i) can be easily proved by analyzing all CCSL constraints defined in Def. 3.1. Prop. 5.4 (ii) is a direct result of Prop. 5.4 (i) by Def.3.3 and Def. 5.13.

Def. 5.12 defines an observer that observes a certain set of clocks from its environment. Now considering environment as a CTSA, we describe the relation that an observer can observe any clock behaviour of the CTSA.

Definition 5.14 (Observation Relation between an Observer and a CTSA). Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA and $\mathcal{M}'[F', C'] = \langle \Pi', Clk', Q', I', \theta', \xi', \mathcal{T}', \mathcal{E}' \rangle$ be an observer, $\mathcal{M}'$ *observes* $\mathcal{M}$ *through* F' , abbreviated as $\mathcal{M}' \text{ obst}_{F'} \mathcal{M}$ (or simply $\mathcal{M}' \text{ obst } \mathcal{M}$), is defined s.t.

$$Clk \cap Clk' \subseteq F'.$$

The condition $Clk \cap Clk' \subseteq F'$ means that the common clock behaviours of $\mathcal{M}$ and $\mathcal{M}'$ must be included in the set of the free clocks of $\mathcal{M}'$, that is, F' . Intuitively, when this condition is satisfied, we can guarantee that $\mathcal{M}'$ can observe any clock behaviour of $\mathcal{M}$ in the sense of Def. 5.12.

For example, given two observers $\mathcal{M}_1[F_1, C_1]$ and $\mathcal{M}_2[F_2, C_2]$ of the definitions $c \triangleq a * b$ and $d \triangleq b * c$ respectively as shown in Fig. 7, we have $F_1 = \{a, b\}$, $C_1 = \{c\}$, $F_2 = \{b, c\}$ and $C_2 = \{d\}$. Since $Clk_1 \cap Clk_2 = \{c, a, b\} \cap \{d, b, c\} = \{b, c\} \subseteq F_2$, so $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$. As shown in Fig. 7, at any state of $\mathcal{M}_2$, no matter what the clock behaviour of $\mathcal{M}_1$ is, one of the two conditions in Def. 5.12 holds. At the state t_2 of $\mathcal{M}_2$, for instance, for a clock behaviour $\{c, a, b\}$ of $\mathcal{M}_1$, there is a transition $\langle t_2, t_3 \rangle$ s.t. $\xi_2(t_3) \cap F_2 = \{c, a, b\} \cap F_2$ holds.

Since for any observers, its set of atomic propositions is empty, the parallel composition of an observer and a CTSA is totally determined by their clock behaviours. Therefore, it is not hard to see that the relation defined in Def. 5.14 actually implies the observation relation defined in Def. 5.11. We have the following proposition.

Proposition 5.5. Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA and $\mathcal{M}'[F', C'] = \langle \Pi', Clk', Q', I', \theta', \xi', \mathcal{T}', \mathcal{E}' \rangle$ be an observer,

if $\mathcal{M}' \text{ obst } \mathcal{M}$, then $\mathcal{M}' \text{ obs } \mathcal{M}$.

For example, for the two observers $\mathcal{M}_1$ and $\mathcal{M}_2$ shown in Fig. 7, we have $\mathcal{M}_2 \text{ obs } \mathcal{M}_1$, so $\mathcal{M}_2$ can proceed given any trace of $\mathcal{M}_1$. For example, for a trace $\pi = s_1 s_4 s_2 s_3 s_3 s_3 \dots$ of $\mathcal{M}_1$, we have a trace $\pi_0 = \langle s_1, t_4 \rangle \langle s_4, t_4 \rangle \langle s_2, t_1 \rangle \langle s_3, t_3 \rangle \langle s_3, t_3 \rangle \langle s_3, t_3 \rangle \dots$ s.t. $\pi_0|_{\mathcal{M}_1} = \pi$.

Proof of Prop. 5.5 To prove $\mathcal{M}' \text{ obs } \mathcal{M}$, it is enough to show that for any trace $\pi \in \text{Trace}(\mathcal{M})$,

- (1) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}'$ s.t. $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}'$;
- (2) for any transition $\langle q, q' \rangle$ of the trace π and any state x of $\mathcal{M}'$ s.t. $\langle q, x \rangle$ is a state of $\mathcal{M} \parallel \mathcal{M}'$, there exists a transition $\langle \langle q, x \rangle, s \rangle$ of $\mathcal{M} \parallel \mathcal{M}'$.

(1): Let $X_0 = \xi(\pi[0]) \cap Clk'$, since $Clk \cap Clk' \subseteq F'$, it is clear that $X_0 \subseteq F'$. By Def. 5.12 there exists an initial state $x_0 \in I'$ s.t. $\xi'(x_0) \cap F' = X_0$. Since $\Pi' = \emptyset$, next we only need to show that $Clk \cap \xi'(x_0) = Clk' \cap \xi(\pi[0])$:

On one hand, for any $t \in Clk \cap \xi'(x_0)$, since $\xi'(x_0) \subseteq Clk'$, we have $t \in Clk \cap \xi'(x_0) \subseteq Clk \cap Clk' \subseteq F'$, so $t \in X_0 = \xi(\pi[0]) \cap Clk'$. On the other hand, for any $t \in Clk' \cap \xi(\pi[0])$, $t \in X_0 = \xi'(x_0) \cap F'$. Since $\xi(\pi[0]) \subseteq Clk$, we have $t \in Clk$, therefore $t \in Clk \cap \xi'(x_0)$.

According to Def. 5.4, $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}'$.

(2): Let $X = F' \cap \xi(q')$, since $X \subseteq F'$, in $\mathcal{M}'[F', C']$ by Def. 5.12 for any state $x \in Q'$ there exists a transition $\langle x, x' \rangle$ s.t. $\xi'(x') \cap F' = X$. Next we show that $Clk \cap \xi'(x') = Clk' \cap \xi(q')$:

On one hand, since $\xi'(x') \subseteq Clk'$, so $Clk \cap \xi'(x') = Clk \cap Clk' \cap \xi'(x') \subseteq F' \cap \xi'(x') = X$. Because $X = F' \cap \xi(q')$ and $\xi(q') \subseteq Clk$, so $X \subseteq Clk$. From $F' \cap \xi'(x') = X$ we have $X \subseteq Clk \cap \xi'(x')$. So $Clk \cap \xi'(x') = X$. On the other hand, since $\xi(q') \subseteq Clk$, so $Clk' \cap \xi(q') = Clk' \cap Clk \cap \xi(q') \subseteq F' \cap \xi(q') = X$. But $F' \subseteq Clk'$, so we immediately get $X \subseteq Clk' \cap \xi(q')$. Thus $Clk' \cap \xi(q') = X$.

According to Def. 5.4, $\langle q', x' \rangle$ is a state of $\mathcal{M} \parallel \mathcal{M}'$, hence $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathcal{M} \parallel \mathcal{M}'$. $\square$

When composing two observers by parallel composition, we need to know what the composed CTSA, as an observer, is like, and what are its sets of free clocks and constrained clocks. This is important when we try to make the parallel composition of the CTSAs of several clock constraints.

For example, consider the two observers $\mathcal{M}_1[F_1, C_1]$ and $\mathcal{M}_2[F_2, C_2]$ of Fig. 7 where we have seen that $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, it is easy to see that their parallel composition $(\mathcal{M}_1 \parallel \mathcal{M}_2)[F, C]$ (shown in Fig. 7) is an observer with $F = F_1 \cup F_2 - (C_1 \cap F_2) = \{a, b\} \cup \{b, c\} - (\{c\} \cap \{b, c\}) = \{a, b\}$ and $C = C_1 \cup C_2 = \{c, d\}$.

We have the following proposition.

Proposition 5.6. Given two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$), if $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, then the product $\mathcal{M}[F, C] = \mathcal{M}_1[F_1, C_1] \parallel \mathcal{M}_2[F_2, C_2]$ is an observer, with

$$\begin{aligned} F &= (F_1 \cup F_2) - (C_1 \cap F_2) \\ &= (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2)), \text{ and} \\ C &= C_1 \cup C_2. \end{aligned}$$

The proof of Prop. 5.6 is given in Appendix B. The main idea behind the proof is that for any clock behaviour $X \subseteq F$, we need to find two suitable clock behaviours, i.e., $\xi(x')$ and $\xi(y')$ in each of the observers

$\mathcal{M}_1$ and $\mathcal{M}_2$ respectively, so that $\xi(\langle x', y' \rangle) = \xi(x') \cup \xi(y')$ is a suitable clock behaviour in $\mathcal{M}$ that observes X in the sense of Def. 5.12.

The result in Prop. 5.6 can be extended to the case for multiple observers of clock constraints, we have the following proposition.

Theorem 5.1. Given a finite set of well-defined clock constraints Φ , we have $\mathcal{M}[F, C] = \parallel_{Cn^+ \in \sim \Phi} \mathcal{M}^{Cn^+}[F_{Cn^+}, C_{Cn^+}]$ is an observer, with

$$F = \bigcup_{Cn^+ \in \sim \Phi} F_{Cn^+} - \left(\bigcup_{Cn^+ \in \sim \Phi} F_{Cn^+} \cap \bigcup_{Cn^+ \in \sim \Phi} C_{Cn^+} \right) \text{ and}$$

$$C = \bigcup_{Cn^+ \in \sim \Phi} C_{Cn^+}.$$

Let $A^{\sim \Phi}$ be the set of the CTSA of all constraints in $\sim \Phi$, the proof can be proceed by induction on the size of $A^{\sim \Phi}$. We omit it here and one can refer to Appendix B for more details. The main idea behind the proof is that in the well-defined set $A^{\sim \Phi}$, we continuously compose the CTSA of constraints in a special order so that the composed CTSA can be easily proved to have an observation relationship with the CTSA we are about to compose.

Prop. 5.6 and Thm. 5.1 are necessary for proving Prop. 5.8 in Sect. 5.4.

5.4. Relation between Clock Constraints and their CTSA

After introducing the observation theory of CCSL in Sect. 5.3, we now turn to focus on analysis of the relation between clock constraints and their corresponding CTSA. It forms the basic for the model checking theory proposed in Sect. 5.5. First we consider the case when no clock definitions are involved (Prop. 5.7), then we extend the result to the general case (Prop. 5.8).

Proposition 5.7 (Relation between Clock Relations and their CTSA). Given a clock schedule σ and a clock relation Cn ,

$$\sigma \models Cn \text{ iff } \sigma|_{\mathcal{M}^{Cn}} \in \text{Sche}(\mathcal{M}^{Cn}) \text{ iff } \sigma|_{\mathcal{M}^{\sim Cn}} \notin \text{Sche}(\mathcal{M}^{\sim Cn}).$$

Prop. 5.7 is trivial according to the CTSA semantics of clock relations given in Table. 3, by considering the CTSA of all 4 types of clock relations: $c_1 \subseteq c_2$, $c_1 \# c_2$, $c_1 \prec c_2$ and $c_1 \preceq c_2$ and their negations: $\sim(c_1 \subseteq c_2)$, $\sim(c_1 \# c_2)$, $\sim(c_1 \prec c_2)$ and $\sim(c_1 \preceq c_2)$. We omit the details in this paper.

When considering a satisfaction relation with a non-empty set of clock definitions as the condition, i.e., the satisfaction relation of the form $\sigma \models Cn, \mathbf{D}$ (where $\mathbf{D} \neq \emptyset$), the situation is a bit complex, because the clocks in Cn may not be pure and may come from $\mathbf{D}$. For any schedule σ that satisfies $\sigma \models Cn, \mathbf{D}$, it not only relates to the schedules of $\mathcal{M}^{Cn}$ (and $\mathcal{M}^{\sim Cn}$), but also to those of $\parallel_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$.

Based on the observation theory built in Sect. 5.3, first we introduce the notion of observation relation between schedules.

Definition 5.15 (Observation Relation between Schedules). Given a schedule $\sigma = B_0 B_1 \dots B_i \dots$ of an observer $\mathcal{M}[F, C]$, for any schedule $\sigma' = B'_0 B'_1 \dots B'_i \dots$, σ observes σ' in $\mathcal{M}[F, C]$, abbreviated as $\sigma \text{ obs}_{\mathcal{M}[F, C]} \sigma'$ (or simply $\sigma \text{ obs } \sigma'$), is defined s.t.

$$\text{for any } i \geq 0, B_i \cap F = B'_i \cap F.$$

Def. 5.15 says that at each time, the part of a clock behaviour of the schedule σ' that belongs to the set of free clocks of the observer $\mathcal{M}$ (i.e. $B'_i \cap F$) is observed by a clock behaviour of the schedule σ (i.e. B_i) in the sense of Def. 5.12.

For example, in the observer $\mathcal{M}_2$ of Fig. 7, $\sigma = \emptyset \{d, b\} \{d, b, c\} \{d, b, c\} \{d, b, c\} \dots$ is a schedule of $\mathcal{M}_2$. It is easy to see that the schedule σ observes the schedule $\sigma' = \{a\} \emptyset \{b\} \{c, a, b\} \{c, a, b\} \{c, a, b\} \dots$ of $\mathcal{M}_1$, i.e., $\sigma \text{ obs}_{\mathcal{M}_2} \sigma'$. We have, for example, $\sigma[0] \cap F_2 = \emptyset \cap \{b, c\} = \emptyset = \{a\} \cap \{b, c\} = \sigma'[0] \cap F_2$, and $\sigma[3] \cap F_2 = \{d, b, c\} \cap \{b, c\} = \{b, c\} = \{c, a, b\} \cap \{b, c\} = \sigma'[3] \cap F_2$.

Given a clock behaviour B , we use $\text{pure}_{\Phi}(B)$ to denote the set of pure clocks of B that appear in the set of clock constraints Φ . When Φ is clear in the context, we often omit it and simply write $\text{pure}(B)$. We lift the

function **pure** and the union operator $\cup$ to sequences of clock behaviours: for any schedule $\sigma_1 = B_1 B_2 \dots B_i \dots$ and $\sigma_2 = B'_1 B'_2 \dots B'_i \dots$, we define

$$\begin{aligned} \mathbf{pure}(\sigma_1) &= \mathbf{pure}(B_1) \mathbf{pure}(B_2) \dots \mathbf{pure}(B_i) \dots, \\ \sigma_1 \cup \sigma_2 &= (B_1 \cup B'_1)(B_2 \cup B'_2) \dots (B_i \cup B'_i) \dots \end{aligned}$$

Proposition 5.8 (Relation between Cn and $\mathcal{M}^{Cn}$). Given a clock schedule σ , a clock relation Cn and a set of clock definitions $\mathbf{D}$, Φ is a set of constraints s.t. $Cn \in \Phi$ and $\Phi^d = \mathbf{D}$, then there exists a schedule $\sigma' \in \text{Sche}(\|_{Cn \in \mathbf{D}} \mathcal{M}^{Cn})$ that satisfies $\sigma' \text{ obs } \sigma$, and moreover,

$$\sigma \models Cn, \mathbf{D} \text{ iff } \sigma' \cup \mathbf{pure}_{\Phi}(\sigma) \models Cn.$$

In Prop. 5.8, $\mathbf{pure}_{\Phi}(\sigma)$ must be considered because there might exist some clocks in Φ but not in $\mathbf{D}$ so that they might not be included in σ' .

For example, let $\mathbf{D} = \{c \triangleq a * b, d \triangleq b + c\}$, $Cn = (c \preceq e)$, let e be a pure clock in Φ , let $\sigma = \{a, b\} \{a, e\} \{a, b\} \{a, e\} \{a, b\} \{a, e\} \dots$ be a schedule that satisfies $\sigma \models Cn, \mathbf{D}$. Then the schedule σ' in the observer $\mathcal{M}^{c \triangleq a * b} \parallel \mathcal{M}^{d \triangleq b + c}$ (which is exactly the observer $\mathcal{M}_1 \parallel \mathcal{M}_2$ shown in Fig. 7) that satisfies $\sigma' \text{ obs } \sigma$ and $\sigma' \cup \mathbf{pure}_{\Phi}(\sigma) \models Cn$ is $\sigma' = \{d, c, a, b\} \{a\} \{d, c, a, b\} \{a\} \{d, c, a, b\} \{a\} \dots$. e is the clock that is pure in σ but not in σ' .

The proof of Prop. 5.8 mainly relies on Lemma B.4 given in Appendix B.

Proof of Prop. 5.8 Let $\sigma = B_0 B_1 \dots B_k \dots$, $\mathcal{M}[F, C] = \|_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$, starting from the initial state of $\mathcal{M}$, by Lemma B.4 we can build a schedule $\sigma' = B'_0 B'_1 \dots B'_k \dots$ s.t. any B'_k ($k \geq 0$) satisfies $B'_k \cap F = B_k \cap F$, and for any non-pure clock d in $\mathbf{D}$, $B_k \models d, \mathbf{D}$ iff $d \in B'_k$. By Def. 5.15 we have $\sigma' \text{ obs } \sigma$. To see $\sigma' \cup \mathbf{pure}(\sigma) \models Cn$, notice that for any clock c in Cn and $\mathbf{D}$, no matter whether it is a pure clock or not, there always be that for any $k \geq 0$, $B_k \models c, \mathbf{D}$ iff $c \in B'_k \cup \mathbf{pure}(B_k)$. Intuitively, this means that the schedule $\sigma' \cup \mathbf{pure}(\sigma)$ exactly contains all clocks satisfied by σ at each instant. $\square$

As a corollary of Prop. 5.8, Thm. 5.2 forms the basic for model checking CCSL constraints in LTL_c (as seen in Sect. 5.5).

In the rest of the paper, we use $F(\sim \Phi)$ and $C(\sim \Phi)$ to express the set of free clocks and the set of constrained clocks of all the observers of the clock constraints in $\sim \Phi$ shown in Prop. 5.3, respectively.

Theorem 5.2. Given a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, T \rangle$, a clock relation Cn and a set of clock constraints $\mathbf{D}$ s.t. $F(\mathbf{D}) \subseteq Clk$ and $C(\mathbf{D}) \cap Clk = \emptyset$, let $\mathcal{M}' = \mathcal{M} \parallel (\|_{Cn \in \mathbf{D}} \mathcal{M}^{Cn})$, for any schedule $\sigma \in \text{Sche}(\mathcal{M})$, there exists a schedule $\sigma' \in \text{Sche}(\mathcal{M}')$ s.t. $\sigma'|_{\mathcal{M}} = \sigma$, and moreover,

$$\sigma \models Cn, \mathbf{D} \text{ iff } \sigma' \models Cn.$$

Proof. Let $N[F, C] = \|_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$, let $\sigma = B_0 B_1 \dots B_k \dots$, by Prop. 5.8 there exists a schedule $\sigma'' = B''_0 B''_1 \dots B''_k \dots$ s.t. $\sigma'' \in \text{Sche}(N)$ and $\sigma'' \text{ obs } \sigma$. Let $\sigma' = B'_0 B'_1 \dots B'_k \dots$ where $B'_k = B_k \cup B''_k$ for all $k \geq 0$. Clearly, $\sigma'|_{\mathcal{M}} = \sigma$. First, we show that $\sigma' \in \text{Sche}(\mathcal{M}')$:

Since $\Pi_N = \emptyset$, by Def. 5.4 we only need to prove that $B_k \cap Clk_N = B''_k \cap Clk$ for any $k \geq 0$. On one hand, for any $t \in B_k \cap Clk_N$, because $C(\mathbf{D}) \cap Clk = C \cap Clk = \emptyset$, thus $t \in F$. From the fact that $B_k \cap F = B''_k \cap F$ (since $\sigma'' \text{ obs } \sigma$) we have $t \in B''_k$. Thus $t \in B''_k \cap B_k \subseteq B''_k \cap Clk$. On the other hand, for any $t \in B''_k \cap Clk$, since $t \in Clk$, so $t \in F$ due to the fact that $t \in Clk_N$, $F = F(\mathbf{D}) \subseteq Clk$ and $C \cap Clk = \emptyset$. Therefore $t \in B''_k \cap F = B_k \cap F \subseteq B_k$. So $t \in B_k \cap B''_k \subseteq B_k \cap Clk_N$.

To see $\sigma' \models Cn$ iff $\sigma \models Cn, \mathbf{D}$, we prove that in fact $\sigma' = \sigma'' \cup \mathbf{pure}(\sigma)$, where $\sigma'' \cup \mathbf{pure}(\sigma) \models Cn$ iff $\sigma \models Cn, \mathbf{D}$ by Prop. 5.8. It is enough to show $B_k = \mathbf{pure}(B_k)$, which is obvious because $C \cap Clk = \emptyset$, i.e., any non-pure clocks of $\mathbf{D}$ are not in Clk . $\square$

5.5. Model Checking of $LTL_c/CCSL$ Specifications

In this subsection, we discuss the model checking of $LTL_c/CCSL$ specifications based on the previous subsections. The main idea behind model checking an $LTL_c/CCSL$ specification given a multi-clock system is to transform both the system model and the $LTL_c/CCSL$ specification into CTSAs, then make reachability

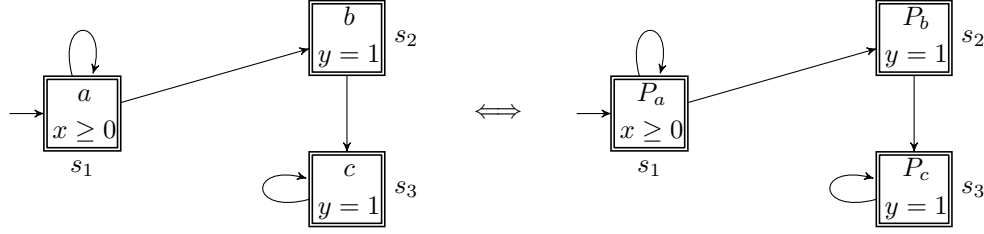


Fig. 9. An example of a CTSA and its corresponding Büchi automaton

analysis on the parallel composition of these CTSA. The transformation from LTL_c formulae to CTSA can be obtained through the transformation from LTL formulae to Büchi automata, since LTL_c formulae in fact can be expressed by LTL formulae according to Sect. 2.2. The transformation from CCSL specifications to CTSA is straightforward according to the CTSA semantics of CCSL constraints (Table. 3 and 4) and the composition between CTSA (Def. 5.4). As shown in Sect. 5.5.1, since CTSA/CTSA can be seen as OTS by taking clocks as special atomic propositions, the model checking of LTL_c /CCSL specifications can actually be realized by reachability analysis on OTS.

In Sect. 5.5.1 we first discuss the transformation from LTL_c to CTSA. Then in Sect. 5.5.2 we discuss the automata-based model checking for LTL_c /CCSL.

5.5.1. Transformation from LTL_c Formulae to CTSA

As indicated in Sect. 2.2, the expressiveness of LTL_c actually lies in that of LTL since the occurrences of each CCSL clock c can be expressed as a special atomic proposition named P_c . Similarly, a CTS and a CTSA can be taken as an OTS and a Büchi automaton (introduced in Appendix A) respectively, by encoding the occurrences of each clock c at each state as an atomic proposition P_c .

As indicated in Def. 5.7, CTSA are a special case of CTSA, so in the following we only define the relation between CTSA and Büchi automata. The relation between CTSA and OTS can be defined in a similar way.

Definition 5.16 (CTSA as a Büchi Automaton). For any CTSA $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$, its corresponding Büchi automaton, denoted as $\mathcal{M}^\mathcal{D}$, is defined as:

$$\mathcal{M}^\mathcal{D} = \langle \Pi^\mathcal{D}, Q, I, \theta^\mathcal{D}, \mathcal{T}, \mathcal{E} \rangle,$$

where $\Pi^\mathcal{D} = \Pi \cup \{P_c \mid c \in Clk\}$, P_c is the atomic proposition indicating the occurrences of the clock c in $\mathcal{M}$, which is distinguished from the propositions in Π . $\theta^\mathcal{D}$ is defined as:

$$\text{for any } q \in Q, \theta^\mathcal{D}(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}.$$

From Def. 5.16 it is easy to see that each CTSA corresponds to exactly one Büchi automaton since they share the same set of states and transitions. Hence if a Büchi automaton $\mathcal{A}$ has a CTSA $\mathcal{M}$ s.t. $\mathcal{M}^\mathcal{D} = \mathcal{A}$, we also denote $\mathcal{M}$ as $\mathcal{A}^{-\mathcal{D}}$, i.e., $\mathcal{M} = \mathcal{A}^{-\mathcal{D}}$.

Similarly, a CTS $\mathcal{M}$ exactly corresponds to an OTS, denoted as $\mathcal{M}^\mathcal{D}$. We omit its formal definition here, which is similar to Def. 5.16. If an OTS $\mathcal{A}$ has a CTS $\mathcal{M}$ s.t. $\mathcal{M}^\mathcal{D} = \mathcal{A}$, we also denote $\mathcal{M}$ as $\mathcal{A}^{-\mathcal{D}}$, i.e., $\mathcal{M} = \mathcal{A}^{-\mathcal{D}}$.

Obviously, a CTSA $\mathcal{M}$ and its corresponding Büchi automaton $\mathcal{M}^\mathcal{D}$ share the same set of traces, i.e., $\text{Trace}(\mathcal{M}) = \text{Trace}(\mathcal{M}^\mathcal{D})$. The behaviour of $\mathcal{M}$ are equivalent to the behaviour of $\mathcal{M}^\mathcal{D}$ in the sense that any clock behaviour in the CTSA is taken as a set of atomic propositions as defined in Def. 5.16.

As an example, Fig. 9 shows the CTS of Fig. 6 $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ (on the left) and its corresponding Büchi automaton $\mathcal{M}^\mathcal{D} = \langle \Pi^\mathcal{D}, Q, I, \theta^\mathcal{D}, \mathcal{T}, \mathcal{E} \rangle$ (on the right). Different from Fig. 6, here the CTS is taken as a special CTSA with all states as accepting states: $\mathcal{E} = \{s_1, s_2, s_3\}$. Two automata $\mathcal{M}$ and $\mathcal{M}^\mathcal{D}$ share the same set of states, the same set of initial states, the same set of transitions and the same set of accepting states. The set of atomic propositions $\Pi^\mathcal{D} = \{x \geq 0, y = 1, P_a, P_b, P_c\}$. The function $\theta^\mathcal{D}$ is defined s.t. $\theta^\mathcal{D}(s_1) = \{x \geq 0, P_a\}$, $\theta^\mathcal{D}(s_2) = \{y = 1, P_b\}$, $\theta^\mathcal{D}(s_3) = \{y = 1, P_c\}$.

Comparing Def. 2.6 and Def. 5.16 we see that for any trace π of $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ with $\langle \pi, \theta, \xi \rangle$ the configuration of π , $\theta^\xi = \theta^\mathcal{D}$ holds. Recall that in Sect. 2.2, we have already shown that an LTL_c formula ϕ can be encoded as an LTL formula $\phi^\mathcal{D}$ in Def. 2.5 and their relation is given in Thm. 2.1. Based on

Thm. 2.1 we now discuss the relation between LTL_c formulae and their encoded LTL formulae in terms of the satisfaction relations w.r.t. transition systems.

Proposition 5.9 (Relation between LTL_c Formulae and LTL Formulae). Given a CTS $\mathcal{M}$ and an LTL_c formula ϕ , for any trace $\pi \in \text{Trace}(\mathcal{M}) = \text{Trace}(\mathcal{M}^\mathcal{D})$,

$$\pi \models_{\mathcal{M}} \phi \text{ iff } \pi \models_{\mathcal{M}^\mathcal{D}} \phi^\mathcal{D}.$$

The relation $\pi \models_{\mathcal{N}} \psi$ is given in Appendix A for an OTS $\mathcal{N}$ and an LTL formula ψ .

Prop. 5.9 can be easily proved from Thm. 2.1.

The proof of Prop. 5.9 Let $\mathcal{M} = \langle \Pi, Q, I, \theta, \xi, \mathcal{T} \rangle$, then $\mathcal{M}^\mathcal{D} = \langle \Pi^\mathcal{D}, Q, I, \theta^\mathcal{D}, \mathcal{T} \rangle$ according to Def. 5.16. By Thm. 2.1 and the fact that $\theta^\mathcal{D} = \theta^\xi$ analyzed above, we have $\pi \models_{\mathcal{M}} \phi$ iff $\pi \models_{\theta, \xi} \phi$ iff $\pi \models_{\theta^\xi} \phi^\mathcal{D}$ iff $\pi \models_{\theta^\mathcal{D}} \phi^\mathcal{D}$ iff $\pi \models_{\mathcal{M}^\mathcal{D}} \phi^\mathcal{D}$. $\square$

With the support of Def. 2.5, Def. 5.16 and Prop. 5.9, we now show that the transformation from an LTL_c formula into a CTSA can be performed through the transformation from its corresponding LTL formula into a Büchi automaton [18]. The next theorem states the relation between an LTL_c formula and its corresponding CTSA.

Theorem 5.3. Given an LTL_c formula ϕ , let $\mathfrak{B}(\phi^\mathcal{D})$ denote the Büchi automaton of the LTL formula $\phi^\mathcal{D}$, then $\mathcal{M}(\mathcal{E}) = \mathfrak{B}(\phi^\mathcal{D})^{-\mathcal{D}}$ is the CTSA of ϕ , i.e., for any trace π , it satisfies that

$$\pi \models_{\mathcal{M}(\mathcal{E})} \phi \text{ iff } \pi \in \text{Trace}(\mathcal{M}(\mathcal{E})).$$

Proof. Let $\mathfrak{B}(\phi^\mathcal{D}) = \langle \Pi, Q, I, \theta, \mathcal{T}, \mathcal{E} \rangle$. Because for any $c \in \mathcal{C}(\phi)$, P_c is a subformula of $\phi^\mathcal{D}$, according to the standard procedure of the transformation from LTL formula into Büchi automaton [18], $A = \{P_c \mid c \in \mathcal{C}(\phi)\}$ is the set of all atomic propositions that correspond to clocks in Π . According to Def. 5.16, the corresponding CTSA of $\mathfrak{B}(\phi^\mathcal{D})$ is

$$\mathcal{M}(\mathcal{E}) = \langle \Pi^{-\mathcal{D}}, \text{Clk}, Q, I, \theta^{-\mathcal{D}}, \xi, \mathcal{T}, \mathcal{E} \rangle,$$

where $\Pi^{-\mathcal{D}} = \Pi - A$, $\text{Clk} = \{c \mid P_c \in A\} = \mathcal{C}(\phi)$, $\theta^{-\mathcal{D}}(q) = \theta(q) - \{P_c \mid P_c \in \theta(q)\}$ for any $q \in Q$, $\xi(q) = \{c \mid P_c \in \theta(q)\}$ for any $q \in Q$.

For any trace π , by Prop. 5.9 we get that $\pi \models_{\mathcal{M}} \phi$ iff $\pi \models_{\mathfrak{B}(\phi^\mathcal{D})} \phi^\mathcal{D}$. By the relation between an LTL formula and its corresponding Büchi automaton [18], there is $\pi \models_{\mathfrak{B}(\phi^\mathcal{D})} \phi^\mathcal{D}$ iff $\pi \in \text{Trace}(\mathfrak{B}(\phi^\mathcal{D}))$. Since $\text{Trace}(\mathfrak{B}(\phi^\mathcal{D})) = \text{Trace}(\mathcal{M}(\mathcal{E}))$, we have $\pi \in \text{Trace}(\mathfrak{B}(\phi^\mathcal{D}))$ iff $\pi \in \text{Trace}(\mathcal{M}(\mathcal{E}))$. $\square$

Algorithm 1: Transformation from LTL_c formula to CTSA

```

1 function  $LTL_c2CTSA(\phi)$ ;
   Input : an  $LTL_c$  formula  $\phi$ 
   Output: the corresponding CTSA  $\mathcal{M}(\mathcal{E}) = \langle \Pi, \text{Clk}, Q, I, \theta, \mathcal{T}, \mathcal{E} \rangle$  of  $\phi$ 
2 encode  $\phi$  into its corresponding LTL formula  $\phi^\mathcal{D}$ ;
3  $A := \text{LTL2Büchi}(\phi^\mathcal{D})$ ; /* run the standard procedure of transforming an LTL formula into
   a Büchi automaton */
4  $\mathcal{M}(\mathcal{E}) := A^{-\mathcal{D}}$ ; /* return the corresponding CBA */

```

Based on Thm. 5.3, we build a main procedure for the transformation from an LTL_c formula into a CTSA, as Algorithm 1. In Algorithm 1, we assume the standard transformation procedure LTL2Büchi from LTL formulae to Büchi automaton in the model checking theory of LTL, whose implementation is out of the scope of this paper. One can refer to [18] for more details.

5.5.2. Automata-based Model Checking for $LTL_c/CCSL$ Specifications

The verification problem for $LTL_c/CCSL$ specifications asks that given a CTS model $\mathcal{M}$ of a multi-clock system and an $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$, whether all traces starting from the initial states of $\mathcal{M}$ satisfy $\langle \phi, \Phi \rangle$. The model checking technique mainly follows the idea behind the model checking theory of LTL [18].

We propose transforming the negation of the LTL_c formula ϕ into a CTSA, named $\mathcal{M}^{\neg\phi}$ (Sect. 5.5.1), encoding the negations of all clock relations Φ^r as a CTSA, named $\mathcal{M}^{\sim\Phi^r}$, and encoding the set of all clock definitions Φ^d as a CTSA, named $\mathcal{M}^{\Phi^d}$. The validity of the specification $\langle\phi, \Phi\rangle$ w.r.t. the CTS $\mathcal{M}$ can then be verified by checking whether there exists a trace starting from the initial states of $\mathcal{M}$ and accepted by the parallel composition of the CTS $\mathcal{M}$, the CTSA $\mathcal{M}^{\neg\phi}$, $\mathcal{M}^{\sim\Phi^r}$ and $\mathcal{M}^{\Phi^d}$ s.t. it is reachable to some accepting state of $\mathcal{M}^{\neg\phi}$ and $\mathcal{M}^{\sim\Phi^r}$.

Before introducing the main theorem — Thm. 5.4 — of the model checking theory, we first show that there always exists an observation relation between a system model $\mathcal{M}$ and the CTSA $\mathcal{M}^{\neg\phi}$ of the negation of an LTL_c formula ϕ .

As stated at the beginning of Sect. 5.3, in the model checking theory of LTL, there always exists an observation relation between a system model and the Büchi automaton of the negation of an LTL formula that specifies the system model. This is guaranteed by the transformation from LTL formulae into Büchi automata [18]. Formally, given an OTS $\mathcal{N} = \langle\Pi, Q, I, \theta, \mathcal{T}\rangle$ and an LTL formula ϕ , we have

$$\mathfrak{B}(\neg\phi) \text{ obs } \mathcal{N},$$

where according to Thm. 5.3, $\mathfrak{B}(\neg\phi)$ denotes the Büchi automaton of the formula $\neg\phi$.

From the relation between CTSA and Büchi automata described in Def. 5.16 and the fact that the transformation from LTL_c formulae into CTSA is performed through the transformation procedure from LTL formulae into Büchi automata, it is not hard to prove that there exists an observation relation between a CTS and the CTSA of the negation of an LTL_c formula.

In the rest of the paper, we use $\Pi(\phi)$ to represent the set of all atomic propositions of a formula ϕ .

Proposition 5.10. Given a CTS $\mathcal{M} = \langle\Pi, Clk, Q, I, \theta, \xi, \mathcal{T}\rangle$ and an LTL_c formula ϕ with $\Pi(\phi) \subseteq \Pi$, let $\mathcal{M}^{\neg\phi} = \mathfrak{B}((\neg\phi)^{\mathfrak{D}})^{-\mathfrak{D}}$, we have

$$\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}.$$

The proof of Prop. 5.10 is given in Appendix B.

Theorem 5.4. Given a CTS $\mathcal{M} = \langle\Pi, Clk, Q, I, \theta, \xi, \mathcal{T}\rangle$ and an $LTL_c/CCSL$ specification $\langle\phi, \Phi\rangle$ with $\Pi(\phi) \subseteq \Pi$, $\mathcal{C}(\phi) \subseteq Clk$, $F(\sim\Phi) \subseteq Clk$ and $C(\sim\Phi) \cap Clk = \emptyset$, let $\mathcal{M}' = \mathcal{M} \parallel \mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\Phi}$ where $\mathcal{M}^{\neg\phi} = \mathfrak{B}((\neg\phi)^{\mathfrak{D}})^{-\mathfrak{D}}$, $\mathcal{M}^{\Phi} = \mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\sim\Phi^r}$, $\mathcal{M}^{\Phi^d} = \parallel_{Cn \in \Phi^d} \mathcal{M}^{Cn}$, and $\mathcal{M}^{\sim\Phi^r} = \parallel_{Cn^+ \in \sim\Phi^r} \mathcal{M}^{Cn^+}$, then for any $q \in I$,

$$q \models_{\mathcal{M}} \langle\phi, \Phi\rangle \text{ iff}$$

$$\text{there is no trace } \pi \in \text{Trace}(\mathcal{M}') \text{ s.t. } \pi[0]_{\mathcal{M}} = q \text{ and } \pi|_{\mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}} \in \text{Trace}(\mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}).$$

Thm. 5.4 says that to check whether a CTS $\mathcal{M}$ satisfies an $LTL_c/CCSL$ specification $\langle\phi, \Phi\rangle$, we make the product $\mathcal{M}'$ of the CTS $\mathcal{M}$ and the CTSA $\mathcal{M}^{\neg\phi}$ and $\mathcal{M}^{\Phi}$. $\mathcal{M}^{\neg\phi}$ is the CTSA of the negation of the formula ϕ , and $\mathcal{M}^{\Phi}$ is the product of the CTSA of the negations of all clock relations in Φ and the CTSA of all clock definitions in Φ . After making this product, we check that there exists no trace π of $\mathcal{M}'$ s.t. 1) it starts from an initial state of $\mathcal{M}$, and 2) its projection on $\mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}$ is accepted by $\mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}$, which means that π passes one of the accepting states of $\mathcal{M}^{\neg\phi}$ or any $\mathcal{M}^{Cn^+}$ (where $Cn^+ \in (\sim\Phi^r)$) for infinitely many times.

The conditions $\Pi(\phi) \subseteq \Pi$ and $\mathcal{C}(\phi) \subseteq Clk$ says that the formula ϕ can only faithfully capture the property in the system model $\mathcal{M}$, and cannot contain any atomic propositions or clocks not in $\mathcal{M}$. The condition $F(\sim\Phi) \subseteq Clk$ says that all clock constraints in Φ can only observe the clock behaviour in $\mathcal{M}$ and should not contain any free clocks not in $\mathcal{M}$. The condition $C(\sim\Phi) \cap Clk = \emptyset$ means that all clocks defined in Φ must be new clocks not appearing in $\mathcal{M}$.

Based on the works in the previous subsections, the proof of Thm. 5.4 can be given below. The dependence relations between different lemmas, propositions and theorems are given in Fig. 5.

The proof of Thm. 5.4 Due to the associativity of the parallel composition (Prop. 5.1), we can write $\mathcal{M}'$ as

$$\mathcal{M}' = \mathcal{M} \parallel \mathcal{M}^{\neg\phi} \parallel (\mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\sim\Phi^r}) = (\mathcal{M} \parallel \mathcal{M}^{\Phi^d}) \parallel (\mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}).$$

Let $\mathcal{M}'' = \mathcal{M} \parallel \mathcal{M}^{\Phi^d}$, $\mathcal{M}^{(3)} = \mathcal{M}^{\neg\phi} \parallel \mathcal{M}^{\sim\Phi^r}$, we proceed the proof by contradictions on both directions.

($\Rightarrow$): Suppose that there exists a trace $\pi_0 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_0|_{\mathcal{M}^{(3)}} \in \text{Trace}(\mathcal{M}^{(3)})$ and $\pi_0[0]_{\mathcal{M}} = q$, we

now prove it contradicts $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$. Since $\pi_0|_{\mathcal{M}^{(3)}} \in \text{Trace}(\mathcal{M}^{(3)})$, by the definition of CTSA (Def. 5.7) and the product of CTSA defined in Sect. 5.1, we know that trace $\pi_0|_{\mathcal{M}^{(3)}}$ must pass one of accepting states of $\mathcal{M}^{-\phi}$ or any $\mathcal{M}^{Cn^+}$ (where $Cn^+ \in \sim \Phi^r$) for infinite times. So we need to consider two situations:

- (1) there exists a constraint $Cn_0^+ \in \sim \Phi^r$ s.t. $\pi_0|_{\mathcal{M}^{Cn_0^+}} \in \text{Trace}(\mathcal{M}^{Cn_0^+})$;
- (2) $\pi_0|_{\mathcal{M}^{-\phi}} \in \text{Trace}(\mathcal{M}^{-\phi})$.

For situation (1), we firstly prove that for any $Cn^+ \in \sim \Phi^r$, $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn^+}}}$ holds. Consider any state u of trace π_0 , we see that $u|_{\mathcal{M}''}$ is a state of trace $\pi_0|_{\mathcal{M}''}$, and $u|_{\mathcal{M}^{Cn^+}}$ is a state of trace $\pi_0|_{\mathcal{M}^{Cn^+}}$. Since $\mathcal{C}(Cn^+) \subseteq F(\sim \Phi^r) \subseteq F(\sim \Phi) \subseteq \text{Clk} \subseteq \text{Clk}_{\mathcal{M}''}$, $\text{Clk}_{\mathcal{M}^{Cn^+}} = \mathcal{C}(Cn^+) \subseteq \text{Clk}_{\mathcal{M}''}$ holds. For any CTS $N = N_1 \parallel N_2$ and any state v of N , since $\text{Clk}_{N_2} \cap \xi_{N_1}(v|_{N_1}) = \text{Clk}_{N_1} \cap \xi_{N_2}(v|_{N_2})$ (by Def. 5.4), we can have

$$\begin{aligned}
 \xi_{N_1}(v|_{N_1}) &= \xi_{N_1}(v|_{N_1}) \cup (\xi_{N_1}(v|_{N_1}) \cap \text{Clk}_{N_2}) \\
 &= \xi_{N_1}(v|_{N_1}) \cup (\xi_{N_2}(v|_{N_2}) \cap \text{Clk}_{N_1}) \\
 &= (\xi_{N_1}(v|_{N_1}) \cap \text{Clk}_{N_1}) \cup (\xi_{N_2}(v|_{N_2}) \cap \text{Clk}_{N_1}) \\
 &= (\xi_{N_1}(v|_{N_1}) \cup \xi_{N_2}(v|_{N_2})) \cap \text{Clk}_{N_1} \\
 &= \xi_N(v) \cap \text{Clk}_{N_1}.
 \end{aligned} \tag{1}$$

Therefore, we have $\xi_{\mathcal{M}''}(u|_{\mathcal{M}''}) = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}''}$ and $\xi_{\mathcal{M}^{Cn^+}}(u|_{\mathcal{M}^{Cn^+}}) = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}^{Cn^+}}$. So $\xi_{\mathcal{M}''}(u|_{\mathcal{M}''}) \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}''} \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}^{Cn^+}}(u|_{\mathcal{M}^{Cn^+}})$. Hence $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn^+}}}$.

Since $\pi_0|_{\mathcal{M}^{Cn_0^+}} \in \text{Trace}(\mathcal{M}^{Cn_0^+})$, $\sigma^{\pi_0|_{\mathcal{M}^{Cn_0^+}}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$. From above we know that $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn_0^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn_0^+}}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$. Let $Cn_0^+ = \sim \text{Rel}_0$ where $\text{Rel}_0 \in \Phi^r$, by Prop. 5.7 we know that $\sigma^{\pi_0|_{\mathcal{M}''}} \not\models \text{Rel}_0$. Let $\pi = \pi_0|_{\mathcal{M}''}|_{\mathcal{M}}$, for any state w of the trace $\pi_0|_{\mathcal{M}''}$, by the equation (1) above, we have $\xi_{\mathcal{M}''}(w) \cap \text{Clk} = \xi(w|_{\mathcal{M}})$, hence we get $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}} = \sigma^\pi$. Because $\sigma^{\pi_0|_{\mathcal{M}''}} \not\models \text{Rel}_0$, by Thm. 5.2 we obtain $\sigma^\pi \not\models \text{Rel}_0, \Phi^d$. Notice that $\pi[0] = q$, this contradicts the assumption that $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$.

For situation (2), by Thm. 5.3, we have $\pi_0|_{\mathcal{M}^{-\phi}} \models_{\mathcal{M}^{-\phi}} \neg \phi$, which is $\pi_0|_{\mathcal{M}^{-\phi}} \not\models_{\mathcal{M}^{-\phi}} \phi$. Let x be any state of the trace π_0 , we show that

- A) for any $p \in \Pi(\phi)$, $p \in \theta(x|_{\mathcal{M}})$ iff $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$;
- B) for any $c \in \mathcal{C}(\neg \phi)$, $c \in \xi(x|_{\mathcal{M}})$ iff $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$.

For A), from $\theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \Pi = \theta(x|_{\mathcal{M}}) \cap \Pi_{\mathcal{M}^{-\phi}}$, $\Pi(\phi) \subseteq \Pi$ and the fact that $\Pi(\phi) = \Pi_{\mathcal{M}^{-\phi}}$, we have $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$ iff $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \Pi$ iff $p \in \theta(x|_{\mathcal{M}}) \cap \Pi_{\mathcal{M}^{-\phi}}$ iff $p \in \theta(x|_{\mathcal{M}})$. For B), from $\xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \text{Clk} = \xi(x|_{\mathcal{M}}) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$, $\mathcal{C}(\neg \phi) = \mathcal{C}(\phi) \subseteq \text{Clk}$ and the fact that $\mathcal{C}(\phi) = \text{Clk}_{\mathcal{M}^{-\phi}}$, we have $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$ iff $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \text{Clk}$ iff $c \in \xi(x|_{\mathcal{M}}) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$ iff $c \in \xi(x|_{\mathcal{M}})$.

By A) and B) it is not hard to see that $\pi_0|_{\mathcal{M}} \not\models_{\mathcal{M}} \phi$ because $\pi_0|_{\mathcal{M}^{-\phi}} \not\models_{\mathcal{M}^{-\phi}} \phi$. This contradicts the assumption that $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$.

($\Leftarrow$): Suppose $q \not\models_{\mathcal{M}} \langle \phi, \Phi \rangle$, then there exists a trace $\pi_0 \in \text{Trace}(q)$ s.t.

- (i) $\sigma^{\pi_0} \not\models \Phi$, or
- (ii) $\pi_0 \not\models_{\mathcal{M}} \phi$.

For the situation (i), we know there exists a clock relation $\text{Rel}_0 \in \Phi^r$ s.t. $\sigma^{\pi_0} \not\models \text{Rel}_0, \Phi^d$, let $Cn_0^+ = \sim \text{Rel}_0$, so $\sigma^{\pi_0} \models Cn_0^+, \Phi^d$. Since $C(\sim \Phi) \cap \text{Clk} = \emptyset$, $C(\Phi^d) \cap \text{Clk} = \emptyset$ hold, thus we have $\text{Clk}_{\mathcal{M}^{\Phi^d}} \cap \text{Clk} = (F(\Phi^d) \cup C(\Phi^d)) \cap \text{Clk} = F(\Phi^d) \cap \text{Clk} \subseteq F(\Phi^d)$. So $\mathcal{M}^{\Phi^d} \text{ obst } \mathcal{M}$ and we have $\mathcal{M}^{\Phi^d} \text{ obs } \mathcal{M}$ by Prop. 5.5. Therefore, there is a trace $\pi_1 \in \text{Trace}(\mathcal{M}'')$ s.t. $\pi_1|_{\mathcal{M}} = \pi_0$. Let w be a state of $\mathcal{M}''$, by equation 1 we have that $\xi_{\mathcal{M}''}(w) \cap \text{Clk} = \xi(w|_{\mathcal{M}})$. So $\sigma^{\pi_1}|_{\mathcal{M}} = \sigma^{\pi_0}$. According to Thm. 5.2 we have $\sigma^{\pi_1} \not\models \text{Rel}_0$, therefore by Prop. 5.7, there is $\sigma^{\pi_1}|_{\mathcal{M}^{Cn_0^+}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$.

Now we show that $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}''$. Since $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}$ (by Prop. 5.10) and the fact that $\Pi(\mathcal{M}^{\Phi^d}) = \emptyset$, it is enough to only consider their clock behaviours and prove that for any trace $\pi \in \text{Trace}(\mathcal{M}'')$,

- (I) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}^{-\phi}$ s.t. $\xi_{\mathcal{M}''}(\pi[0]) \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk}_{\mathcal{M}''}$;

- (II) for any transition $\langle q, q' \rangle$ of the trace π and any state x of $\mathcal{M}^{-\phi}$ s.t. $\langle q, x \rangle$ is a state of $\mathcal{M}'' \parallel \mathcal{M}^{-\phi}$, there exists a state $\langle q', x' \rangle$ of $\mathcal{M}'' \parallel \mathcal{M}^{-\phi}$ s.t. $\xi_{\mathcal{M}''}(q') \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}^{-\phi}}(x') \cap \text{Clk}_{\mathcal{M}''}$.

We only prove (I), the proof of (II) is similar. Let $\pi[0] = \langle y_1, y_2 \rangle \in \mathcal{M} \parallel \mathcal{M}^{\Phi^d}$, since $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}$, there exists an initial state x_0 of $\mathcal{M}^{-\phi}$ s.t. $\xi(y_1) \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk}$. Now we prove that $\xi_{\mathcal{M}''}(\pi[0]) \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk}_{\mathcal{M}''}$. On one hand, for any $t \in \xi_{\mathcal{M}''}(\pi[0]) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$, $t \in (\xi(y_1) \cup \xi_{\mathcal{M}^{\Phi^d}}(y_2)) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$. If $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2)$, since $\text{Clk}_{\mathcal{M}^{-\phi}} = \mathcal{C}(\phi) \subseteq \text{Clk}$, $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2) \cap \text{Clk}$ holds. By $\mathcal{M}^{\Phi^d} \text{ obs } \mathcal{M}$ proved above, we have $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2) \cap \text{Clk} = \xi(y_1) \cap \text{Clk}_{\mathcal{M}^{\Phi^d}}$. So $t \in \xi(y_1)$. Therefore, $t \in \xi(y_1) \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk} \subseteq \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk}_{\mathcal{M}''}$. On the other hand, for any $t \in \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk}_{\mathcal{M}''}$, $t \in \xi_{\mathcal{M}^{-\phi}}(x_0) \subseteq \text{Clk}_{\mathcal{M}^{-\phi}} = \mathcal{C}(\phi) \subseteq \text{Clk}$. So we have $t \in \xi_{\mathcal{M}^{-\phi}}(x_0) \cap \text{Clk} = \xi(y_1) \cap \text{Clk}_{\mathcal{M}^{-\phi}} \subseteq (\xi(y_1) \cup \xi_{\mathcal{M}^{\Phi^d}}(y_2)) \cap \text{Clk}_{\mathcal{M}^{-\phi}} = \xi_{\mathcal{M}''}(\pi[0]) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$.

By $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}''$, there exists a trace $\pi_2 \in \text{Trace}(\mathcal{M}'' \parallel \mathcal{M}^{-\phi})$ s.t. $\pi_2|_{\mathcal{M}''} = \pi_1$. Since $\text{Clk}_{\mathcal{M} \sim \Phi^r} \cap \text{Clk}_{\mathcal{M}'' \parallel \mathcal{M}^{-\phi}} = F(\sim \Phi^r) \cap \text{Clk}_{\mathcal{M}'' \parallel \mathcal{M}^{-\phi}} \subseteq F(\sim \Phi^r)$, by Def. 5.14 we have $\mathcal{M} \sim \Phi^r \text{ obst } \mathcal{M}'' \parallel \mathcal{M}^{-\phi}$, therefore $\mathcal{M} \sim \Phi^r \text{ obs } \mathcal{M}'' \parallel \mathcal{M}^{-\phi}$ by Prop. 5.5. Hence there is a trace $\pi_3 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_3|_{\mathcal{M}'' \parallel \mathcal{M}^{-\phi}} = \pi_2$. But π_3 contradicts the assumption.

For situation (ii), we immediately obtain $\pi_0 \models_{\mathcal{M}} \neg\phi$, so $\pi_0 \in \text{Trace}(\mathcal{M}^{-\phi})$ by Thm. 5.3. Since $\Pi(\phi) \subseteq \text{Clk}$, we have $\pi_0 \in \text{Trace}(\mathcal{M} \parallel \mathcal{M}^{-\phi})$. From $C(\sim \Phi) \subseteq \text{Clk} = \emptyset$ and $\mathcal{C}(\phi) \subseteq \text{Clk}$, we have $\text{Clk}_{\mathcal{M}^{\Phi^d} \parallel \mathcal{M} \sim \Phi^r} \cap \text{Clk}_{\mathcal{M} \parallel \mathcal{M}^{-\phi}} = (F(\sim \Phi) \cup C(\sim \Phi)) \cap (\text{Clk} \cup \mathcal{C}(\phi)) = (F(\sim \Phi) \cup C(\sim \Phi)) \cap \text{Clk} \subseteq F(\sim \Phi)$. By Def. 5.14 we get that $\mathcal{M}^{\Phi^d} \parallel \mathcal{M} \sim \Phi^r \text{ obst } \mathcal{M} \parallel \mathcal{M}^{-\phi}$, so $\mathcal{M}^{\Phi^d} \parallel \mathcal{M} \sim \Phi^r \text{ obs } \mathcal{M} \parallel \mathcal{M}^{-\phi}$ by Prop. 5.5. Therefore there exists a trace $\pi_1 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_1|_{\mathcal{M} \parallel \mathcal{M}^{-\phi}} = \pi_0$, which violates the assumption. $\square$

Algorithm 2: Model Checking of $LTL_c/CCSL$ Specifications

```

1 function MC_LTLc-CCSL ( $\mathcal{M}, \langle \phi, \Phi \rangle$ );          /* where  $\Pi(\phi) \subseteq \Pi$ ,  $\mathcal{C}(\phi) \subseteq \text{Clk}$ ,  $F(\Phi) \subseteq \text{Clk}$  and
2    $C(\Phi) \cap \text{Clk} = \emptyset$ ;  $\Phi$  is bounded */
3   Input : a CTS  $\mathcal{M}$ , an  $LTL_c/CCSL$  specification  $\langle \phi, \Phi \rangle$ 
4   Output: a boolean value  $Res$  ( $Res = true$  iff  $\mathcal{M}$  satisfies  $\langle \phi, \Phi \rangle$ )
5    $\mathcal{M}^{-\phi} := LTLc2CTSA(\neg\phi)$ ;                      /* encoding formula  $\neg\phi$  into a CTSA */
6    $\mathcal{M}^{\Phi} := (\parallel_{Cn \in \Phi^d} \mathcal{M}^{Cn}) \parallel (\parallel_{Cn^+ \in \sim \Phi^r} \mathcal{M}^{Cn^+})$ ; /* encoding clock-constraint set  $\Phi$  into a CTSA
7   */
8    $\mathcal{M}'(\mathcal{E}') := \mathcal{M} \parallel \mathcal{M}^{-\phi} \parallel \mathcal{M}^{\Phi}$ ;
9    $Res := RB\_Alys(I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}')$ ; /* run the standard procedure for reachability analysis on
10  the product  $\mathcal{M}'$  */

```

Based on Thm. 5.4, we propose a main procedure, as Algorithm 2, for model checking a subset of $LTL_c/CCSL$ specifications in which the set of CCSL constraints of an LTL_c formula is bounded. Algorithm 2 mainly follows the idea behind the algorithm for model checking LTL formulae [18]. The transformation from a clock definition Cn to its CTS $\mathcal{M}^{Cn}$, or from the negation of a clock relation Cn to its CTSA $\mathcal{M}^{\sim Cn}$, is straightforward according to Table 4. The reachability analysis is realized by procedure $RB_Alys(I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}')$, which checks whether there exists a trace of $\mathcal{M}'$ starting from one of states in $I_{\mathcal{M}'}$ (i.e. the initial states) s.t. it passes one of states in $\mathcal{E}'$ for infinitely many times. For each state in $\mathcal{E}'$ that is reachable to a state in $I_{\mathcal{M}'}$, procedure RB_Alys searches that whether there exists a cycle that contains the state.

We omit the details of procedure RB_Alys in this paper. According to Def. 5.16, CTSs/CTSAs and OTS/Büchi automata share the same set of states and transitions, therefore, procedure RB_Alys is similar to the standard procedure for reachability analysis on the product of an OTS and a Büchi automaton of an LTL formula, which can be found in relevant references such as [18].

5.6. Model Checking Complexity

The worst-case time complexity of Algorithm 2 is

$$O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi)),$$

where ϕ , $\mathcal{M}$ and Φ are given in Algorithm 2. The length of the formula ϕ — $|\phi|$ — is defined as the number of logical operators that appear in ϕ . We use $|\mathcal{A}| = |Q_{\mathcal{A}}| + |\mathcal{T}_{\mathcal{A}}|$ to express the number of states and transitions of any CTS/CTSA $\mathcal{A}$. $N(\Phi) = |\mathcal{M}^{\Phi}|$, where $\mathcal{M}^{\Phi}$, given in Algorithm 2, represents the product of the CTSAs of all constraints in $\sim \Phi$. It is easy to see that $N(\Phi)$ not only depends on the size of Φ but also on the clock expressions of Φ themselves. For example, the CTSA of constraints $\Phi_1 = \{c_1 \prec c, c \triangleq c_1 \$1, c \prec c_2, c_2 \prec c_1\}$ has a number of states and transitions that is much smaller than that of the CTSA of constraints $\Phi_2 = \{c_1 \prec c, c \triangleq c_1 \$1000, c \prec c_2, c_2 \prec c_1\}$, even though $|\Phi_1| = |\Phi_2|$. The cause of the difference is from the clock definitions $c \triangleq c_1 \$1$ and $c \triangleq c_1 \$1000$.

The analysis of the complexity of Algorithm 2 is given as follows, where the symbols ϕ , $\mathcal{M}$, $\mathcal{M}'$ and Φ are given in Algorithm 2.

In procedure LTL_c2CTSA of Algorithm 1, due to [18], the complexity of procedure LTL2Büchi for transforming LTL formulae into Büchi automata is $2^{O(|\phi^{\mathfrak{D}}|)}$. $2^{O(|\phi^{\mathfrak{D}}|)}$ is also the upper bound of the number of states and transitions of the Büchi automaton of $\phi^{\mathfrak{D}}$: $\mathfrak{B}(\phi^{\mathfrak{D}})$. Since $\mathfrak{B}(\phi^{\mathfrak{D}})$ and the CTSA of ϕ , i.e., $\mathfrak{B}(\phi^{\mathfrak{D}})^{-\mathfrak{D}}$, share the same set of states and transitions according to Def. 5.16, $2^{O(|\phi^{\mathfrak{D}}|)}$ is also the upper bound of the number of states and transitions of $\mathfrak{B}(\phi^{\mathfrak{D}})^{-\mathfrak{D}}$. On the other hand, it is not hard to see that $O(|\phi^{\mathfrak{D}}|) = O(|\phi|)$ according to the definition of the transformation $\mathfrak{D}$ in Def. 2.5. Therefore, $|\mathcal{M}^{\neg\phi}|$ lies in $2^{O(|\phi|)}$. Hence the upper bound of $|\mathcal{M}'|$ is $O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi))$. According to the model checking theory of LTL [18], the worst-case time and space complexity of procedure RB_Alys(I, TS, S) is linear in $|TS|$, i.e., $O(|TS|)$. So in Algorithm 2, the complexity of procedure RB_Alys($I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}_0$) is $O(|\mathcal{M}'|) = O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi))$, which obviously is also the complexity of Algorithm 2.

6. Related Work and Further Discussions

6.1. Temporal Logics for Real-time Systems

Variations of temporal logics have been proposed for specifying real-time systems. Here we choose some of the most representative temporal logics among them which are closely related to our proposed *LTL_c*, and make comparisons between them.

Metric temporal logic (MTL) [15] is a logic proposed in early years for specifying properties that involve explicit time. It extends propositional temporal logic with bounded temporal operators like $\Diamond_{<t}$ to express a temporal property with a real-time constraint. For example, formula $\Diamond_{<10} p$ means that p must be true within 10 time units. In MTL, each state is mapped to a non-negative integer indicating an explicit real time. The time model of MTL is thus a discrete one, described as a sequence of non-negative integers. The expression bound to a temporal operator (e.g. $\Diamond_{<t}$) in fact plays the role of a filter that extracts the states whose associated time values are less than t . The bound operator (e.g. $\Box_c$) in *LTL_c* is inspired by this idea. Compared to MTL, *LTL_c* only involves logical time instants indicated by clocks, rather than explicit time values. *LTL_c/CCSL* specifications only focus on properties related to logical clocks, not properties related to time values. Our *LTL_c* can naturally express a partial-ordered time model indicated by multiple clocks defined as primitives in *LTL_c*, while the time model of MTL is total-ordered.

Metric interval temporal logic (MITL) [16] is based on a continuous time model where each state corresponds to an interval of time (e.g. (1.2, 2.4]) during which the system is in that state. Behaviour of a system is thus modelled as a sequence of states whose corresponding timed intervals are joined end to end. Any temporal operator in MITL is bound to a timed interval (e.g. $\Diamond_{(1.2, 2.4]}$), confining the operators to only work on the states during this period. For example, formula $\Diamond_{(1.2, 2.4]} \phi$ means that ϕ will hold from time 1.2 to time 2.4 in the future. Different from MTL, the time domain of MITL is real. Compared to MITL, the time model of *LTL_c* is discrete, which is more suitable for specifying discrete-event systems which are based on logical clocks.

Signal temporal logic (STL) [24] is for specifying continuous-timed behaviours, such as the trajectories of hybrid systems or the behaviour of physical plants (often modelled as differential equations). The syntax of STL is the same as a subset of MITL called ‘bounded MITL’ but defined in a different semantics. In STL, it is considered whether a continuous real-valued signal satisfies a STL formula, rather than a sequence of timed states as in MITL. For example, for a continuous real-valued signal $x(t) = \sin(\omega t)$, consider to check whether x satisfies a formula $\Diamond_{[3,5]}(x > 0.7)$. Due to the undecidability of dealing with the continuity, STL is mainly

used for monitoring systems. Recently some variations of STL have been proposed for specifying different types of real-time properties. For example, [25] presents a time-frequency logic for specifying time-frequency properties; [26] proposes an extension of STL for expressing various dynamic aspects of oscillations. Like MITL, STL mainly focus on specifying continuous-timed behaviours, while LTL_c is specially designed for specifying properties related to logical clocks.

The logics discussed above, possessing either discrete or continuous time models, are all based on a total-order time model. Next we introduce some logics that can describe a partial-order time model as LTL_c . Compared to them, in LTL_c we use CCSL to further specify the logical constraints between clocks and combine LTL_c and CCSL for specification purpose.

C. Liu proposed a temporal logic with clocks (TLC) [27] for specifying reactive systems with multiple granularities of time, i.e., multiple clocks. TLC extends the traditional LTL by associating each formula to a clock name, like $(\Box\phi)_c$, indicating that the formula $\Box\phi$ is defined at the projected trace, whose states are exactly those where the clock c ticks, of a global trace. By projection, it means that for those states not in the projected trace, formula $\Box\phi$ is not defined. This is different from LTL_c , where a formula, say $\Box_c\phi$, is defined on a global state. Clock c just plays the role of a “filter”, that when talking about satisfaction, excludes those states where c does not tick on a global trace. That is why in LTL_c , we only need to combine clocks to atomic propositions and temporal operators. In LTL_c , we want a formula to describe a trace filtered by different clocks w.r.t. to a global trace, rather than projecting the global trace into a subtrace with a clock, which can be further projected by other clocks. For example, in formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ describing Property (2) in Fig. 2, c_2 does not have to be a subclock of clock c_1 (i.e., in the projective trace by clock c_1). c_1 and c_2 just perform different filtering on the global trace. Instead of a model checking approach as we develop for LTL_c , TLC was mainly developed for verification in a theorem-proving approach [28].

[29] proposed a clock operator @ on LTL, and defined a logic called LTL° . Similar to TLC, in LTL° , each LTL formula ϕ is bound to a clock c by $\phi@c$, meaning that formula ϕ holds along a trace where clock c ticks. The function of a clock in LTL° is similar to LTL_c . A clock acts as a filter that excludes the states where it does not tick w.r.t. a global trace⁵. Although in LTL° it also allows that a clock ticks finitely, the expressiveness of LTL° , like LTL_c , just remains the same as LTL. In LTL_c , according to the scenario we aim at, we think there is no necessity to introduce finite clocks. Because the events representing the notions of time in a distributed system usually do not stop, until meeting a deadlock or being interrupted by other signals or events. In the former case, the behaviour of the system is equivalent by adding a self loop on the deadlocked state so that a clock can keep ticking infinitely; In the latter case, we have a special operator $\curvearrowright$ in CCSL (see Table 2) to express that a clock is interrupted by another clock. Despite sharing the same expressiveness, the “design paradigms” behind the two logics are a bit different. While the Next operator **X** in LTL° works on a global state, all temporal operators in LTL_c can only work on the states filtered by their associated clocks. This is exactly what we want, for a local temporal property should have nothing to do with an irrelevant global state. For example, formula $\mathbf{X}_c\phi$ means that ϕ holds *at the next instant* of clock c , while in LTL° , formula $(X\phi)@c$ means *at the next global state*, c must tick and ϕ holds. This means that formula $(X\phi)@c$ can actually “see” the next global state while $\mathbf{X}_c\phi$ cannot.

6.2. Languages for Modelling/Specifying Multi-clock Systems

The term “multi-clock system” used in this paper denotes a type of distributed real-time systems which are described by discrete time models and in which different components are based on different granularities of time. Below we list some famous specification/modelling languages which have been used to specify/model multi-clock systems from different aspects and at different abstraction levels. We mainly discuss two languages — MARTE/CCSL and Signal — which are closely related to the design of our proposed LTL_c /CCSL. We make comparisons between them and LTL_c /CCSL.

MARTE [10] is a profile of UML for modelling and analyzing real-time and embedded systems. It extends the time model of UML, which only considers physical time, to a multiform time model that considers both physical and logical time [30]. In MARTE, clock is the basic element to model different forms of time. A clock can be referred to different time bases. For example, an ideal clock is referred to a denser time domain to model continuous physical time; A logical clock is referred to a discrete time domain to model a sequence

⁵ rather than the word “filter” used in this paper, in [29], the authors rather called it “projection”. But note that it is not the same type of projections as in TLC.

of events. MARTE supports modelling heterogenous systems, where different notions of time of different components can be referred to different clocks. These clocks can be coordinated by CCSL, a companion language of MARTE which was invented to facilitate expressing complex logical constraints between clocks. In our work, we use CCSL to specify the logical constraints between clocks in LTL_c formulae. Compared to MARTE/CCSL, $LTL_c/CCSL$ is not intended to be a modelling language. It rather focuses on high-level specification and only supports modelling logical time.

Signal is a synchronous programming language that was designed for high-level specification of reactive systems [7]. It is a data-flow language whose semantics is based on equations of vectors. Each vector represents an infinite sequence of data and is associated to a clock that captures its occurrence at each instant. An equation of vectors captures a constraint (or say a relation) between the clocks and the values of the vectors. In Signal, a system is made of a set of equations, where different clocks (of vectors) are constrained through the equations in a loose way. Unlike other synchronous languages such as Lustre [31] and Esterel [6] which require the schedule of clocks to be unique, in Signal, the behaviour of a clock at each instant is non-deterministic and thus clocks form a partial-ordered time model. Signal has been widely used in academic and industry communities with the support of a powerful tool [32]. Compared to Signal, the semantics of $LTL_c/CCSL$ is based on automata, rather than a set of mathematical equations. $LTL_c/CCSL$ separates the concerns of the clocks and values of data, while in Signal clock constraints are indicated by the equations themselves. It is interesting to compare their expressiveness, which could be one of our future work.

Other data-flow languages such as Ptolemy and Giotto also reflect the multi-clock feature. Ptolemy [9] is a modelling language that supports heterogeneous modelling/specification of embedded systems. It is based on an actor model — a dataflow-style model that supports different types of MoCs which model both continuous and discrete dynamics. Ptolemy supports a type of dataflow models called *synchronous-reactive models* [22]. As synchronous languages like Signal, it supports the synchronous execution mechanism in which the executions of events take no time and several events can occur simultaneously. Another type of dataflow models in Ptolemy called *dataflow models of computation* [22] focus on the dependency between the flows of data but ignore discussing the relations between different flows. Giotto [33] is similar to synchronous-reactive model of Ptolemy but the executions of events take time instead of being instantaneous and simultaneous. Each event is triggered at the ticks of some clock but is also associated with a logical execution time. Giotto mainly focuses on the real time constraints of embedded systems rather than the logical time constraints considered in Signal.

Comparing all of these languages above with $LTL_c/CCSL$, none of them stresses the multi-clock feature in a model of temporal logics.

7. Conclusion and Future Work

In this paper, we mainly propose a specification language $LTL_c/CCSL$ for specifying and verifying multi-clock systems, in which different components are based on local clocks that forms a partially-ordered time model. We first build the syntax and semantics of LTL_c and introduce an adaptive semantics of CCSL independently, then combine them together to obtain $LTL_c/CCSL$. After defining $LTL_c/CCSL$, we propose a model checking approach to verify $LTL_c/CCSL$ specifications based on the model checking theory of LTL and the observation theory of CCSL. And we illustrate how $LTL_c/CCSL$ can be used to specify local temporal properties of a multi-clock system through an example.

We mainly consider our future work in the following three aspects:

1. The next step of our work is to apply $LTL_c/CCSL$ in specifying and verifying multi-clock systems in practice, such as Signal programs, Ptolemy models, MARTE models, etc. Relevant verification prototype tools can be developed for $LTL_c/CCSL$ according to the specific application domains. We also consider implementing $LTL_c/CCSL$ in famous model checking tools like Spin [34] or Maude [35], or consider developing a translator from $LTL_c/CCSL$ to the languages accepted by these tools.
2. In this paper, we only consider logical clocks. We also consider to extend LTL_c with the *ideal clock* in CCSL, to also capture physical time in a multi-clock system. The ideal clock in CCSL models the global continuous physical time. Based on the ideal clock, we can define a discrete clock, whose ticks are tagged with physical time values, using an operator called “DiscretizedBy” [13]. For example, the expression $c \triangleq \text{idealClock DiscretizedBy } 1s$ means to define a clock c that ticks every 1s. Since *idealClock*

is continuous, CTS/CTSA is no longer enough for describing the semantics of the extended LTL_c . A timed version of CTS/CTSA would be required, which is based on timed automata [36].

3. Another future work would focus on the runtime verification of $LTL_c/CCSL$ specifications. To this end, we shall propose a finite-trace semantics for $LTL_c/CCSL$ and an $LTL_c/CCSL$ monitoring algorithm. A prototype tool can also be developed.

8. Acknowledgements

We thank Prof. Yixiang Chen and Huibiao Zhu for their comments and advises about this work.

This work is partially supported by the Youth Project of National Science Foundation of China (No. 62102329), the Project of National Science Foundation of Chongqing (No. cstc2021jcyj-bsh0204), the Capacity Development Grant of Southwest University (No. SWU116007), and the Key Projects of National Science Foundation of China (No. 61732019, No. 62032019).

References

- [1] E. A. Lee, Cyber physical systems: design challenges, in: 2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC), 2008, pp. 363–369.
- [2] K. Ashton, That “Internet of Things” thing, RFID Journal 22 (7) (2009) 97–114.
- [3] E. A. Lee, A. L. Sangiovanni-Vincentelli, A framework for comparing models of computation, IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 17 (12) (1998) 1217–1229.
- [4] A. Gamatié, T. Gautier, P. Le Guernic, J. Talpin, Polychronous design of embedded real-time applications, ACM Trans. Softw. Eng. Methodol. 16 (2) (2007) 9.
- [5] L. Lamport, Time, clocks, and the ordering of events in a distributed system, Commun. ACM 21 (7) (1978) 558–565.
- [6] G. Berry, G. Gonthier, The Esterel synchronous programming language: design, semantics, implementation, Science of Computer Programming 19 (2) (1992) 87–152.
- [7] A. Benveniste, P. Le Guernic, C. Jacquemot, Synchronous programming with events and relations: the SIGNAL language and its semantics, Science of Computer Programming 16 (2) (1991) 103–149.
- [8] <https://www.mathworks.com/products/simulink.html>.
- [9] C. Ptolemaeus (Ed.), System Design, Modeling, and Simulation using Ptolemy II, Ptolemy.org, 2014.
- [10] OMG, UML profile for MARTE: Modeling and analysis of real-time embedded systems, Tech. rep., OMG (2011).
- [11] P. Le Guernic, J.-P. Talpin, J.-C. Le Lann, Polychrony for system design, Journal of Circuits, Systems and Computers 12 (03) (2003) 261–303.
- [12] E. A. Lee, M. Lohstroh, Time for all programs, not just real-time programs, in: T. Margaria, B. Steffen (Eds.), Leveraging Applications of Formal Methods, Verification and Validation, Springer International Publishing, Cham, 2021, pp. 213–232.
- [13] F. Mallet, Clock constraint specification language: specifying clock constraints with UML/MARTE, Innovations in Systems and Software Engineering 4 (3) (2008) 309–314.
- [14] A. Pnueli, The temporal logic of programs, in: 18th Annual Symposium on Foundations of Computer Science (sfcs 1977), 1977, pp. 46–57.
- [15] R. Alur, T. Henzinger, Real-time logics: complexity and expressiveness, Information and Computation 104 (1) (1993) 35–77.
- [16] R. Alur, T. Feder, T. A. Henzinger, The benefits of relaxing punctuality, J. ACM 43 (1) (1996) 116–146.
- [17] F. Mallet, R. de Simone, Correctness issues on MARTE/CCSL constraints, Science of Computer Programming 106 (2015) 78–92.
- [18] C. Baier, J. P. Katoen, Principles of Model Checking, The MIT Press, 2008.
- [19] C. André, Syntax and semantics of the clock constraint specification language (CCSL), Research Report RR-6925, INRIA (2009).
- [20] R. Gascon, F. Mallet, J. Deantoni, Logical time and temporal logics: Comparing UML MARTE/CCSL and PSL, in: 2011 Eighteenth International Symposium on Temporal Representation and Reasoning, 2011, pp. 141–148.
- [21] F. Boulanger, A. Dogui, C. Hardebolle, C. Jacquet, D. Marcadet, I. Prodan, Semantic adaptation using CCSL clock constraints, in: J. Kienzle (Ed.), Models in Software Engineering, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 104–118.
- [22] E. A. Lee, S. A. Seshia, Introduction to Embedded Systems: A Cyber-physical Systems Approach, MIT Press, 2011.
- [23] F. Mallet, J. Millo, R. de Simone, Safe CCSL specifications and marked graphs, in: 11th ACM/IEEE International Conference on Formal Methods and Models for Codesign, MEMCODE 2013, IEEE, 2013, pp. 157–166.
- [24] O. Maler, D. Nickovic, Monitoring temporal properties of continuous signals, in: Y. Lakhnech, S. Yovine (Eds.), Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 152–166.
- [25] A. Donzé, O. Maler, E. Bartocci, D. Nickovic, R. Grosu, S. Smolka, On temporal logic and signal processing,

- in: S. Chakraborty, M. Mukund (Eds.), *Automated Technology for Verification and Analysis*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 92–106.
- [26] L. Brim, P. Dluho, D. afnek, T. Vejpustek, STL*: extending signal temporal logic with signal-value freezing operator, *Information and Computation* 236 (2014) 52 – 67, special Issue on Hybrid Systems and Biology.
- [27] C. Liu, M. A. Orgun, Dealing with multiple granularity of time in temporal logic programming, *J. Symb. Comput.* 22 (5/6) (1996) 699–720.
- [28] C. Liu, M. A. Orgun, Verification of reactive systems using temporal logic with clocks, *Theoretical Computer Science* 220 (2) (1999) 377–408.
- [29] C. Eisner, D. Fisman, J. Havlicek, A. McIsaac, D. Van Campenhout, The definition of a temporal clock operator, in: J. C. M. Baeten, J. K. Lenstra, J. Parrow, G. J. Woeginger (Eds.), *Automata, Languages and Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 857–870.
- [30] C. André, F. Mallet, R. de Simone, Modeling time(s), in: G. Engels, B. Opdyke, D. C. Schmidt, F. Weil (Eds.), *Model Driven Engineering Languages and Systems*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 559–573.
- [31] P. Caspi, D. Pilaud, N. Halbwachs, J. A. Plaice, LUSTRE: a declarative language for real-time programming, in: *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, POPL '87*, Association for Computing Machinery, New York, NY, USA, 1987, pp. 178–188.
- [32] <http://www.irisa.fr/espresso/Polychrony>.
- [33] T. A. Henzinger, B. Horowitz, C. M. Kirsch, Giotto: A time-triggered language for embedded programming, in: T. A. Henzinger, C. M. Kirsch (Eds.), *Embedded Software*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 166–184.
- [34] G. J. Holzmann, The model checker SPIN, *IEEE Trans. Softw. Eng.* 23 (5) (1997) 279–295.
- [35] M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Oliet, J. Meseguer, C. Talcott, *All About Maude - A High-Performance Logical Framework, How to Specify, Program and Verify Systems in Rewriting Logic*, Vol. 4350 of *Lecture Notes in Computer Science*, Springer, 2007.
- [36] R. Alur, D. L. Dill, A theory of timed automata, *Theor. Comput. Sci.* 126 (2) (1994) 183–235.

A. Preliminaries of LTL and Ordinary Transition Systems

The syntax of linear temporal logic (LTL) is given as follows:

$$\phi ::= p \mid \neg\phi \mid \phi \wedge \phi \mid \mathbf{X}\phi \mid \phi\mathbf{U}\phi,$$

where $\mathbf{X}$, $\mathbf{U}$ are two temporal operators called *next* and *until* respectively. Intuitively, $\mathbf{X}\phi$ means that along a trace, the next state satisfies ϕ , while $\phi_1\mathbf{U}\phi_2$ means that along a trace, there exists a state satisfying ϕ_2 , and before that state all states satisfy ϕ_1 .

An ordinary configuration is a pair $\langle\pi, \theta\rangle$ where π is a trace defined in Def. 2.3 and $\theta : S(\pi) \rightarrow 2^\Pi$ is a mapping from each state of the trace to a set of atomic propositions just defined in Def. 2.3.

Given an LTL formula ϕ , for any configuration $\langle\pi, \theta\rangle$, the trace semantics of ϕ , denoted by $\pi \models_\theta \phi$, is defined as follows:

- (1) $\pi \models_\theta p$ iff $p \in \theta(\pi[0])$.
- (2) $\pi \models_\theta \neg\phi$ iff $\pi \not\models_\theta \phi$.
- (3) $\pi \models_\theta \phi_1 \wedge \phi_2$ iff $\pi \models_\theta \phi_1$ and $\pi \models_\theta \phi_2$.
- (4) $\pi \models_\theta \mathbf{X}\phi$ iff $\pi[1, \infty] \models_\theta \phi$.
- (5) $\pi \models_\theta \phi_1\mathbf{U}\phi_2$ iff there exists an $i \geq 0$ s.t. $\pi[i, \infty] \models_\theta \phi_2$, and for all $0 \leq j < i$, $\pi[j, \infty] \models_\theta \phi_1$.

An ordinary transition system (OTS) is a 5-tuple $\mathcal{N} = \langle\Pi, Q, I, \theta, \mathcal{T}\rangle$ where all components Π , Q , I , θ and $\mathcal{T}$ are defined just as in Def. 5.1. $\text{Trace}(\mathcal{N})$, $\pi[i]$, $\pi[0, i]$, $\pi[i, \infty]$, $\pi[e]$ and $\text{Trace}(q)$ are defined just as their counterparts in CTS in Sect. 5.1.

A Büchi automaton is an OTS with a set of accepting states. It can be defined as a 6-tuple $\mathcal{N}(\mathcal{E}) = \langle\Pi, Q, I, \theta, \mathcal{T}, \mathcal{E}\rangle$, where $\mathcal{E} \subseteq Q$. A trace π is accepted by $\mathcal{N}(\mathcal{E})$ is defined similarly as in CTSA in Def. 5.7.

OTSs can be seen as a special type of Büchi automata where all states are accepting states.

Given an OTS $\mathcal{N} = \langle\Pi, Q, I, \theta, \mathcal{T}\rangle$ and an LTL formula ϕ , for any trace $\pi \in \text{Trace}(\mathcal{N})$, the satisfaction relation between $\mathcal{N}$ and ϕ , denoted by $\pi \models_{\mathcal{N}} \phi$, can be defined as:

$$\pi \models_{\mathcal{N}} \phi \text{ iff } \pi \models_\theta \phi.$$

For any state $q \in Q$, the semantics of the LTL formula ϕ : $q \models_{\mathcal{N}} \phi$, is defined s.t.

$$\text{for any } \pi \in \text{Trace}(q), \pi \models_{\mathcal{N}} \phi.$$

B. Proofs of some Lemmas, Propositions and Theorems

Fig. 5 shows the dependence relations between the main lemmas, propositions and theorems in this paper.

The proof of Prop. 5.1 Let $N_1 = (\mathcal{M}_1 \parallel \mathcal{M}_2) \parallel \mathcal{M}_3 = \langle \Pi^1, Clk^1, Q^1, I^1, \theta^1, \xi^1, \mathcal{T}^1 \rangle$ and $N_2 = \mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \mathcal{M}_3) = \langle \Pi^2, Clk^2, Q^2, I^2, \theta^2, \xi^2, \mathcal{T}^2 \rangle$, obviously $I_1 = I_2$, $Clk^1 = Clk^2$, $I^1 = I^2$, $\theta^1 = \theta^2$ and $\xi^1 = \xi^2$ can be easily proved provided that $Q^1 = Q^2$. Therefore, it is sufficient to only show that $Q^1 = Q^2$ and $\mathcal{T}^1 = \mathcal{T}^2$, which are equivalent to prove that for any state q ,

- (1) q is an initial state of N_1 iff q is an initial state of N_2 .
- (2) If q is in both N_1 and N_2 , then $\langle q, q' \rangle$ is a transition of N_1 iff $\langle q, q' \rangle$ is a transition of N_2 .

In the following proof of (1) and (2), we only consider the condition involving clock behaviours and ignore the corresponding condition involving atomic propositions since it is similar.

(1): For any initial state u of N_1 , let $u = \langle g, u_3 \rangle$ with g being a state of $\mathcal{M}_1 \parallel \mathcal{M}_2$ and u_3 being a state of $\mathcal{M}_3$, by Def. 5.4 we have

$$Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3) = Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g). \quad (2)$$

Since $g \in \mathcal{M}_1 \parallel \mathcal{M}_2$, let $g = \langle u_1, u_2 \rangle$ with u_1 being a state of $\mathcal{M}_1$ and u_2 being a state of $\mathcal{M}_2$, again by Def. 5.4 we have

$$Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u_1). \quad (3)$$

By equations 2) and (3) now we show that

- (a) $Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$ and
- (b) $Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1) = Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$.

For (a), on one hand, for any $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2)$, we have $t \in \xi_{\mathcal{M}_2}(u_2) \subseteq \xi_{\mathcal{M}_1}(u_1) \cup \xi_{\mathcal{M}_2}(u_2) = \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$, hence $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. By equation (2) we know that $t \in \xi_{\mathcal{M}_3}(u_3)$ and $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} = Clk_{\mathcal{M}_1} \cup Clk_{\mathcal{M}_2}$. If $t \in Clk_{\mathcal{M}_2}$, then we immediately get $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. If $t \in Clk_{\mathcal{M}_1}$, then $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2)$, by equation (3) we still have $t \in Clk_{\mathcal{M}_2}$. On the other hand, for any $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$, we have $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. By equation (2) we can get $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. So $t \in \xi_{\mathcal{M}_1}(u_1) \cup \xi_{\mathcal{M}_2}(u_2)$. If $t \in \xi_{\mathcal{M}_2}(u_2)$ we immediately get $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2)$. If $t \in \xi_{\mathcal{M}_1}(u_1)$, by equation (3) we also have $t \in \xi_{\mathcal{M}_2}$.

For (b), on one hand, for any $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$, if $t \in Clk_{\mathcal{M}_2}$, by equation (3) we have $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2)$, therefore $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$. If $t \in Clk_{\mathcal{M}_3}$, since $t \in \xi_{\mathcal{M}_1}(u_1)$, $t \in \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$, so by equation (2) there is $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. From equation (3) and the fact that $t \in \xi_{\mathcal{M}_1}(u_1)$, we can see that $t \in Clk_{\mathcal{M}_1}$ must hold. Hence $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$. On the other hand, for any $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$, if $t \in \xi_{\mathcal{M}_2}(u_2)$, then by equation (3) we know $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u_1)$, so $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$. If $t \in \xi_{\mathcal{M}_3}(u_3)$, by equation (2) there is $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. So $t \in \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. From equation (3) and the fact that $t \in Clk_{\mathcal{M}_1}$ we can see that $t \in \xi_{\mathcal{M}_1}(u_1)$ must hold. Therefore $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$.

According to Def. 5.4, from (a) we know that $\langle u_2, u_3 \rangle$ is an initial state of $\mathcal{M}_2 \parallel \mathcal{M}_3$, and from (b) we know that q is an initial state of N_2 .

The other direction can be proved in a similar way and we omit it here.

(2): For any state q of both N_1 and N_2 and any transition $\langle q, q' \rangle$ of N_1 , let $q' = \langle g', u'_3 \rangle$, by Def. 5.4 we have

$$Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u'_3) = Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g'). \quad (4)$$

Let $g' = \langle u'_1, u'_2 \rangle$, again by Def. 5.4 we have

$$Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u'_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u'_1). \quad (5)$$

Following a similar proof procedure as the proof of (a) and (b) shown above in the proof of (1) (by replacing u_1, u_2, u_3, g with u'_1, u'_2, u'_3, g' everywhere), we can prove that

- (a') $Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u'_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u'_3)$ and
- (b') $Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u'_1) = Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u'_2, u'_3 \rangle)$.

According to Def. 5.4, from (a') we know that $\langle u'_2, u'_3 \rangle$ is an initial state of $\mathcal{M}_2 \parallel \mathcal{M}_3$, so $\langle q|_{\mathcal{M}_2 \parallel \mathcal{M}_3}, \langle u'_2, u'_3 \rangle \rangle$ is a transition of $\mathcal{M}_2 \parallel \mathcal{M}_3$. From (b') we know that q' is a state of N_2 , so $\langle q, q' \rangle$ is a transition of N_2 .

□

The proof of Prop. 5.6 Firstly, since $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, we have $\text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$. So $C_2 \cap F_1 = \emptyset$. Thus we have

$$\begin{aligned} F &= (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2)) \\ &= (F_1 \cup F_2) - ((F_1 \cap C_1) \cup (F_1 \cap C_2) \cup (F_2 \cap C_1) \cup (F_2 \cap C_2)) \\ &= (F_1 \cup F_2) - (\emptyset \cup \emptyset \cup (F_2 \cap C_1) \cup \emptyset) \\ &= (F_1 \cup F_2) - (F_2 \cap C_1). \end{aligned}$$

Now we show that F and C are sets of free clocks and constrained clocks of the observer $\mathcal{M}$ respectively.

Let $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$, for any clock behaviour $X \subseteq F$, we need to prove that

- (1) there exists an initial state q_0 of $\mathcal{M}$ s.t. $\xi(q_0) \cap F = X$;
- (2) for any state q of $\mathcal{M}$, there exists a transition $\langle q, q' \rangle$ s.t. $\xi(q') \cap F = X$.

(1): Since $X \cap F_1 \subseteq F_1$, so by Def. 5.12 there exists an initial state x_0 of $\mathcal{M}_1$ s.t. $\xi_1(x_0) \cap F_1 = X \cap F_1$. Let $K = (X \cap F_2) \cup (\xi_1(x_0) \cap F_2) \subseteq F_2$, by Def. 5.12 we know there is an initial state y_0 of $\mathcal{M}_2$ s.t. $\xi_2(y_0) \cap F_2 = K$. Now we prove that $\text{Clk}_2 \cap \xi_1(x_0) = \text{Clk}_1 \cap \xi_2(y_0)$:

On one hand, for any $t \in \text{Clk}_2 \cap \xi_1(x_0)$, since $t \in \xi_1(x_0) \subseteq \text{Clk}_1$, so $t \in \text{Clk}_1 \cap \text{Clk}_2$. By $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$ we know $t \in \text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$. So $t \in \xi_1(x_0) \cap F_2 \subseteq K$. Since $\xi_2(y_0) \cap F_2 = K$, $t \in \xi_2(y_0)$. Therefore $t \in \text{Clk}_1 \cap \xi_2(y_0)$. On the other hand, for any $t \in \text{Clk}_1 \cap \xi_2(y_0)$, we get $t \in \xi_2(y_0) \subseteq \text{Clk}_2$. Since $\text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$, so $t \in \text{Clk}_1 \cap \xi_2(y_0) \subseteq \text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$. Because $t \in \xi_2(y_0)$ and $t \in F_2$, so $t \in K$, thus $t \in (X \cap F_2) \cup (\xi_1(x_0) \cap F_2)$. Consider t for two cases: 1) If $t \in X \cap F_2$, then easy to see that $t \in F_1$ from the fact that $t \notin C_1$ but $t \in \text{Clk}_1$. So $t \in X \cap F_1 = \xi_1(x_0) \cap F_1 \subseteq \xi_1(x_0)$. Hence $t \in \text{Clk}_2 \cap \xi_1(x_0)$. 2) If $t \in \xi_1(x_0) \cap F_2$, we immediately obtain $t \in \xi_1(x_0)$. So $t \in \text{Clk}_2 \cap \xi_1(x_0)$.

According to Def. 5.4, we know that $\langle x_0, y_0 \rangle$ is an initial state of $\mathcal{M}$. Next we show that $\xi(\langle x_0, y_0 \rangle) \cap F = (\xi_1(x_0) \cup \xi_2(y_0)) \cap F = X$:

On one hand, for any $t \in X$, $t \in X \subseteq F$, so $t \in F_1 \cup F_2$. If $t \in F_1$, then $t \in X \cap F_1$, so $t \in \xi_1(x_0) \cap F_1$, similar for the case $t \in F_2$. Hence $t \in \xi_1(x_0) \cup \xi_2(y_0)$. On the other hand, for any $t \in (\xi_1(x_0) \cup \xi_2(y_0)) \cap F$, consider two situations: 1) If $t \in \xi_1(x_0)$, then $t \in \xi_1(x_0) \subseteq \text{Clk}_1$. Because $t \in F$, so $t \in F_1 \cup F_2$ and $t \notin C_1 \cap F_2$. Easy to see that $t \in F_1$, otherwise if $t \notin F_1$, then $t \in F_2$. Since $t \notin C_1 \cap F_2$, so $t \notin C_1$. But $t \in \text{Clk}_1$, so it must be $t \in F_1$, which causes the contradiction. From $t \in \xi_1(x_0)$ and $t \in F_1$ we obtain $t \in \xi_1(x_0) \cap F_1 = X \cap F_1$, thus $t \in X$. 2) If $t \in \xi_2(y_0) \subseteq \text{Clk}_2$, easy to see that $t \in F_2$. If not so, since $t \in F = (F_1 \cup F_2) - (C_1 \cap F_2)$, we know that $t \in F_1$. So there is $t \in \xi_2(y_0) \cap F_1 \subseteq \text{Clk}_2 \cap \text{Clk}_1$. But $\text{Clk}_2 \cap \text{Clk}_1 \subseteq F_2$ due to $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, which contradicts the assumption that $t \notin F_2$. From $t \in \xi_2(y_0)$ and $t \in F_2$ we have $t \in \xi_2(y_0) \cap F_2 = K = (X \cap F_2) \cup (\xi_1(x_0) \cap F_2)$. Since $t \notin \xi_1(x_0)$, we have $t \in X \cap F_2$, hence $t \in X$.

(2): For any state $\langle x, y \rangle$ of $\mathcal{M}$, since $X \cap F_1 \subseteq F_1$, so by Def. 5.12 there exists a transition $\langle x, x' \rangle$ in $\mathcal{M}_1$ s.t. $\xi_1(x') \cap F_1 = X \cap F_1$. Let $K = (X \cap F_2) \cup (\xi_1(x') \cap F_2) \subseteq F_2$, by Def. 5.12 we know there is a transition $\langle y, y' \rangle$ in $\mathcal{M}_2$ s.t. $\xi_2(y') \cap F_2 = K$. By following a similar procedure of the proof of $\text{Clk}_2 \cap \xi_1(x_0) = \text{Clk}_1 \cap \xi_2(y_0)$ above in (1) (explicitly, by simply replacing x_0, y_0 with x' and y' respectively), we can prove $\text{Clk}_2 \cap \xi_1(x') = \text{Clk}_1 \cap \xi_2(y')$. Therefore $\langle x', y' \rangle$ is a state of $\mathcal{M}$, and according to Def. 5.4 $\langle \langle x, y \rangle, \langle x', y' \rangle \rangle$ is a transition of $\mathcal{M}$. Again, following a similar procedure of the proof of $\xi(\langle x_0, y_0 \rangle) \cap F = X$ above in (1) (by simply replacing x_0, y_0 with x' and y' respectively), we can prove $\xi(\langle x', y' \rangle) \cap F = X$.

For C , we have $C = \text{Clk} - F = ((\text{Clk}_1 \cup \text{Clk}_2) - (F_1 \cup F_2)) \cup (C_1 \cap F_2) = (C_1 \cup C_2) \cup (C_1 \cap F_2) = C_1 \cup C_2$.

□

Lemma B.1 shows a relationship between the observation relation and the dependence relation.

Lemma B.1. Given two observers $\mathcal{M}_1$ and $\mathcal{M}_2$, $\mathcal{M}_1 \text{ obst } \mathcal{M}_2$ iff

- a) $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$ and $\mathcal{M}_2 \not\rightsquigarrow \mathcal{M}_1$, or
- b) $\mathcal{M}_1, \mathcal{M}_2$ are directly independent, i.e., $\mathcal{M}_1 \not\rightsquigarrow \mathcal{M}_2$ and $\mathcal{M}_2 \not\rightsquigarrow \mathcal{M}_1$.

Proof. Let $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, \text{Clk}_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$), it is not hard to see that

$$\text{Clk}_1 \cap \text{Clk}_2 \subseteq F_1 \text{ iff } C_1 \cap \text{Clk}_2 = \emptyset \text{ iff } C_1 \cap \text{Clk}_2 = \emptyset \text{ and } C_2 \cap \text{Clk}_1 \supseteq \emptyset.$$

38

Yuanrui Zhang, Frédéric Mallet, Min Zhang and Zhiming Liu

If $C_1 \cap Clk_2 = \emptyset$ and $C_2 \cap Clk_1 \supset \emptyset$, then a) holds. If $C_1 \cap Clk_2 = \emptyset$ and $C_2 \cap Clk_1 = \emptyset$, then b) holds.

□

The proof of Thm. 5.1 relies on Lemma B.1, Lemma B.2 and Lemma B.3.

Lemma B.2. Given a finite set of observers A ($|A| \geq 2$), if A is well defined, then there exist $\mathcal{M}_1, \mathcal{M}_2 \in \Phi$ s.t. $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$ or $\mathcal{M}_1 \uparrow \mathcal{M}_2$.

Proof. We show that in A , if there exists a relation $\mathcal{M}_1 \rightsquigarrow \mathcal{N}_1$ between observers $\mathcal{M}_1$ and $\mathcal{N}_1$, then there must exist a relation $\mathcal{O}_1 \rightsquigarrow^o \mathcal{O}_2$ for some observers $\mathcal{O}_1$ and $\mathcal{O}_2$.

Define two properties:

- $P_1(A, \mathcal{M})$: A is a set of observers in which each observer $\mathcal{O}$ satisfies $\mathcal{O} = \mathcal{M}$ or $\mathcal{O} \rightsquigarrow^+ \mathcal{M}$.
- $P_2(A, \mathcal{M})$: A is a set of observers in which each observer $\mathcal{O}$ satisfies $\mathcal{O} = \mathcal{M}$ or $\mathcal{M} \rightsquigarrow^+ \mathcal{O}$.

Set $D_1 = \{\mathcal{M}_1\}$, $E_1 = \{\mathcal{N}_1\}$, clearly $P_1(D_1, \mathcal{M}_1)$ and $P_2(E_1, \mathcal{N}_1)$ hold.

For the initial step (step 1), suppose $\mathcal{M}_1 \not\rightsquigarrow^o \mathcal{N}_1$, then by Def. 5.13 there is an observer $\mathcal{N}_2$ different from all elements in D_1 and E_1 s.t. $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{N}_2 \rightsquigarrow^+ \mathcal{N}_1$. Without loss of generality we can assume there is an observer $\mathcal{M}_2$ s.t. $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}_2 \rightsquigarrow \mathcal{N}_2$, where $\mathcal{M}_2$ may equal to $\mathcal{M}_1$. Let $D_2 = D_1 \cup \{\mathcal{M}_2\}$ and $E_2 = E_1 \cup \{\mathcal{N}_2\}$, it is easy to see that $P_1(D_2, \mathcal{M}_2)$ and $P_2(E_2, \mathcal{N}_2)$ hold.

Now suppose that at any step n ($n \geq 2$), there are $\mathcal{M}_n \rightsquigarrow \mathcal{N}_n$, D_n and E_n s.t. $P_1(D_n, \mathcal{M}_n)$ and $P_2(E_n, \mathcal{N}_n)$ hold. Suppose $\mathcal{M}_n \not\rightsquigarrow^o \mathcal{N}_n$, then by Def. 5.13 we know that there is an observer $\mathcal{N}_{n+1}$ different from $\mathcal{M}_n$ and $\mathcal{N}_n$ s.t. $\mathcal{M}_n \rightsquigarrow^+ \mathcal{N}_{n+1} \rightsquigarrow^+ \mathcal{N}_n$. It is not hard to see that $\mathcal{N}_{n+1}$ must not equal to other elements in both D_n and E_n , or a contradiction will occur. For example, if $\mathcal{N}_{n+1} \in D_n$, then by $P_1(D_n, \mathcal{M}_n)$ we have $\mathcal{N}_{n+1} \rightsquigarrow^+ \mathcal{M}_n$, which contradicts the fact that A is well defined. Similar for $\mathcal{N}_{n+1} \in E_n$. Therefore $\mathcal{N}_{n+1} \notin D_n \cup E_n$. Without loss of generality we can assume there is an observer $\mathcal{M}_{n+1}$ s.t. $\mathcal{M}_n \rightsquigarrow^+ \mathcal{M}_{n+1} \rightsquigarrow \mathcal{N}_{n+1}$, where $\mathcal{M}_{n+1}$ may equal to $\mathcal{M}_n$. Let $D_{n+1} = D_n \cup \{\mathcal{M}_{n+1}\}$ and $E_{n+1} = E_n \cup \{\mathcal{N}_{n+1}\}$, it is easy to see that $P_1(D_{n+1}, \mathcal{M}_{n+1})$ and $P_2(E_{n+1}, \mathcal{N}_{n+1})$ hold.

Continuing the steps we can see that if for every $\mathcal{M}_n \rightsquigarrow \mathcal{N}_n$ ($n \geq 1$), $\mathcal{M}_n \not\rightsquigarrow^o \mathcal{N}_n$, then the set $D_n \cup E_n$ will grow to infinitely large which violates the assumption that A is finite. So there must be $\mathcal{M}_i \rightsquigarrow^o \mathcal{N}_i$ for some $i \geq 1$.

□

Lemma B.3. Given a set of observers A and any two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle \in A$ ($i = 1, 2$) s.t. either $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$ or $\mathcal{M}_1 \uparrow \mathcal{M}_2$ holds, if A is well defined, then $A' = (A \cup \{\mathcal{M}_1 \parallel \mathcal{M}_2\}) - \{\mathcal{M}_1, \mathcal{M}_2\}$ is also well defined.

Proof. Let $\mathcal{M} = \mathcal{M}_1 \parallel \mathcal{M}_2$. Suppose A' is not well defined, then there must exist a relation $\mathcal{M} \rightsquigarrow \mathcal{M}''$ for some observer $\mathcal{M}'' \in A'$. Because if the interdependence relation happens between two observers $\mathcal{N}_1$ and $\mathcal{N}_2$ that is different from $\mathcal{M}$, there must be a path $\mathcal{N}_1 \rightsquigarrow^+ \mathcal{M} \rightsquigarrow^+ \mathcal{N}_2$ or $\mathcal{N}_2 \rightsquigarrow^+ \mathcal{M} \rightsquigarrow^+ \mathcal{N}_1$ that has something to do with $\mathcal{M}$, otherwise $\mathcal{N}_1$ and $\mathcal{N}_2$ are dependent on each other in the old set A , which contradicts the assumption that A is well defined. Assume $\mathcal{N}_1 \rightsquigarrow^+ \mathcal{M} \rightsquigarrow^+ \mathcal{N}_2$, then we have $\mathcal{M} \rightsquigarrow^+ \mathcal{N}_2$ and $\mathcal{N}_2 \rightsquigarrow^+ \mathcal{N}_1 \rightsquigarrow^+ \mathcal{M}$, hence $\mathcal{M} \rightsquigarrow \mathcal{N}_2$.

Suppose $\mathcal{M} \rightsquigarrow \mathcal{M}''$, without loss of generality, we can assume there are two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle \in A$ ($i = 3, 4$) satisfying $\mathcal{M} \rightsquigarrow \mathcal{M}_3 \rightsquigarrow^+ \mathcal{M}'' \rightsquigarrow^+ \mathcal{M}_4 \rightsquigarrow \mathcal{M}$. With $\mathcal{M} \rightsquigarrow \mathcal{M}_3$ and $\mathcal{M}_4 \rightsquigarrow \mathcal{M}$, by Def. 5.13 and Prop.5.6, we have

$$C_3 \cap (Clk_1 \cup Clk_2) \neq \emptyset \text{ and } (C_1 \cup C_2) \cap Clk_4 \neq \emptyset$$

hold. If $C_3 \cap Clk_1 \neq \emptyset$ and $C_1 \cap Clk_4 \neq \emptyset$ hold (similar for the case $C_3 \cap Clk_2 \neq \emptyset$ and $C_2 \cap Clk_4 \neq \emptyset$), by Def. 5.13 we know that $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_3 \rightsquigarrow^+ \mathcal{M}'' \rightsquigarrow^+ \mathcal{M}_4 \rightsquigarrow \mathcal{M}_1$, which contradicts the fact that A is well defined. If $C_3 \cap Clk_1 = \emptyset$ and $C_2 \cap Clk_4 \neq \emptyset$ hold, then by Def. 5.13 there is $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_3 \rightsquigarrow^+ \mathcal{M}'' \rightsquigarrow^+ \mathcal{M}_4 \rightsquigarrow \mathcal{M}_2$, which contradicts the fact that $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$. If $C_3 \cap Clk_2 \neq \emptyset$ and $C_1 \cap Clk_4 \neq \emptyset$ hold, then by Def. 5.13 $\mathcal{M}_2 \rightsquigarrow \mathcal{M}_3 \rightsquigarrow^+ \mathcal{M}'' \rightsquigarrow^+ \mathcal{M}_4 \rightsquigarrow \mathcal{M}_1$ hold, which contradicts the fact that A is well defined.

In summary, A' must be well defined.

□

The proof of Thm. 5.1 relies on Lemma B.1, B.2 and B.3.

The proof of Thm. 5.1 We proceed by induction on the size of $A^{\sim \Phi}$. When $|A^{\sim \Phi}| = 2$, let $A^{\sim \Phi} = \{\mathcal{M}_1[F_1, C_1], \mathcal{M}_2[F_2, C_2]\}$, by Lemma B.1 we know that either $\mathcal{M}_1 \text{ obst } \mathcal{M}_2$ or $\mathcal{M}_2 \text{ obst } \mathcal{M}_1$ holds. Assume $\mathcal{M}_2 \text{ obst } \mathcal{M}_1$,

by Prop. 5.6 we obtain $\mathcal{M}[F, C] = \mathcal{M}_1 \parallel \mathcal{M}_2$, where $F = (F_1 \cup F_2) - (C_1 \cap F_2)$. Since $\mathcal{M}_2 \text{ obs } \mathcal{M}_1$, it is easy to see that $C_2 \cap F_1 = \emptyset$. Therefore $F = (F_1 \cup F_2) - ((C_1 \cap F_2) \cup (C_2 \cap F_1) \cup (C_1 \cap F_1) \cup (C_2 \cap F_2)) = (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2))$, $C = \text{clk}_{\mathcal{M}} - F = C_1 \cup C_2$. Now we show that this proposition holds for any $|A^{\sim\Phi}| > 2$, under the assumption that it holds for any B with $|B| < |A^{\sim\Phi}|$.

According to the associativity of operator $\parallel$ (Def. 5.1), by Lemma B.2 and B.3 we know that by gradually making product between two observers that have a direct dependent relationship, we can obtain a product of $|A^{\sim\Phi}| - 1$ observers: $\mathcal{M}[F, C] = \parallel_{C_n^+ \in \sim\Phi'} \mathcal{M}^{C_n^+}$, where $\sim\Phi'$ is the set of the constraints of the composed observers, it satisfies that $\sim\Phi' \subseteq (\sim\Phi)$ and $|\sim\Phi'| = |\sim\Phi| - 1 = |A^{\sim\Phi}| - 1$. For example, in $A^{\sim\Phi}$, firstly we choose two observers $\mathcal{M}_1, \mathcal{M}_2$ s.t. $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$ or $\mathcal{M}_1 \uparrow \mathcal{M}_2$, we compute $\mathcal{M}_1 \parallel \mathcal{M}_2$ and obtain a new well-defined set $A_1 = A^{\sim\Phi} \cup \{\mathcal{M}_1 \parallel \mathcal{M}_2\} - \{\mathcal{M}_1, \mathcal{M}_2\}$. We continue this process in A_1 and obtain the sets $A_2, A_3, \dots$, until the size of the set, named A_k ($k \geq 1$), is reduced to 2. By hypothesis, F, C satisfy

$$F = \bigcup_{C_n^+ \in \sim\Phi'} F_{C_n^+} - \left(\bigcup_{C_n^+ \in \sim\Phi'} F_{C_n^+} \cap \bigcup_{C_n^+ \in \sim\Phi'} C_{C_n^+} \right) \text{ and } C = \bigcup_{C_n^+ \in \sim\Phi'} C_{C_n^+}.$$

Let $A_k = \{\mathcal{M}[F, C], \mathcal{M}'[F', C']\}$, from the building procedure of $\mathcal{M}$ we know that A_k is well defined. Let $\mathcal{M}''[F'', C''] = \mathcal{M} \parallel \mathcal{M}'$, by Def. 5.6 no matter whether $\mathcal{M} \text{ obst } \mathcal{M}'$ or $\mathcal{M}' \text{ obst } \mathcal{M}$, we have

$$F'' = (F \cup F') - ((F \cup F') \cap (C \cup C')) \text{ and } C'' = C \cup C'.$$

Obviously $C'' = \bigcup_{C_n^+ \in \sim\Phi} C_{C_n^+}$. Let $U = \bigcup_{C_n^+ \in \sim\Phi} F_{C_n^+} \cup \bigcup_{C_n^+ \in \sim\Phi} C_{C_n^+}$ be the set of all clocks in the context, $\bar{X} = \{x \mid x \in U \text{ and } x \notin X\}$ be the complementary set of X , then $X - Y = X \cap \bar{Y}$. F'' can be computed as follows:

$$\begin{aligned} F'' &= X_3 - (X_3 \cap X_4) = X_3 \cap \overline{(X_3 \cap X_4)} = X_3 \cap (\overline{X_3} \cup \overline{X_4}) = X_3 \cap \overline{X_4} \\ &= ((X_1 \cap \overline{(X_1 \cap C)}) \cup F') \cap \overline{X_4} = (X_1 \cup F') \cap \overline{((X_1 \cap C) \cup F')} \cap \overline{X_4} \\ &= (X_1 \cup F') \cap \overline{((X_1 \cap C) \cap \overline{F'}) \cup X_4} \\ &= (X_1 \cup F') \cap \overline{((X_1 \cap \overline{F'}) \cup X_4) \cap (C \cup X_4)} \\ &= (X_1 \cup F') \cap \overline{((X_1 \cap \overline{F'}) \cup X_4) \cap X_4} \\ &= (X_1 \cup F') \cap \overline{X_4} = (X_1 \cup F') - ((X_1 \cup F') \cap X_4) \\ &= \bigcup_{C_n^+ \in \sim\Phi} F_{C_n^+} - \left(\bigcup_{C_n^+ \in \sim\Phi} F_{C_n^+} \cap \bigcup_{C_n^+ \in \sim\Phi} C_{C_n^+} \right). \end{aligned}$$

where $X_1 = \bigcup_{C_n^+ \in \sim\Phi'} F_{C_n^+}$, $X_2 = X_1 - (X_1 \cap C)$, $X_3 = X_2 \cup F'$ and $X_4 = C \cup C'$. $\square$

Lemma B.4. For a clock behaviour B and a set of well-defined clock definitions $\mathbf{D}$, we can construct a state x in the observer $\mathcal{M}[F, C] = \parallel_{C_n \in \mathbf{D}} \mathcal{M}^{C_n}$ which satisfies one of the following two cases:

- (a) either x is an initial state of $\mathcal{M}$ s.t. $\xi_{\mathcal{M}}(x) \cap F = B \cap F$, or
- (b) $\langle q, x \rangle$ is a transition starting from a state q s.t. $\xi_{\mathcal{M}}(x) \cap F = B \cap F$.

Moreover, it satisfies that

$$\text{for any clock } d \text{ in } \mathbf{D}, \text{ if } d \text{ is not pure, then } B \models d, \mathbf{D} \text{ iff } d \in \xi_{\mathcal{M}}(x).$$

Proof. We proceed the proof by induction on the steps of a procedure for constructing such a state x .

We begin by Step 1.

Step 1. Let $A = \{\mathcal{M}^{C_n}\}_{C_n \in \mathbf{D}}$, since $\mathbf{D}$ is well defined, we can find a clock definition $d_0 \triangleq e_0$ s.t. for all other $\mathcal{N} \in A - \{\mathcal{M}^{d_0 \triangleq e_0}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}^{d_0 \triangleq e_0}$ or $\mathcal{N} \uparrow \mathcal{M}^{d_0 \triangleq e_0}$ holds. Let $B_0 = B - C$, i.e., B_0 is obtained by removing all constrained clocks in B . In the observer $\mathcal{M}^{d_0 \triangleq e_0}[F_0, C_0]$ (where $F_0 = \mathcal{C}(e_0)$, $C_0 = \{d_0\}$ according to Prop. 5.3), for the clock behaviour $B_0 \cap F_0$, by Def. 5.12, there is a state s_0 satisfying the case (a) or (b) (let ξ_0 be the clock labelling function of $\mathcal{M}^{d_0 \triangleq e_0}$):

case (a): there exists an initial state s_0 s.t. $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$;

case (b): from the state $q|_{\mathcal{M}^{d_0 \triangleq e_0}}$, there exists a transition $\langle q|_{\mathcal{M}^{d_0 \triangleq e_0}}, s_0 \rangle$ s.t. s_0 s.t. $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$.

In both cases above, we have $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$.

Next we show that the following two propositions hold:

- (1) $\xi_0(s_0) \cap F_0 = B \cap F_0$.
- (2) For any $d \in C_0$, $B \models d, \mathbf{D}$ iff $d \in \xi_0(s_0)$.

For (1), it obviously holds because $C \cap F_0 = \emptyset$.

As for (2), since $C_0 = \{d_0\}$, we only need to prove $B \models d_0, \mathbf{D}$ iff $d_0 \in \xi_0(s_0)$. Without loss of generality, we assume that the clock expression e_0 is a Union expression, i.e., $e_0 = c_1 + c_2$ for some clocks c_1 and c_2 . Other cases can be proved in a similar way. If $B \models d_0, \mathbf{D}$, then by the semantics of $d_0 \triangleq e_1 + e_2$ in Table 2 we have $B \models c_1, \mathbf{D}$ or $B \models c_2, \mathbf{D}$. Assume $B \models c_1, \mathbf{D}$, since $\mathcal{M}^{d_0 \triangleq e_0}$ does not depend on any observers, c_1, c_2 must be pure clocks. So according to the semantics of $d_0 \triangleq c_1 + c_2$ in Table 2 we have $c_1 \in B$. Since c_1 is a pure clock, it can not be defined in other clock definitions, so we have $c_1 \notin C$, hence $c_1 \in B - C = B_0$. Because $c_1 \in B_0 \cap F_0$, so $c_1 \in \xi_0(s_0) \cap F_0 \subseteq \xi_0(s_0)$. According to the observer $\mathcal{M}^{d_0 \triangleq c_1 + c_2}$ in Table 4 it is easy to see that $d_0 \in \xi_0(s_0)$. On the other hand, if $d_0 \in \xi_0(s_0)$, by the observer $\mathcal{M}^{d_0 \triangleq c_1 + c_2}$ in Table 4 we know that either $c_1 \in \xi_0(s_0)$ or $c_2 \in \xi_0(s_0)$ holds. Assume $c_1 \in \xi_0(s_0)$, because $c_1 \in F_0$, we have $c_1 \in \xi_0(s_0) \cap F_0 = B_0 \cap F_0 \subseteq B$. Since c_1 is pure, we have $B \models c_1, \mathbf{D}$, therefore $B \models d_0, \mathbf{D}$ by the semantics of $d_0 \triangleq c_1 + c_2$ in Table 2.

Let $B_1 = B_0 \cup \xi_0(s_0)$, $\mathbf{D}_1 = \mathbf{D} - \{d_0 \triangleq e_0\}$, $\mathcal{M}_0 = \mathcal{M}^{d_0 \triangleq e_0}$ and $A_1 = (A \cup \mathcal{M}_0) - \{\mathcal{M}^{d_0 \triangleq e_0}\}$. Obviously $\mathbf{D}_1$ and A_1 are well defined.

[the end of Step 1]

Now suppose that after Step n ($n \geq 1$), we obtain a sequence of clock definitions $d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_{n-1} \triangleq e_{n-1}$, a sequence of sets of clock definitions $\mathbf{D}_1, \dots, \mathbf{D}_n$ where $\mathbf{D}_k = \mathbf{D} - \{d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_{k-1} \triangleq e_{k-1}\}$ ($1 \leq k \leq n$), a sequence of observers $\mathcal{M}_0, \dots, \mathcal{M}_{n-1}$ where $\mathcal{M}_k[F_k, C_k] = \parallel_{i=0}^k \mathcal{M}^{d_i \triangleq e_i}$ ($0 \leq k \leq n-1$), a sequence of states $s_0, \dots, s_{n-1}$, a sequence of clock behaviours $B_1, \dots, B_n$ where $B_k = B_0 \cup \xi_{k-1}(s_{k-1})$ (where ξ_{k-1} is the labelling function of the observer $\mathcal{M}_{k-1}$, $1 \leq k \leq n$), and a sequence of sets of observers $A_1, \dots, A_n$ where $A_k = (A \cup \mathcal{M}_{k-1}) - \bigcup_{i=0}^{k-1} \{\mathcal{M}^{d_i \triangleq e_i}\}$ ($1 \leq k \leq n$). They satisfy that

- (A) $\mathbf{D}_k$ and A_k are well defined ($1 \leq k \leq n$);
- (B) $F_k = \bigcup_{i=0}^k F_i - (\bigcup_{i=0}^k F_i \cap \bigcup_{i=0}^k C_i)$ and $C_k = \bigcup_{i=0}^k C_i$, where $0 \leq k \leq n-1$, F_i, C_i are the sets of free clocks and cosntrained clocks of the observer $\mathcal{M}_i$ respectively obtained at Step $i+1$;
- (C) for any $\mathcal{N} \in A_k - \{\mathcal{M}_{k-1}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}_{k-1}$ or $\mathcal{N} \uparrow \mathcal{M}_{k-1}$ holds ($1 \leq k \leq n$).

In addition, it satisfies that

- (I) s_k is a state of $\mathcal{M}_k$ s.t. $\xi_{\mathcal{M}_k}(s_k) \cap F_k = B \cap F_k$ holds, where $0 \leq k \leq n-1$.
- (II) For any $d \in C_k$, $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}_k}(s_k)$, where $0 \leq k \leq n-1$.

Clearly, in Step 1, all conditions (A), (B), (C), (I), (II) hold, now we construct Step $n+1$ from the assumption of Step n .

Step $n+1$. Since $\mathbf{D}_n$ is well defined, we can select a clock definition $d_n \triangleq e_n$ in $\mathbf{D}_n$ s.t. for any $\mathcal{N} \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}^{d_n \triangleq e_n}$ or $\mathcal{N} \uparrow \mathcal{M}^{d_n \triangleq e_n}$ holds. From the condition (C) above it is easy to see that in the set A_n , $\mathcal{M}^{d_n \triangleq e_n} \rightsquigarrow^o \mathcal{M}_{n-1}$ or $\mathcal{M}^{d_n \triangleq e_n} \uparrow \mathcal{M}_{n-1}$ holds. This is because if there exists a $\mathcal{N}'$ s.t. $\mathcal{M}^{d_n \triangleq e_n} \rightsquigarrow^+ \mathcal{N}' \rightsquigarrow^+ \mathcal{M}_{n-1}$, then we immediately get that $\mathcal{N}'$ violates the way of how $d_n \triangleq e_n$ is selected. By Lemma B.1 we know $\mathcal{M}^{d_n \triangleq e_n} \text{ obst } \mathcal{M}_{n-1}$. In the observer $\mathcal{M}^{d_n \triangleq e_n}[F', C']$ (where $F' = \mathcal{C}(e_n)$, $C' = \{d_n\}$), similar to the Step 1, for the behaviour $B_n \cap F'$, by Def. 5.12, there is a state s' satisfying the case (a) or (b), and no matter it satisfies the case (a) or (b), we both have

$$\xi'(s') \cap F' = B_n \cap F'$$

(let ξ' be the labelling function of $\mathcal{M}^{d_n \triangleq e_n}$).

Let $\mathcal{M}_n[F_n, C_n] = \mathcal{M}^{d_n \triangleq e_n} \parallel \mathcal{M}_{n-1}$, by Prop. 5.6 we see that $F_n = (F' \cup F_{n-1}) - (C' \cup C_{n-1})$ and $C_n = C' \cup C_{n-1}$. Let $s_n = \langle s', s_{n-1} \rangle$. We now prove that the conditions (I) and (II) hold for $k = n$, i.e., the following two propositions hold:

- (1) s_n is a state of $\mathcal{M}_n$ s.t. $\xi_{\mathcal{M}_n}(s_n) \cap F_n = B \cap F_n$ holds.
- (2) For any $d \in C_n$, $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}_n}(s_n)$.

For (1), we first show that s_n is a state of $\mathcal{M}_n$. Let Clk' , Clk_{n-1} be the set of clocks of the observers $\mathcal{M}^{d_n \triangleq e_n}$ and $\mathcal{M}_{n-1}$ respectively. By Def. 5.4 we need to show $\xi_{n-1}(s_{n-1}) \cap Clk' = \xi'(s') \cap Clk_{n-1}$. On one hand, for any $t \in \xi_{n-1}(s_{n-1}) \cap Clk'$, since $\mathcal{M}^{d_n \triangleq e_n} \text{ obst } \mathcal{M}_{n-1}$, we have $t \in \xi_{n-1}(s_{n-1}) \cap Clk' \subseteq Clk_{n-1} \cap Clk' \subseteq F'$. From $t \in \xi_{n-1}(s_{n-1})$ we also have $t \in B_n$ because $B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. Thus $t \in B_n \cap F' = \xi'(s') \cap F'$. Therefore $t \in \xi'(s')$. From $t \in \xi_{n-1}(s_{n-1}) \subseteq Clk_{n-1}$ we obtain $t \in \xi'(s') \cap Clk_{n-1}$. On the other hand, for any $t \in \xi'(s') \cap Clk_{n-1}$, since $t \in \xi'(s') \cap Clk_{n-1} \subseteq Clk' \cap Clk_{n-1} \subseteq F'$, we get $t \in \xi'(s') \cap F' = B_n \cap F'$. Thus $t \in B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. If $t \in B_0$, then $t \notin C$, from $t \in Clk_{n-1}$ we get that $t \in F_{n-1}$. From the fact that $B_0 \subseteq B$ we get $t \in \xi_{n-1}(s_{n-1})$ because $\xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$. From $t \in \xi'(s') \subseteq Clk'$ we obtain $t \in \xi_{n-1}(s_{n-1}) \cap Clk'$.

Now we show that $\xi_{\mathcal{M}_n}(s_n) \cap F_n = B \cap F_n$. On one hand, for any $t \in \xi_{\mathcal{M}_n}(s_n) \cap F_n = (\xi_{n-1}(s_{n-1}) \cup \xi'(s')) \cap F_n$, we consider two situations: a) If $t \in \xi_{n-1}(s_{n-1})$, from $t \in F_n$, $t \in \xi_{n-1}(s_{n-1}) \subseteq Clk_{n-1}$ and the fact that $F_n = (F' \cup F_{n-1}) - (C' \cup C_{n-1})$, we can get $t \in F_{n-1}$. So $t \in \xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$, hence $t \in B$; b) if $t \in \xi'(s')$, then $t \in \xi'(s') \subseteq Clk'$. Again from $t \in F_n$ we have $t \in F'$. Therefore $t \in \xi'(s') \cap F' = B_n \cap F'$. So $t \in B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. Since $t \notin \xi_{n-1}(s_{n-1})$, we get that $t \in B_0$. From $B_0 = B - C$ we immediately obtain that $t \in B$. On the other hand, for any $t \in B \cap F_n$, from $t \in F_n$ and the fact that for any $\mathcal{N} \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, $\mathcal{M}^{d_n \triangleq e_n} \not\sim \mathcal{N}$ and $\mathcal{M}_{n-1} \not\sim \mathcal{N}$ hold, we can have $t \notin C$. First, since $t \in F_n$, $t \notin C' \cup C_{n-1}$. If $t \in C_s$ for some observer $\mathcal{N}_s[F_s, C_s] \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, we have $t \in C_s \cap F' \subseteq C_s \cap Clk'$ or $t \in C_s \cap F_{n-1} \subseteq C_s \cap Clk_{n-1}$, which violates the fact that $\mathcal{M}^{d_n \triangleq e_n} \not\sim \mathcal{N}_s$ or $\mathcal{M}_{n-1} \not\sim \mathcal{N}_s$. From $B_0 = B - C$ we get $t \in B_0 \subseteq B_n$. We consider two situations: c) If $t \in F'$, then $t \in B_n \cap F'$, so $t \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$; d) if $t \in F_{n-1}$, then $t \in B \cap F_{n-1}$, therefore $t \in \xi_{n-1}(s_{n-1}) \subseteq \xi_{\mathcal{M}_n}(s_n)$.

For (2), we only need to prove that for any $d \in C'$ (which is d_n), $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}_n}(s_n)$ since for any $d' \in C_{n-1}$, by the condition (II) in the assumption of Step n , we already have $B \models d', \mathbf{D}$ iff $d' \in \xi_{n-1}(s_{n-1})$. Since $\xi_{n-1}(s_{n-1}) \subseteq \xi_{n-1}(s_{n-1}) \cup \xi'(s') = \xi_{\mathcal{M}_n}(s_n)$, we have $d' \in \xi_{\mathcal{M}_n}(s_n)$.

Without loss of generality we assume that the clock expression e_n is a Union expression, i.e., $e_n = c_1 + c_2$ for some clocks c_1, c_2 . Other cases are similar and are omitted here. On one hand, if $B \models d_n, \mathbf{D}$, then by the expression e_n we have $B \models c_1, \mathbf{D}$ or $B \models c_2, \mathbf{D}$ holds. Assume $B \models c_1, \mathbf{D}$, we consider two cases when c_1 is a pure clock or not: a) If c_1 is pure, by the semantics of clock definitions in Table 2, $c_1 \in B$. Since c_1 can not be defined in other clock definitions, we have $c_1 \notin C$. Therefore $c_1 \in B - C$, hence $c_1 \in B_0 \subseteq B_n$. Since $c_1 \in F'$, we have $c_1 \in B_n \cap F' = \xi'(s') \cap F' \subseteq \xi'(s')$. According to the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4, we get $d_n \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$. b) If c_1 is not pure, it must be defined in some clock definition(s). So $c_1 \in C$. It is not hard to see that it must be $c_1 \in C_{n-1}$ because except for $\mathcal{M}_{n-1}$, $\mathcal{M}^{d_n \triangleq e_n} \not\sim \mathcal{N}$ for any other $\mathcal{N} \in A_n$. Because $B \models c_1, \mathbf{D}$, so by the condition (II) in the assumption of Step n above we have $c_1 \in \xi_{n-1}(s_{n-1})$. Since $\xi'(s') \cap Clk_{n-1} = \xi_{n-1}(s_{n-1}) \cap Clk'$ as proved above, we know that $c_1 \in \xi'(s')$. Again by the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4 we have $d_n \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$.

On the other hand, suppose $d_n \in \xi_{\mathcal{M}_n}(s_n)$, we have $d_n \in \xi'(s') \cup \xi_{n-1}(s_{n-1})$. It is not hard to see that $d_n \in \xi'(s')$ because $\mathcal{M}_{n-1} \not\sim \mathcal{M}^{d_n \triangleq e_n}$. By the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4 we know that $c_1 \in \xi'(s')$ or $c_2 \in \xi'(s')$ holds. Assume $c_1 \in \xi'(s')$, since $c_1 \in \xi'(s') \cap F' = B_n \cap F'$, we have $c_1 \in B_n$. As above, we consider two cases when c_1 is a pure clock or not: a) If c_1 is pure, $c_1 \notin C$. Since $c_1 \in B_n$, $c_1 \in B_0$ or $c_1 \in \xi_{n-1}(s_{n-1})$ holds. If $c_1 \in B_0$, we immediately have $c_1 \in B$. If $c_1 \in \xi_{n-1}(s_{n-1})$, it must be that $c_1 \in F_{n-1}$. So $c_1 \in \xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$, hence $c_1 \in B$. Anyhow we have $c_1 \in B$. Since c_1 is pure, $B \models c_1, \mathbf{D}$. So $B \models d_n, \mathbf{D}$ according to the semantics of $d_n \triangleq c_1 + c_2$ in Table 2. b) If c_1 is not pure, for the same reason stated above, it must be that $c_1 \in C_{n-1}$. Because $c_1 \in B_n$, $c_1 \in \xi_{n-1}(s_{n-1})$ holds since $B_0 \cap C = \emptyset$. By the condition (II) of Step n , we immediately have $B \models c_1, \mathbf{D}$. Again by the semantics of $d_n \triangleq c_1 + c_2$ in Table 2 we get $B \models d_n, \mathbf{D}$.

By now we have proved the conditions (1) and (2) above.

Let $B_{n+1} = B_n \cup \xi_n(s_n)$, since $\xi_n(s_n) \supseteq \xi_{n-1}(s_{n-1})$, we have $B_{n+1} = B_0 \cup \xi_{n-1}(s_{n-1}) \cup \xi_n(s_n) = B_0 \cup \xi_n(s_n)$. Let $\mathbf{D}_{n+1} = \mathbf{D}_n - \{d_n \triangleq e_n\} = \mathbf{D} - \{d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_n \triangleq e_n\}$, $A_{n+1} = (A_n \cup \{\mathcal{M}_n\}) - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\} = (A \cup \{\mathcal{M}_n\}) - \bigcup_{i=0}^n \{\mathcal{M}^{d_i \triangleq e_i}\}$.

We now show that the conditions (A), (B), (C) shown above in the assumption of Step n also hold for

Step $n + 1$. Obviously $\mathbf{D}_{n+1}$ is well defined. By Lemma B.3 A_{n+1} is well defined. By Prop. 5.6 and Thm. 5.1 it is easy to see that $F_n = \bigcup_{i=0}^{n-1} F_i - (\bigcup_{i=0}^{n-1} F_i \cap \bigcup_{i=0}^k C_i)$ and $C_n = \bigcup_{i=0}^{n-1} C_i$. To show that the condition (C) also holds for Step $n + 1$, we observe that for any $\mathcal{N} \in A_{n+1} - \{\mathcal{M}_n\}$, there is $\mathcal{M}_n \not\sim^+ \mathcal{N}$. If not, suppose $\mathcal{M}_n \rightsquigarrow \mathcal{N}_0$ for some $\mathcal{N}_0$, then $C_{\mathcal{N}_0} \cap \text{Clk}_{\mathcal{M}_n} \neq \emptyset$, so $C_{\mathcal{N}_0} \cap (\text{Clk}' \cup \text{Clk}_{n-1}) \neq \emptyset$. Then we have $\mathcal{M}^{d_n \triangleleft e_n} \rightsquigarrow \mathcal{N}_0$ or $\mathcal{M}_{n-1} \rightsquigarrow \mathcal{N}_0$, which contradicts the fact that $\mathcal{M}^{d_n \triangleleft e_n} \not\rightsquigarrow \mathcal{N}_0$ and $\mathcal{M}_{n-1} \not\rightsquigarrow \mathcal{N}_0$.

[the end of Step $n + 1$]

Following the procedure of constructing the state x shown above, suppose $|\mathbf{D}| = l$, then after l steps, we can get $\mathcal{M}[F, C] = \mathcal{M}_l[F_l, C_l] = \bigcup_{i=0}^{l-1} \mathcal{M}^{d_i \triangleleft e_i}$ and we can construct a unique state $x = s_l = \langle s_0, s_1, \dots, s_{l-1} \rangle$ of $\mathcal{M}$ with $\xi_{\mathcal{M}}(x) = \xi_0(s_0) \cup \dots \cup \xi_{l-1}(s_{l-1})$, which satisfies the case (a) or (b). $C = C_l = \bigcup_{i=0}^{l-1} C_{\mathcal{M}^{d_i \triangleleft e_i}}$ is exactly the set of clocks that are not pure in $\mathbf{D}$. For any $d \in C$, we have $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}}(x)$.

□

The proof of Prop. 5.10 By Def. 5.16, the corresponding OTS of $\mathcal{M}$ is $\mathcal{M}^{\mathcal{D}} = \langle \Pi^{\mathcal{D}}, Q, I, \theta^{\mathcal{D}}, \mathcal{T} \rangle$ where $\Pi^{\mathcal{D}} = \Pi \cup \{P_c \mid c \in \text{Clk}\}$ and for any $q \in Q$, $\theta^{\mathcal{D}}(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}$. Let $\mathcal{M}^{-\phi} = \mathfrak{B}((\neg\phi)^{\mathcal{D}})^{-\mathcal{D}} = \langle \Pi_1, \text{Clk}_1, Q_1, I_1, \theta_1, \xi_1, \mathcal{T}_1, \mathcal{E}_1 \rangle$, by Def. 5.16 we know that its corresponding Büchi automaton is $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) = \langle \Pi_1^{\mathcal{D}}, Q_1, I_1, \theta_1^{\mathcal{D}}, \mathcal{T}_1, \mathcal{E}_1 \rangle$, where $\Pi_1^{\mathcal{D}} = \Pi_1 \cup \{P_c \mid c \in \text{Clk}_1\}$ and for any $q \in Q$, $\theta_1^{\mathcal{D}}(q) = \theta_1 \cup \{P_c \mid c \in \xi_1(q)\}$. According to the observation relation between an OTS and the Büchi automaton of an LTL formula, we have $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) \text{ obs } \mathcal{M}^{\mathcal{D}}$. Since $(\neg\phi)^{\mathcal{D}} = \neg\phi^{\mathcal{D}}$, $\mathfrak{B}(\neg\phi^{\mathcal{D}}) \text{ obs } \mathcal{M}^{\mathcal{D}}$.

To prove $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}$, we only need to show that for any trace $\pi \in \text{Trace}(\mathcal{M})$,

- (1) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}^{-\phi}$ s.t. $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$;
- (2) for any transition $\langle q, q' \rangle$ of the trace π and any state x of $\mathcal{M}^{-\phi}$ s.t. $\langle q, x \rangle$ is a state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$, there exists a transition $\langle \langle q, x \rangle, s \rangle$ of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$.

(1): By $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) \text{ obs } \mathcal{M}^{\mathcal{D}}$, we know that there is an initial state x_0 of $\mathfrak{B}((\neg\phi)^{\mathcal{D}})$ s.t. $\theta^{\mathcal{D}}(\pi[0]) \cap \Pi_1^{\mathcal{D}} = \theta_1^{\mathcal{D}}(x_0) \cap \Pi^{\mathcal{D}}$ holds. Let $C_1 = \{P_c \mid c \in \xi(\pi[0])\}$, $C_2 = \{P_c \mid c \in \text{Clk}_1\}$, $C_3 = \{P_c \mid c \in \xi_1(x_0)\}$ and $C_4 = \{P_c \mid c \in \text{Clk}\}$. By the fact that each P_c ($c \in \text{Clk} \cup \text{Clk}_1$) is a special atomic proposition and is different from all other general atomic propositions in Π and Π_1 , we have

$$\begin{aligned} \theta^{\mathcal{D}}(\pi[0]) \cap \Pi_1^{\mathcal{D}} &= (\theta(\pi[0]) \cup C_1) \cap (\Pi_1(x_0) \cup C_2) \\ &= (\theta(\pi[0]) \cap \Pi_1) \cup (C_1 \cap C_2), \\ \theta_1^{\mathcal{D}}(x_0) \cap \Pi^{\mathcal{D}} &= (\theta_1(x_0) \cup C_3) \cap (\Pi \cup C_4) \\ &= (\theta_1(x_0) \cap \Pi) \cup (C_3 \cap C_4). \end{aligned}$$

Therefore, we get $(\theta(\pi[0]) \cap \Pi_1) = (\theta_1(x_0) \cap \Pi)$ and $(C_1 \cap C_2) = (C_3 \cap C_4)$ separately. By $(C_1 \cap C_2) = (C_3 \cap C_4)$ and the one-to-one correspondence between a clock c and its corresponding atomic proposition P_c , there is $\xi(\pi[0]) \cap \text{Clk}_1 = \xi_1(x_0) \cap \text{Clk}$. Hence, $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$.

(2): From $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) \text{ obs } \mathcal{M}^{\mathcal{D}}$, we know that there is a state $\langle q', x' \rangle$ of $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) \parallel \mathcal{M}^{\mathcal{D}}$ s.t. $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathfrak{B}((\neg\phi)^{\mathcal{D}}) \parallel \mathcal{M}^{\mathcal{D}}$. So $\theta^{\mathcal{D}}(x') \cap \Pi_1^{\mathcal{D}} = \theta_1^{\mathcal{D}}(q') \cap \Pi^{\mathcal{D}}$. Let $C_1 = \{P_c \mid c \in \xi(s_2)\}$, $C_2 = \{P_c \mid c \in \text{Clk}_1\}$, $C_3 = \{P_c \mid c \in \xi_1(q')\}$ and $C_4 = \{P_c \mid c \in \text{Clk}\}$. Similar as above in (1), we can prove that both $\theta(x') \cap \Pi_1 = \theta_1(q') \cap \Pi$ and $\xi(x') \cap \text{Clk}_1 = \xi_1(q') \cap \text{Clk}$ hold. Therefore $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$. □

C. CTSA Semantics of Clock Definitions

Table 4 shows the semantics of clock definitions in CCSL.

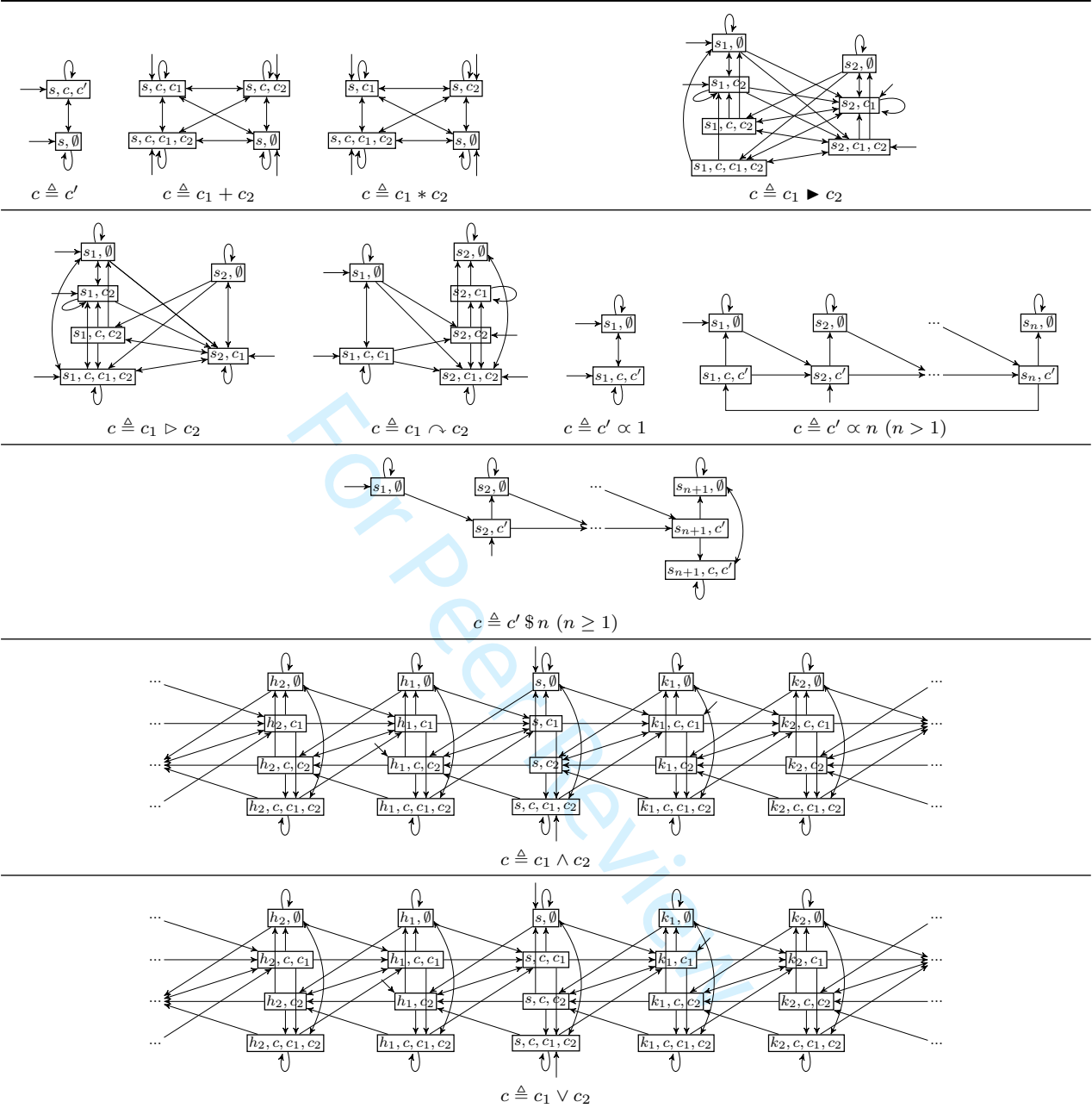


Table 4. CTSA semantics of clock definitions